Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Contents

Reference

Quick Reference

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 1 - Meet the X-Touch

Why "Project XTC"?

XTC stands for X-Touch Companion. This guide is designed to sit alongside the official Behringer, Bitwig and DrivenByMoss documentation, bringing everything together into one practical handbook. If the name raises a knowing smile, that's perfectly fine - but throughout this book, XTC always means X-Touch Companion.

If you've bought a Behringer X-Touch, you've probably already discovered two things. First, it feels like a serious piece of studio equipment. Second, learning to use it effectively with Bitwig can be surprisingly difficult.

The Behringer manual explains the hardware. Bitwig explains the DAW. DrivenByMoss explains the extension. What has been missing is a guide that explains how all three work together. That is the purpose of Project XTC.

Who This Book Is For

This guide assumes you already know the basics of using a computer and Bitwig Studio. It does not assume any previous knowledge of MCU, MIDI protocols or control surfaces. Every important idea is introduced only when it becomes useful.

What the X-Touch Is

The X-Touch is a hardware control surface. It does not process audio or replace Bitwig. It provides tactile control over track selection, transport, mixing, PLUG-IN, automation and much more.

The Four Principles

1. Bitwig is in charge.
2. The selected track matters.
3. Modes change meanings.
4. Feedback is as important as input.

Before You Continue...

Don't worry about memorising every button yet. In the next chapter we'll take a guided tour of the X-Touch so every control has a name and a purpose before we start using it.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 2 - A Tour of the Hardware

Before learning individual features, it's worth becoming familiar with the physical layout of the X-Touch. You don't need to memorise every control. The goal is simply to help you recognise where things are, so later chapters can refer to them naturally.

The Eight Channel Strips

The heart of the X-Touch consists of eight identical channel strips. Each strip includes a motor fader, a touch-sensitive V-Pot with an LED ring, a scribble strip display, and buttons such as SELECT, SOLO, MUTE and REC. Because every strip is identical, once you understand one, you understand them all.

The Master Section

To the right of the eight channels is the master section. It includes the master fader and additional navigation controls.

Assignment Buttons

The TRACK, SEND, PAN, PLUG-IN, EQ and INSTRUMENT buttons change what the V-Pots control.

GLOBAL VIEW

The USER button in GLOBAL VIEW opens Browser Mode when using DrivenByMoss.

Reality Check

If the number of buttons feels intimidating, remember that you rarely use them all at once.

Before You Continue...

The next chapter introduces the mental model that makes the X-Touch predictable.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 3 - The Mental Model

Most difficulties people experience with the X-Touch come from thinking of it as a mixer with fixed controls. It isn't. The X-Touch is better understood as a window into Bitwig.

Bitwig Is Always in Charge

The X-Touch follows Bitwig's current state. If Bitwig changes, the controller reflects that change.

Selection Comes First

Before adjusting anything ask: What is currently selected? Successful workflows nearly always begin with SELECT.

Modes Change Meaning

The same controls perform different jobs in different contexts. Understanding modes is more important than memorising combinations.

Always Read the Feedback

Motor faders, LED rings and scribble strips constantly tell you what Bitwig is doing.

A Simple Workflow

SELECT -> Observe -> Adjust -> Listen

Notice that adjustment comes third, not first.

> Reality Check
> If the X-Touch behaves unexpectedly, check the selected track, current mode and scribble strips first.

Field Note

The best X-Touch users don't memorise hundreds of functions. They ask, "What is Bitwig telling me right now?"

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 4 - Banks and Channels

One of the first things to understand about the X-Touch is that it always provides eight physical channel strips, regardless of how many tracks exist in your Bitwig project.

If your project contains more than eight tracks, the X-Touch simply changes which eight tracks are currently under your control. This is where the ideas of channels and banks become important.

Channels

Each motor fader represents a channel.

In the simplest case:

X-Touch	Bitwig
Channel 1	Track 1
Channel 2	Track 2
Channel 3	Track 3
...	...
Channel 8	Track 8

If your project contains eight tracks or fewer, this relationship never changes.

The X-Touch behaves very much like a traditional hardware mixer.

Banks

Once your project grows beyond eight tracks, the controller cannot display every track at once.

Instead, it displays a bank of eight consecutive tracks.

The first bank contains:

• Tracks 1–8

The second bank contains:

• Tracks 9–16

The third bank contains:

• Tracks 17–24

and so on.

Changing bank simply changes which group of eight tracks is currently mapped onto the eight physical channel strips.

> Field Note
> The hardware never changes.
>
> Only the relationship between the hardware and the Bitwig project changes.

BANK Buttons

The BANK ◀ and BANK ▶ buttons move one complete bank at a time.

For example, if you are currently controlling Tracks 1–8:

• Press BANK ▶ once.
• The scribble strips now display Tracks 9–16.
• The motor faders immediately move to the stored levels for those tracks.

Press BANK ◀ to return to the previous bank.

Watching the motor faders reposition themselves is one of the most satisfying demonstrations of what a motorised control surface can do.

CHANNEL Buttons

The CHANNEL ◀ and CHANNEL ▶ buttons work differently.

Instead of moving by eight tracks, they move by one track.

Imagine you are currently controlling:

Fader	Track
1	Track 1
2	Track 2
3	Track 3
4	Track 4

Fader	Track
5	Track 5
6	Track 6
7	Track 7
8	Track 8

Press CHANNEL ▶ once.

Now the mapping becomes:

Fader	Track
1	Track 2
2	Track 3
3	Track 4
4	Track 5
5	Track 6
6	Track 7
7	Track 8
8	Track 9

Every channel shifts along by one track.

This makes it easy to bring a particular track into view without jumping an entire bank.

Reality Check
If a track seems to have "disappeared", it has probably moved outside the currently visible bank.

Use the BANK or CHANNEL buttons to bring it back into view.

Reading the Scribble Strips

Whenever you change bank or channel offset, the scribble strips update immediately.

Experienced users naturally glance at the displays before touching a fader.

This simple habit avoids accidental edits to the wrong track.

Why This Matters

Understanding banks and channels makes the rest of the X-Touch much easier to understand.

Whether you are controlling volume, sends, plug-ins or automation, the same principle always applies:

The X-Touch controls the tracks that are currently visible.

Everything else in the controller builds upon this idea.

> Exercise
> Create a Bitwig project containing at least twelve tracks.
>
> Use the BANK ▶ and BANK ◀ buttons to move between the first and second banks.
>
> Then experiment with the CHANNEL ▶ button and watch how the scribble strips and motor faders update.
>
> Pay particular attention to which track appears on Channel 1 after each movement.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 5 - Modes

The X-Touch is often described as having "lots of buttons". That is true, but it is only half the story.

The more important fact is that many of those buttons do different jobs at different times.

Understanding modes is the key to understanding the X-Touch.

Once this idea becomes familiar, the controller feels logical rather than mysterious.

What Is a Mode?

A mode changes the meaning of one or more controls.

The physical hardware never changes.

Instead, the X-Touch changes how it interprets your button presses, encoder turns and fader movements.

Think of the controller as speaking several different "languages".

Each mode uses the same hardware to perform a different set of tasks.

> Field Note
> A common mistake is to think that every button has exactly one purpose.
>
> On the X-Touch, many controls are intentionally multifunctional.

Figure 5.1 illustrates this idea. Although the hardware remains exactly the same, selecting a different mode changes the role of the controls rather than the controls themselves.

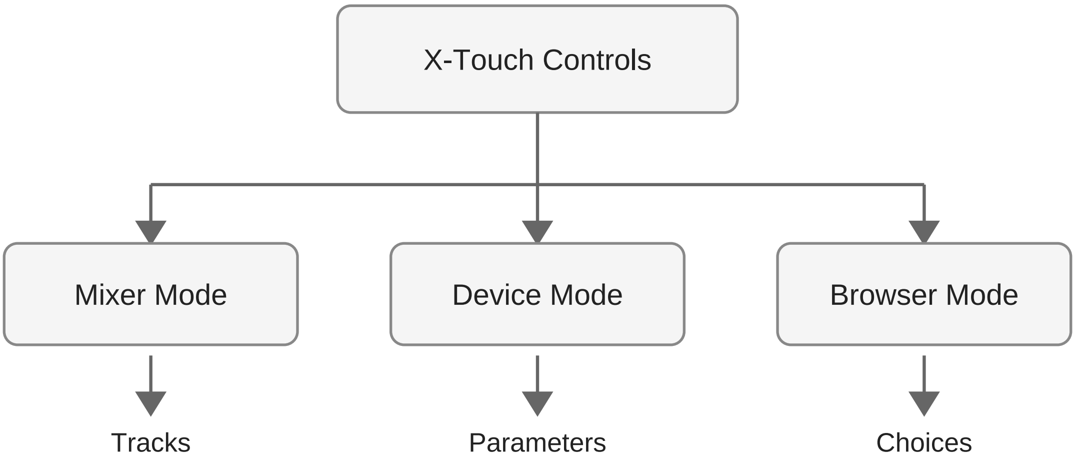


Figure 5.1 — The X-Touch uses the same physical controls in many different operating modes.

The Current Mode Is Always Visible

Fortunately, the X-Touch rarely leaves you guessing.

The scribble strips, LEDs and button illumination usually indicate the current operating mode.

Before pressing a control, take a quick look at the feedback the controller is already providing.

Experienced users develop this habit very quickly.

The Primary Modes

DrivenByMoss makes extensive use of the X-Touch's mode system.

The most frequently used modes include:

- TRACK
- SEND
- PAN/SURROUND
- PLUG-IN
- EQ
- INSTRUMENT
- USER

Each mode changes what the rotary encoders control and, in some cases, what appears on the scribble strips.

The important point is that these are not different layouts of the controller. They are simply different interpretations of the same hardware.

Throughout the remainder of this book we shall examine each of these modes in detail.

TRACK Mode

TRACK mode is the mode most users spend the majority of their time in.

The controller focuses on the Bitwig tracks themselves.

Typical operations include:

- selecting tracks
- adjusting volume
- muting
- soloing
- recording
- automation

Think of TRACK mode as the controller's "home".

Whenever you become unsure where you are, returning to TRACK mode is often a sensible starting point.

SEND Mode

SEND mode changes the rotary encoders so that they adjust send levels instead of pan.

Instead of controlling the stereo position of each track, the encoders now control how much of the signal is sent to an effects bus such as a reverb or delay.

Exactly which send is being adjusted depends upon the current Bitwig project and the selected send.

> Reality Check
> If turning an encoder suddenly changes a reverb or delay send instead of the pan position, the controller is almost certainly in SEND mode.

PAN/SURROUND Mode

PAN/SURROUND mode returns the rotary encoders to controlling the stereo (or surround) position of each track.

For most stereo projects this simply means moving sounds left or right within the mix.

PLUG-IN Mode

PLUG-IN mode is one of the X-Touch's most powerful capabilities.

Rather than controlling the mixer, the encoders are assigned to parameters exposed by the currently selected Bitwig device.

The scribble strips display parameter names, allowing the hardware to become a tactile extension of the software interface.

Later chapters explore this workflow in depth.

INSTRUMENT Mode

INSTRUMENT mode provides direct access to software instrument parameters.

Depending upon the selected Bitwig device, the encoders may control filter cutoff, resonance, envelopes, oscillator settings or many other synthesis controls.

The exact assignments depend entirely upon the instrument currently in focus.

USER Mode

USER mode deserves special attention.

Unlike the other operating modes, USER mode is not tied to a particular mixer function.

DrivenByMoss uses USER mode to expose additional Bitwig features.

For example, pressing USER immediately opens Bitwig's Browser. The scribble strips are repurposed to display browser information, allowing sounds and presets to be selected directly from the X-Touch.

> Field Note
> Because USER mode is completely software-defined, it is capable of doing far more than its name might suggest.
>
> Many users overlook it at first, yet it quickly becomes one of the most frequently used buttons on the controller.

Modes Are Temporary

Changing mode does not permanently alter the controller.

It simply changes what the controls do at that moment.

You are free to move between modes whenever your workflow requires.

The motor faders, scribble strips and LEDs update automatically to reflect the current context.

Thinking in Modes

One of the biggest steps towards mastering the X-Touch is learning to ask a simple question:

"What mode am I in?"

Many apparent "problems" disappear the moment you realise that the controller is behaving exactly as expected—it is simply operating in a different mode.

Once this becomes second nature, the X-Touch feels far less complicated than it first appears.

Exercise
Open any Bitwig project.

Press each of the available mode buttons in turn and observe how the scribble strips, encoder assignments and button illumination change.

Do not try to memorise every function immediately.

Instead, become comfortable with the idea that the X-Touch continually adapts its controls to the task at hand.

Recognising the current mode is far more valuable than memorising individual button assignments.

Chapter 6 - The SELECT Button

The SELECT button is arguably the most important button on the X-Touch.

New users often assume that its only purpose is to highlight a track. In fact, SELECT determines where the controller's attention is focused. Many of the X-Touch's most powerful features depend upon this simple action.

Once you understand SELECT, the behaviour of Device Mode, Browser Mode and many other functions becomes much easier to understand.

Selecting a Track

Each channel strip has its own SELECT button.

Pressing one tells the X-Touch:

"This is the track I want to work with."

Nothing is changed simply by selecting a track.

Instead, you are telling the controller where future actions should take place.

Think of SELECT as moving your attention rather than changing your project.

> Field Note
> The X-Touch always has a current focus.
>
> Many commands operate on the currently selected track, so selecting the correct track before performing another action quickly becomes second nature.

Focus Rather Than Selection

Although the button is labelled SELECT, it is often more helpful to think of it as a FOCUS button.

When you select a different track:

• Device Mode follows that track.
• Browser Mode follows that track.

• Plug-in parameters follow that track.
• Instrument controls follow that track.

The selected track becomes the centre of attention for the controller.

This idea of focus appears repeatedly throughout the rest of the book.

Watching the Controller Respond

One of the strengths of DrivenByMoss is the excellent feedback it provides.

After pressing a SELECT button you may notice changes to:

• the scribble strips
• encoder assignments
• parameter names
• button illumination
• device information

These are not separate operations.

They are all consequences of changing the controller's focus.

The X-Touch immediately adapts to the newly selected track.

SELECT Comes First

As you become more familiar with the controller, a natural workflow begins to emerge:

1. Select the track.
2. Choose the required mode.
3. Perform the adjustment.

Whether you are controlling a plug-in, adjusting sends or browsing presets, the first step is almost always the same.

Press SELECT.

> Reality Check
> If the X-Touch appears to be controlling the "wrong" plug-in or instrument, the most likely explanation is that a different track is currently selected.
>
> Before assuming something is broken, check which SELECT button is illuminated.

SELECT and the Motor Faders

A common misunderstanding is that selecting a track should cause the motor faders to move.

They do not.

The motor faders represent the currently visible bank of channels.

SELECT changes the controller's focus, not the mixer layout.

This distinction is important.

The faders show where the tracks are.

SELECT shows which one you are working on.

SELECT in Device Mode

Device Mode always follows the selected track.

Imagine a project containing:

- drums
- bass
- piano
- synthesizer
- vocals

Each track contains different devices.

Simply pressing SELECT on another channel instantly changes which devices the controller is connected to.

The rotary encoders, parameter names and scribble strips all update automatically.

No additional configuration is required.

SELECT in Browser Mode

Browser Mode behaves in exactly the same way.

The currently selected track determines where new devices, presets or samples will be inserted.

This makes SELECT an important part of everyday workflow.

Rather than opening the Browser and then deciding where something should go, experienced users select the destination track first.

Developing Good Habits

The most effective X-Touch users develop a very simple habit:

SELECT → Observe → Adjust

Notice that adjustment comes last.

First establish the correct focus.

Then observe what the controller is telling you.

Only then make changes.

This small change in thinking prevents many mistakes.

> Field Note
> One of the recurring themes throughout this book is that the X-Touch is constantly providing feedback.
>
> The more you learn to observe that feedback before pressing buttons, the more natural the controller becomes.

A Typical Workflow

A common sequence might look like this:

1. Press SELECT on the desired track.
2. Press PLUG-IN.
3. Adjust the required parameters.
4. Press TRACK to return to mixer control.

Although simple, this workflow is repeated countless times during a typical production session.

It quickly becomes second nature.

The Bigger Picture

At first glance, SELECT appears to be a very ordinary button.

In reality, it sits at the centre of the X-Touch's operating philosophy.

The controller is always asking the same question:

"Which track am I working on?"

The SELECT button provides the answer.

Understanding this idea of focus is one of the biggest steps towards mastering the X-Touch.

Exercise
Open a Bitwig project containing several tracks.

Press the SELECT button on each channel strip in turn.

After each selection:

- enter Device Mode
- enter Browser Mode
- observe the scribble strips
- observe the encoder assignments

Notice that the hardware itself never changes.

Only the controller's focus changes.

Once you begin thinking in terms of focus rather than simply selection, many other features of the X-Touch become much easier to understand.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 7 - Displays and Feedback

The X-Touch is much more than a collection of switches, knobs and motor faders.

It is constantly communicating with you.

Almost every action performed in Bitwig is reflected somewhere on the controller. Likewise, almost every action you perform on the controller is reflected in Bitwig.

Understanding this two-way conversation is one of the keys to using the X-Touch confidently.

More Than a Remote Control

Many MIDI controllers simply send commands to the computer.

The X-Touch goes much further.

It continuously receives information from Bitwig and updates its displays, LEDs and motor faders to match the current state of the project.

Rather than being a passive controller, it becomes an extension of the software itself.

> Field Note
> A useful way to think about the X-Touch is that it is a window into Bitwig.
>
> Instead of constantly looking at the computer monitor, you can often see exactly what is happening simply by glancing at the controller.

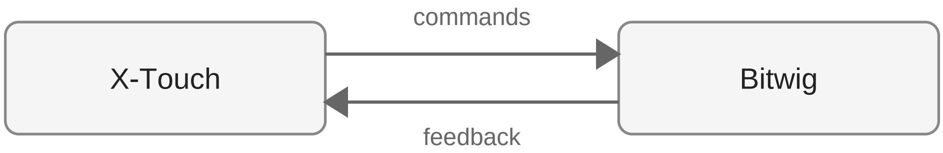


Figure 7.1 — Information flows continuously in both directions between Bitwig and the X-Touch.

Scribble Strips

The eight LCD scribble strips are one of the X-Touch's defining features.

Most of the time they display track names, but that is only one of their many roles.

Depending upon the current operating mode they may display:

- track names
- device names
- plug-in parameters
- browser entries
- send names
- automation information

The important point is not to memorise every display.

Instead, remember that the scribble strips are always showing the information that is most relevant to your current task.

LED Rings

Surrounding each rotary encoder is an LED ring.

These provide immediate visual feedback about the value being controlled.

Different parameters use different display styles.

For example:

- pan controls are centred
- continuous values fill progressively
- switches simply indicate on or off

After a short time you begin to recognise these patterns instinctively.

Often you can tell what a control is doing before you even touch it.

Illuminated Buttons

Many buttons on the X-Touch include built-in LEDs.

These indicate the current state of important functions such as:

- MUTE
- SOLO
- RECORD
- transport operations
- automation modes

The illuminated buttons are constantly reflecting the current state of Bitwig.

They are not merely indicators of what you have pressed.

Motor Faders

The motor faders provide perhaps the most dramatic form of feedback.

Whenever you load a project, switch banks or recall automation, the faders immediately move to their new positions.

This movement is not simply a visual effect.

It confirms that the controller has received updated information from Bitwig and is synchronised with the project.

> Reality Check
> Never try to prevent a motor fader from moving.
>
> The motors are positioning the fader to match Bitwig's current value.
>
> Allow them to move freely.

Feedback Builds Confidence

Imagine selecting a different track.

Within a fraction of a second you may notice:

- new track names
- different plug-in parameters
- updated LED rings
- illuminated buttons changing state

All of this happens automatically.

The controller is telling you that its focus has changed.

The more you learn to trust this feedback, the more natural the X-Touch becomes.

Observe Before You Adjust

One habit separates experienced users from beginners.

Beginners often press buttons first and try to understand the result afterwards.

Experienced users do the opposite.

Before making any adjustment they glance at the controller and ask:

- Which track is selected?
- Which mode am I in?
- What are the scribble strips showing?
- What do the LED rings indicate?
- Which buttons are illuminated?

Only then do they begin making changes.

This simple habit avoids countless mistakes.

> Field Note
> Throughout this book we return to a single idea:
>
> Observe before you adjust.
>
> The X-Touch is continually telling you what it is doing.
>
> Learning to read that information is more valuable than memorising button combinations.

The Controller Is Always Working

Even when you are not touching it, the X-Touch is active.

Automation moves.

Track selection changes.

Devices change.

Projects load.

Throughout all of this the controller quietly updates itself to remain synchronised with Bitwig.

That constant feedback is one of the reasons the X-Touch feels so different from simpler MIDI controllers.

Looking Ahead

The next chapters explore the controller's physical controls in more detail.

As you learn about the rotary encoders, motor faders and transport controls, remember that every control performs two roles:

It accepts your input.

It also provides valuable feedback.

Understanding both roles is essential to mastering the X-Touch.

Exercise
Open any Bitwig project and spend two minutes simply observing the controller.

Without pressing any buttons, identify:

- the selected track
- the current mode
- the information shown on the scribble strips
- the LED ring positions
- illuminated buttons
- motor fader positions

Now select a different track.

Watch how the controller updates before touching anything else.

Developing the habit of reading the controller before operating it will make every subsequent chapter easier to understand.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 8 - Modifiers: SHIFT, OPTION, CONTROL and ALT

Modifiers: SHIFT, OPTION, CONTROL and ALT

So far, most of the controls in this guide have been considered one at a time.

Press a button.

Turn a V-Pot.

Move a fader.

But the X-Touch has far more functions than it has physical controls.

One of the ways DrivenByMoss solves this is through modifier buttons.

The four important modifiers are:

- SHIFT
- OPTION
- CONTROL
- ALT

Hold one of these while operating another control, and the second control may perform a different function.

If you are familiar with keyboard shortcuts, the principle is already familiar:

```
control
   +
modifier
   =
another function
```

The important thing is not to memorise every possible combination immediately.

It is to understand the pattern.

One Surface, More Functions

Consider a control we already know: the Jog Wheel.

Turn it normally and it changes the transport position.

But DrivenByMoss can reinterpret that same physical movement when a modifier is held:

```
Jog Wheel
     │
     ├── normally       → transport position
     │
     ├── OPTION         → tempo
     │
     ├── CONTROL        → loop start
     │
     └── ALT            → loop length
```

The Jog Wheel has not physically changed.

Its context has.

This is the same principle introduced earlier in this guide when we looked at modes.

A modifier creates a temporary context.

Modifiers Are Temporary Modes

There is a useful way to think about the four modifier buttons:

Holding a modifier temporarily changes the meaning of another control.

A normal mode may remain active until you leave it.

A modifier usually lasts only while you hold its button.

Conceptually:

```
Normal context
      │
      │ hold OPTION
      ▼
OPTION context
      │
      │ release OPTION
      ▼
Normal context
```

This makes modifiers particularly useful for secondary operations.

The main function remains immediately available, while the less frequently needed function sits behind a modifier.

SHIFT

SHIFT is probably the modifier you will encounter most often.

It frequently provides:

• a secondary version of an operation;
• finer control;
• reverse movement through a sequence.

For example, when adjusting some continuous parameters, SHIFT gives finer resolution.

With the Jog Wheel:

```
Jog Wheel
    → move transport position

SHIFT + Jog Wheel
    → move transport position more finely
```

In other contexts SHIFT selects a related secondary operation.

You will see examples throughout the following chapters.

Do not assume that SHIFT always means "fine adjustment", however.

Its exact meaning depends on the control and the current mode.

OPTION

OPTION often changes what the operation acts upon or exposes a related operation.

For example, BANK and CHANNEL normally move the controller's view through the project.

With OPTION held, DrivenByMoss can instead use those controls to move objects within the project.

Conceptually:

```
BANK / CHANNEL
      │
      ├── normally
      │      move the VIEW
      │
      └── OPTION
             move an OBJECT
```

OPTION is also important in several workflows we will meet later.

For example:

```
OPTION + MARKER
```

creates a marker.

And:

```
OPTION + REWIND
OPTION + FORWARD
```

navigate between markers.

OPTION therefore does not have one universal definition.

Think of it as:

Give this control its alternative operation.

CONTROL

CONTROL frequently exposes a structural operation or another dimension of control.

One particularly useful example appears in Device Mode.

Holding CONTROL allows the V-Pots to expose devices so that a particular device can be selected directly.

CONTROL also changes the Jog Wheel:

```
CONTROL + Jog Wheel
```

adjusts the loop start.

With SHIFT added:

```
CONTROL + SHIFT + Jog Wheel
```

provides finer adjustment.

Again, CONTROL does not have one fixed meaning.

The current mode still matters.

ALT

ALT is another modifier used to expose alternative parameter operations.

With the Jog Wheel:

```
ALT + Jog Wheel
```

adjusts loop length.

Adding SHIFT gives finer control:

```
ALT + SHIFT + Jog Wheel
```

adjusts loop length more precisely.

ALT also appears in other specialised workflows, including operations concerned with clip length.

These will make more sense when we reach the relevant chapters.

Modifiers Can Be Combined

Modifiers are not necessarily used alone.

SHIFT can be combined with another modifier.

The Jog Wheel gives us an excellent example:

Control	Function
Jog Wheel	Transport position
SHIFT + Jog Wheel	Fine transport position
OPTION + Jog Wheel	Tempo
OPTION + SHIFT + Jog Wheel	Fine tempo
CONTROL + Jog Wheel	Loop start
CONTROL + SHIFT + Jog Wheel	Fine loop start
ALT + Jog Wheel	Loop length
ALT + SHIFT + Jog Wheel	Fine loop length

There is a pattern here.

SHIFT does not change what OPTION, CONTROL or ALT selects.

Instead, it changes the precision with which that parameter is adjusted.

So:

```
OPTION
   ↓
Tempo

OPTION + SHIFT
   ↓
Tempo, but finer
```

This is exactly the sort of relationship worth learning.

It reduces the number of apparently unrelated commands you need to remember.

Modifier Patterns, Not Modifier Rules

You may already have noticed some tendencies:

```
SHIFT
   → secondary / finer / reverse

OPTION
```

```
   → alternative operation

CONTROL
   → structural or additional control

ALT
   → another specialised parameter
```

These are useful mental shortcuts.

But they are not rules.

DrivenByMoss assigns modifier combinations according to what is useful in a particular context.

For example:

```
OPTION + MARKER
```

creates a marker.

There is no useful way to derive "create marker" from a universal definition of OPTION.

You simply learn that command as part of the Marker workflow.

The purpose of recognising modifier patterns is therefore not to predict every command.

It is to make the controller easier to understand when you encounter one.

Order Matters Less Than the Combination

When this guide writes:

```
OPTION + MARKER
```

it means:

1. hold OPTION;
2. press MARKER;
3. release the buttons.

Likewise:

```
CONTROL + SHIFT + Jog Wheel
```

means:

1. hold CONTROL;
2. hold SHIFT;
3. turn the Jog Wheel;
4. release the modifiers.

The `+` notation means use these controls together.

It does not mean that you should press them as a rapid sequence.

Modifier Buttons and Modes Work Together

Modifiers do not replace modes.

They work inside them.

This distinction is important.

Suppose the controller is in Device Mode.

The V-Pots already have meanings determined by Device Mode.

Now hold OPTION.

The V-Pots can acquire another set of meanings appropriate to that mode.

So the complete context may be thought of as:

```
Physical control
      +
Current mode
      +
Held modifier
      =
Current function
```

This extends the mental model from Chapter 3.

The function of a physical control is not necessarily inherent in the control itself.

It emerges from context.

The Display Is Your Ally

Modifier combinations can make the surface seem complicated if you try to remember everything from button labels alone.

Don't.

As discussed in Chapter 7, the displays are part of the control system.

When a modifier or mode changes what controls mean, look at the feedback the X-Touch and Bitwig provide.

The controller should be treated as a conversation:

```
You change context
      ↓
DrivenByMoss changes assignments
      ↓
The displays provide feedback
      ↓
You make the next decision
```

This becomes especially important in Device Mode, Browser Mode and the advanced edit modes.

Don't Learn the Modifier Table

It is tempting at this point to make a huge table containing every possible combination of:

```
SHIFT
OPTION
CONTROL
ALT
```

with every button on the X-Touch.

That would turn this guide into exactly the kind of manual we are trying to avoid.

Instead, learn modifier combinations with the task they perform.

When we study markers, we will learn:

```
OPTION + MARKER
```

because it is part of the marker workflow.

When we study Device Mode, we will learn CONTROL and OPTION combinations because they help navigate devices and parameter pages.

When we study recording, we will learn the RECORD modifiers because they make sense in the context of recording.

The modifier is not the subject.

The job you are trying to do is the subject.

A Useful Mental Model

At this point we can expand our model of the X-Touch considerably.

Earlier we had:

```
physical control
      ↓
current mode
      ↓
current function
```

Now we can add modifiers:

```
                ┌──────────────┐
                │ Current Mode │
                └──────┬───────┘
                       │
                       │
                       ▼
```

```
┌────────────────┐    context    ┌─────────────────┐
│ Physical       │───────────────│ Current         │
│ Control        │               │ Function        │
└────────────────┘               └─────────────────┘
                        ▲
                        │
                ┌──────────────┐
                │ Modifier     │
                │ if held      │
                └──────────────┘
```

You do not need to consciously think through this diagram every time you touch the controller.

Its purpose is to explain why the same V-Pot, button or wheel can perform so many different jobs without the surface becoming physically enormous.

What Comes Next

Now that we understand modifiers, the next chapters can use them without stopping repeatedly to explain what SHIFT, OPTION, CONTROL and ALT are.

We will see them applied to:

- V-Pots;
- motor faders;
- transport;
- the Jog Wheel;
- devices;
- Browser navigation;
- mixer modes;
- markers;
- automation;
- groups;
- recording.

The individual combinations will be introduced when they become useful.

The Important Idea

If there is one thing to take away from this chapter, it is this:

A modifier temporarily changes the context of another control.

You do not need to memorise every modifier combination.

Learn the basic controls first.

Then learn the modified versions as part of real workflows.

Once that pattern becomes familiar, SHIFT, OPTION, CONTROL and ALT stop looking like four more sets of commands to memorise.

They become something much more useful:

four ways of asking the X-Touch to do a little more.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 9 - V-Pots (Rotary Encoders)

The eight rotary encoders above the channel strips are among the most versatile controls on the X-Touch.

Behringer refers to them as V-Pots.

At first glance they look like ordinary rotary knobs. In reality, each V-Pot combines three useful features:

- a rotary encoder
- a push switch
- an LED ring

More importantly, their purpose changes according to the current mode.

A V-Pot might control pan in one moment, a send level in another, and a plug-in parameter a few seconds later.

This ability to adapt is what makes the V-Pots so powerful.

Encoders Rather Than Pots

Despite the name V-Pot, these controls are not conventional potentiometers.

A traditional potentiometer has physical end stops. Turn it fully anticlockwise and eventually it can go no further. The physical position of the knob therefore represents the current value.

The X-Touch V-Pots work differently.

They are endless rotary encoders.

You can continue turning them in either direction indefinitely.

This matters because the same physical encoder can control many different parameters without its position becoming meaningless when you change from one parameter to another.

Field Note
The word V-Pot is inherited from the Mackie Control terminology.

In practical terms, think of it as a pushable endless rotary encoder surrounded by an LED display.

Why Endless Encoders Matter

Imagine that an ordinary knob is controlling pan and is currently turned fully to the left.

You then switch modes and assign that same knob to a plug-in parameter whose current value is 80%.

What should happen?

With a conventional potentiometer there is an immediate disagreement between the physical position of the knob and the value in the software.

An endless encoder has no such problem.

The X-Touch simply changes the LED ring to show the new value.

The encoder is immediately ready to adjust it.

This is another example of the controller adapting itself to the current context.

The LED Ring

Each V-Pot is surrounded by a ring of LEDs.

As we saw in the previous chapter, this is an important part of the X-Touch's feedback system.

The LEDs provide a visual indication of the value currently assigned to the encoder.

For a pan control, for example, the display can indicate whether the signal is positioned:

- left
- centre
- right

For other parameters the LEDs may instead represent a value progressing from minimum to maximum.

The important point is that the LED ring represents the current software value, not a permanent physical position of the encoder.

> Reality Check
> When you change modes and the LED rings suddenly change, nothing has gone wrong.
>
> The encoders are simply displaying the values of the parameters they now control.

Turning a V-Pot

Turning an encoder changes the currently assigned parameter.

Which parameter that happens to be depends upon context.

For example, the V-Pots may control:

- track pan
- send levels
- device parameters
- instrument parameters
- plug-in parameters
- browser navigation

You therefore cannot answer the question:

"What does this V-Pot control?"

without first answering another question:

"What mode am I in?"

This is exactly the mental model introduced earlier in the book.

The hardware remains the same.

Its meaning changes.

Pressing a V-Pot

The V-Pots can also be pressed.

This is easy to overlook when first using the X-Touch because they look primarily like rotary controls.

The push action gives DrivenByMoss another command that can be associated with each encoder.

Its precise function depends upon the current context.

In one mode it may perform an action associated with the parameter being displayed; in another, it may be used for navigation or selection.

Do not assume that pressing an encoder always performs the same operation.

Instead, look at the current mode and the information displayed on the scribble strips.

> Field Note
> The V-Pot's push action is one of the reasons it is useful to think of the encoder and the scribble strip beneath it as a pair.
>
> The display tells you the context.
>
> The V-Pot lets you interact with it.

V-Pots and Scribble Strips

The relationship between the V-Pots and the scribble strips is fundamental.

Each encoder sits directly above a display.

That physical arrangement is deliberate.

When a mode assigns parameters to the encoders, the corresponding scribble strips can tell you what those parameters are.

Conceptually, each channel becomes:

```
     LED ring
        |
     V-Pot
        |
  Scribble Strip
        |
     Fader
```

The V-Pot provides the input.

The LED ring shows the value.

The scribble strip provides the context.

Together they form a small control surface within the larger controller.

Pan Control

One of the simplest uses of the V-Pots is track panning.

When the controller is operating in the appropriate pan mode, each encoder controls the pan position of its corresponding channel.

Turn left and the signal moves towards the left.

Turn right and it moves towards the right.

The LED ring provides immediate visual feedback.

This is a good mode in which to become familiar with the physical feel of the encoders because the relationship between movement and result is easy to understand.

Send Control

Switch to SEND mode and the same eight encoders take on a different role.

Instead of controlling pan, they can now adjust send levels.

The physical hardware has not changed.

Only the meaning assigned to it has changed.

This is precisely why endless encoders are so well suited to a controller such as the X-Touch.

The LED rings simply update to show the new values.

Device and Plug-In Control

The real power of the V-Pots becomes apparent when controlling devices.

In Device or PLUG-IN mode, the encoders can be mapped to parameters belonging to the currently selected device.

Instead of:

```
Pan    Pan    Pan    Pan    Pan    Pan    Pan    Pan
```

you might see parameters representing:

```
Cutoff   Reso   Attack   Decay   Sustain   Release   Drive   Mix
```

The scribble strips identify the parameters.

The LED rings show their current values.

The V-Pots adjust them.

Suddenly the X-Touch becomes much more than a mixer.

It becomes a hands-on device editor.

One Encoder, Many Jobs

This is worth emphasising.

The first V-Pot is not permanently:

Pan for Track 1

Nor is it permanently:

Parameter 1

It is simply the first encoder in the current context.

Change bank and it may control another track.

Change mode and it may control another type of parameter.

Select another device and it may acquire another purpose entirely.

Understanding this prevents one of the most common sources of confusion when learning a control surface.

Turn, Look, Listen

When adjusting a V-Pot, you have several sources of information available at once.

You can:

- feel the encoder movement
- watch the LED ring
- read the scribble strip
- observe the corresponding change in Bitwig
- hear the result

That combination makes the V-Pots particularly effective for tasks such as sound design.

Instead of concentrating entirely on a graphical plug-in interface, you can begin to adjust parameters by ear.

> Field Note
> Sometimes the best display in a music production system is no display at all.
>
> Once you know which parameter a V-Pot controls, try listening to the result rather than watching the computer screen.

V-Pots and Focus

The previous chapters introduced two important questions:

"What mode am I in?"

and:

"Which track is selected?"

V-Pots add a third:

"What parameter is this encoder controlling?"

Fortunately, you normally do not need to remember the answer.

The X-Touch tells you.

Read the scribble strip.

Look at the LED ring.

Then make the adjustment.

Once again:

Observe before you adjust.

The Bigger Picture

The V-Pots demonstrate almost everything that makes the X-Touch different from a simple MIDI controller.

They are:

- context-sensitive
- bidirectional
- multifunctional
- visually informative
- integrated with the current selection and mode

A single row of eight encoders can therefore control a remarkable amount of Bitwig.

The trick is not to memorise every possible assignment.

The trick is to understand the context.

Once you do that, the V-Pots become some of the most intuitive controls on the entire surface.

Exercise
Open a Bitwig project containing several tracks and devices.

Begin with the V-Pots controlling pan.

Turn several encoders and observe both the LED rings and Bitwig.

Next, switch to SEND mode.

Notice how the LED rings immediately change to represent the send values.

Finally, select a track containing a device and enter the appropriate device or plug-in mode.

Read the parameter names shown on the scribble strips and adjust several of them using the V-Pots.

Throughout the exercise, keep asking three questions:

1. Which track is selected?
2. Which mode am I in?
3. What parameter does this V-Pot currently represent?

Do not try to memorise the assignments.

Instead, practise reading the controller until the answer becomes obvious from the feedback it provides.

Chapter 10 - Motor Faders

The nine motorised faders are probably the first thing most people notice about the X-Touch.

Eight belong to the channel strips, while the ninth sits separately as the master fader.

Move a fader and Bitwig responds.

Change a value in Bitwig and the fader moves by itself.

Switch banks and all eight channel faders can spring into new positions almost simultaneously.

It is impressive to watch, but the motors are not there simply for show.

They solve one of the fundamental problems of using physical controls with software.

The Problem with Ordinary Faders

Imagine a simple MIDI controller with eight conventional faders.

The first fader is controlling Track 1, whose volume is currently fairly high.

Now switch to another bank.

That same physical fader might now control Track 9, whose volume is much lower.

The hardware fader is still sitting near the top of its travel, while the software value it represents is somewhere near the bottom.

The two disagree.

Different controllers solve this problem in different ways, often using techniques such as value pickup or soft takeover.

The X-Touch takes a much more direct approach.

It simply moves the fader.

Field Note
Motorisation means that the physical position of a fader can always reflect the value currently held by Bitwig.

The hardware adapts to the software rather than asking you to compensate for the difference.

Faders as Input and Output

It is tempting to think of a fader purely as an input device.

You move it.

Bitwig receives a new value.

But an X-Touch motor fader works in both directions.

```
You move the fader
        │
        ▼
      Bitwig

      Bitwig
        │
        ▼
The fader moves
```

This makes the fader both a control and a display.

Its position is information.

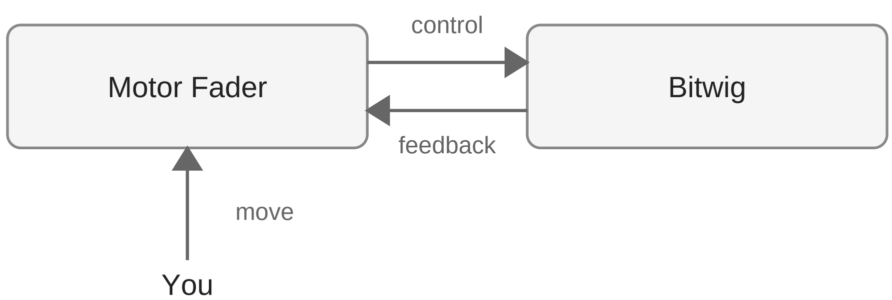


Figure 9.1 — The motor fader is both an input control and a physical display of Bitwig's current value.

Eight Channel Faders

The eight main faders correspond to the eight channels in the current bank.

In a simple project this might mean:

```
Fader      1     2     3     4     5     6     7     8
Track      1     2     3     4     5     6     7     8
```

Move to the next bank and those same physical faders might represent:

```
Fader      1     2     3     4     5     6     7     8
Track      9    10    11    12    13    14    15    16
```

As soon as the bank changes, the motors reposition the faders to show the values belonging to the new tracks.

This is the physical manifestation of the banking system introduced in Chapter 4.

Banking Makes Sense of the Motors

Without motorisation, changing banks would create an immediate mismatch between the physical faders and the newly displayed tracks.

With motorised faders, the transition is obvious.

Press a bank control and three things happen together:

- the visible tracks change
- the scribble strips update
- the faders move to their new positions

The X-Touch has physically reconfigured itself for the new bank.

> Reality Check
> If several faders suddenly move when you change banks, nothing unexpected has happened.
>
> They are simply taking up the positions belonging to the newly displayed tracks.

Touch-Sensitive Faders

The X-Touch faders are not merely motorised.

They are also touch-sensitive.

The controller can detect when your finger is resting on a fader.

This is particularly important when working with automation.

There is a significant difference between:

"The fader is at this position."

and:

"The user is currently touching the fader."

Knowing when you have physically taken control allows Bitwig and the controller to behave appropriately during automation operations.

We will encounter this distinction again when we look at automation in more detail.

Do Not Fight the Motors

When a motor fader moves by itself, let it move.

Do not hold it in position or deliberately push against the motor.

The movement is feedback from Bitwig.

The fader is trying to show you the value that currently exists in the project.

If you want to change that value, wait for the fader to settle and then move it normally.

> Field Note
> A moving fader is not the X-Touch doing something to your mix.
>
> It is the X-Touch showing you something that has already happened in Bitwig.
>
> That distinction is important.

Automation Comes Alive

Motor faders become particularly valuable when automation is playing.

A track's volume may change continuously during a song.

On a conventional controller, the physical fader would remain stationary while the software value moved.

On the X-Touch, the motor fader can follow those changes.

You can therefore see the automation physically happening.

A vocal may rise slightly during a chorus.

A pad may fade away.

An effects return may increase during a transition.

The corresponding faders can move as those changes occur.

This is one of the moments when the distinction between software and hardware begins to disappear.

Writing Automation

Touch sensitivity and motorisation become especially useful when writing automation.

You can grab a fader and perform a level change naturally with your hand rather than drawing an automation curve with a mouse.

Bitwig records the movement.

When the automation is played back, the fader can reproduce it.

The physical gesture has become part of the project.

For many users this feels much closer to working on a traditional mixing console.

The Master Fader

To the right of the eight channel faders is a ninth motorised fader.

This is the master fader.

Unlike the channel faders, it is not part of the eight-channel banking system.

Its role is associated with the master output rather than whichever eight tracks happen to be visible.

This separation is useful because the master remains available while you move through the project's channel banks.

Faders and SELECT

Remember the distinction introduced in Chapter 6.

A fader represents a track within the current bank.

The SELECT button identifies the track that currently has your attention.

Moving a fader does not mean that you have changed the controller's focus.

Likewise, selecting a track does not mean that its fader must move.

These are different concepts:

```
Fader position  →  Track level

SELECT          →  Current focus
```

Keeping those two ideas separate prevents a surprising amount of confusion.

Faders as Feedback

Chapter 7 described the various ways in which the X-Touch communicates with you.

Motor faders are perhaps its most physical form of feedback.

You do not have to read a number to know roughly where a track's level is.

You can simply look at the fader.

In fact, you can often see the overall shape of a mix by looking across all eight faders at once.

That is something a row of numerical values on a computer screen does not communicate nearly as naturally.

Mixing with Your Hands

This is where the X-Touch begins to change the experience of using a DAW.

With a mouse, adjusting several track levels usually means moving one virtual fader at a time.

With the X-Touch you have eight physical faders beneath your fingers.

You can bring one track down while raising another.

You can balance several channels together.

You can make small changes without repeatedly pointing at different objects on the screen.

Mixing becomes a physical activity again.

> Field Note
> The advantage of physical faders is not simply that they replace virtual ones.
>
> The real advantage is that you can control several things at once.
>
> That is something a mouse is fundamentally bad at doing.

Trust the Position

A useful habit is to trust what the faders are showing you.

When you change banks, allow them to settle.

When automation is playing, watch them move.

When you open an existing project, notice how the X-Touch reconstructs the mix physically in front of you.

The faders are not approximations of the software state.

They are part of the feedback system that keeps the controller and Bitwig synchronised.

The Bigger Picture

The motor faders demonstrate the same principle we encountered with the V-Pots.

The X-Touch does not expect its physical controls to have one permanent meaning.

Instead, the hardware adapts itself to the current context.

The V-Pots achieve this by having no fixed physical position and using LED rings to display their values.

The faders achieve it differently.

They physically move.

Two different engineering solutions serve the same underlying idea:

The controller should reflect the current state of the software.

That principle lies at the heart of the X-Touch.

Exercise
Open a Bitwig project containing more than eight tracks.

First, look at the positions of the eight channel faders.

Change to the next bank and watch what happens.

Notice how the scribble strips and faders update together.

Move several faders and confirm that the corresponding Bitwig levels follow them.

Now change those same levels using Bitwig's mixer and watch the X-Touch faders respond.

If the project contains volume automation, play a section containing that automation and observe the appropriate fader.

Finally, return to the first bank.

The faders should return to the positions belonging to those tracks.

The aim of this exercise is not simply to practise moving faders.

It is to experience the central idea of this chapter:

The motor faders are both controls and physical displays.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 11 - Transport Controls

After learning about modes, selection, V-Pots and motor faders, the Transport section of the X-Touch may come as something of a relief.

Most of it does exactly what you expect.

The familiar REWIND, FAST FORWARD, STOP, PLAY and RECORD buttons provide immediate control over Bitwig's transport.

But the Transport section is more than a convenient replacement for clicking Play on the computer screen.

Combined with the jog wheel and navigation controls, it allows you to move around a project, locate a position and control playback without repeatedly reaching for the mouse.

The Five Transport Buttons

At the bottom-right of the X-Touch are five large transport buttons:

- REWIND
- FAST FORWARD
- STOP
- PLAY
- RECORD

Their layout deliberately resembles the transport controls found on tape machines, hardware recorders and traditional mixing systems.

If you have used almost any audio equipment before, they should feel immediately familiar.

PLAY

Press PLAY to start playback from the current position.

Pressing PLAY is one of the simplest examples of the two-way relationship between the X-Touch and Bitwig.

The controller sends the command.

Bitwig begins playback.

The X-Touch then reflects the resulting transport state through its illuminated controls and displays.

Even here, the controller is both issuing a command and providing feedback.

STOP

Press STOP to stop playback.

Simple as this sounds, having a large physical STOP button becomes surprisingly valuable.

There is no need to locate a small transport control on the computer screen.

Your hand quickly learns where STOP is, and after a while you may find yourself using it without looking at the controller at all.

> Field Note
> Transport controls are particularly good candidates for developing muscle memory.
>
> Unlike context-sensitive controls, their basic purpose remains predictable.
>
> Once your hand knows where PLAY and STOP are, controlling playback becomes almost unconscious.

RECORD

The RECORD button controls Bitwig's recording transport.

Recording involves more than simply pressing one button — tracks must be armed correctly and Bitwig must be ready to record — but once those conditions are satisfied, RECORD gives you direct physical control over the process.

This becomes especially useful when recording instruments or automation.

Instead of preparing everything and then reaching for the mouse, you can remain focused on the performance.

Reality Check
Pressing RECORD does not automatically make every track record.

Track arming and the transport's recording state are separate concepts.

Always check that the intended track is correctly armed before beginning a take.

REWIND and FAST FORWARD

The REWIND and FAST FORWARD buttons allow you to move backwards and forwards through the project.

Their exact behaviour is governed by the integration between the X-Touch, DrivenByMoss and Bitwig, but their purpose is straightforward:

Move the current playback position.

For coarse navigation they provide a quick way of moving through the project without touching the mouse.

For more deliberate positioning, however, another control becomes particularly useful.

The Jog Wheel

The large wheel above the transport buttons is the jog wheel.

It provides a physical way of moving through the project timeline.

Turn it one way to move backwards.

Turn it the other way to move forwards.

Unlike clicking somewhere on a graphical timeline, the jog wheel encourages you to think in terms of movement from the current position.

This can feel much more natural when searching for the beginning of a phrase, a transition or a particular point in a recording.

Coarse and Fine Navigation

The transport buttons and jog wheel complement each other.

Think of them broadly as two approaches to navigation:

```
REWIND / FAST FORWARD
          │
          ▼
     Move quickly
```

```
      Jog Wheel
          │
          ▼
  Position precisely
```

You do not need to choose one method exclusively.

A typical workflow might involve moving rapidly towards the required part of the project and then using the jog wheel to refine the position.

> Field Note
> The most useful navigation method is usually the one that requires the least thought.
>
> Use the transport buttons when you want to move.
>
> Use the jog wheel when you want to arrive somewhere specific.

The Time Display

Navigation becomes much more useful when you can see where you are.

The X-Touch's main display provides positional information that helps you keep track of the current location within the project.

As you move through the timeline, the display updates accordingly.

Once again, input and feedback work together:

```
Turn jog wheel
      │
      ▼
Bitwig position changes
      │
      ▼
Display updates
```

You move.

The controller tells you where you have arrived.

Transport and the Computer Screen

It is worth noticing how many routine mouse movements disappear once the transport controls become familiar.

Without a control surface, a typical editing or recording session may involve repeatedly moving the pointer between:

- the arrangement
- the mixer

- device controls
- the transport
- the timeline

With the X-Touch, many transport operations remain permanently beneath your hand.

You can keep the computer screen focused on the information that actually needs a graphical display.

Transport While Mixing

Physical transport controls are especially useful while mixing.

Imagine adjusting several faders during playback.

You hear something you want to investigate.

Instead of moving your hand away from the controller and locating Bitwig's transport:

1. Press STOP.
2. Navigate backwards.
3. Press PLAY.
4. Listen again.
5. Continue adjusting the mix.

Your attention remains on the sound and the controller rather than shifting repeatedly between hardware and screen.

This may seem like a small improvement.

Repeated hundreds of times during a session, it becomes a substantial one.

Transport While Recording

The same principle applies when recording.

A typical sequence might be:

1. Select the required track.
2. Arm it for recording.
3. Navigate to the starting position.
4. Press RECORD.
5. Perform the part.
6. Press STOP.
7. Return to the beginning of the take.
8. Press PLAY to review it.

The computer is still doing all the recording.

The X-Touch simply allows the routine operations surrounding that recording to happen physically.

That keeps your attention where it belongs: on the performance.

Visual Feedback

The transport section also reinforces the principle introduced in Chapter 7:

Observe before you adjust.

Illuminated transport buttons tell you about the current state.

The position display tells you where you are.

Bitwig's own interface provides further confirmation.

Do not treat the physical controls and the software interface as two independent systems.

They are two views of the same transport state.

Developing Muscle Memory

Transport is one of the best places to begin operating the X-Touch without constantly looking at it.

The five transport buttons are large, consistently positioned and easy to distinguish by location.

With practice, your hand begins to find them automatically.

The jog wheel is equally unmistakable.

This is an important step towards a more tactile workflow.

You stop thinking:

"Where is the Play button?"

and simply press it.

That small change is one of the reasons a control surface can make a DAW feel more like an instrument and less like a computer application.

A Step Towards Mouse-Lite

The Transport section gives us an early glimpse of the workflow we shall explore more fully in Chapter 13.

The aim is not necessarily to eliminate the mouse.

There are many tasks for which a mouse remains an excellent tool.

The aim is to stop reaching for it when a physical control is faster, clearer or more natural.

Starting playback is a perfect example.

You could move the pointer to Bitwig's PLAY button and click it.

But when a large physical PLAY button is already beneath your hand, why would you?

> Field Note
> A Mouse-Lite workflow is not about refusing to use the mouse.
>
> It is about using the most appropriate tool for each job.
>
> For transport operations, that tool is very often the X-Touch.

The Bigger Picture

The Transport section may be one of the least mysterious parts of the X-Touch, but it plays an important role in the overall workflow.

The faders allow you to mix.

The V-Pots allow you to adjust parameters.

SELECT establishes focus.

The displays tell you what is happening.

And the Transport section lets you control when it happens.

Together, these controls begin to form a complete working environment.

You are no longer simply controlling individual Bitwig parameters.

You are operating the session.

Exercise
Open an existing Bitwig project.

For the duration of this exercise, avoid using the mouse for transport operations.

Using only the X-Touch:

1. Start playback.
2. Stop playback.
3. Move backwards through the project.
4. Move forwards through the project.
5. Use the jog wheel to locate a particular section.
6. Start playback from that position.
7. Stop again.

Now choose a short section of the project and practise repeatedly navigating to it and playing it.

Do not concentrate on speed.

Instead, concentrate on allowing your hand to learn where the transport controls are.

The goal is to reach the point where PLAY, STOP and basic navigation no longer require conscious thought.

That is one more step towards a Mouse-Free — or Mouse-Lite — workflow.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 12 - Device Mode

So far, much of our attention has been focused on the mixer.

We have selected tracks, changed banks, adjusted levels, controlled pan and navigated through the project.

But a Bitwig track is much more than a channel in a mixer.

It may contain instruments, audio effects and other devices, each with parameters of its own.

Device Mode is where the X-Touch begins to reach inside the selected track and give you physical control over those devices.

From Mixer to Device

In normal mixer operation, the eight channel strips represent tracks.

Conceptually, the controller looks something like this:

```
X-Touch

   │
   ▼

Tracks

   │
   ├── Drums
   ├── Bass
   ├── Synth
   ├── Guitar
   └── Vocals
```

Device Mode takes us one level deeper.

A selected track might contain:

```
Synth Track

   │
   ├── Instrument
   ├── EQ
   ├── Compressor
   └── Delay
```

Once a device is selected, we can go deeper again:

```
Selected Device

   │
   ├── Parameter 1
   ├── Parameter 2
   ├── Parameter 3
   └── ...
```

This hierarchy is one of the most useful ways to understand Device Mode.

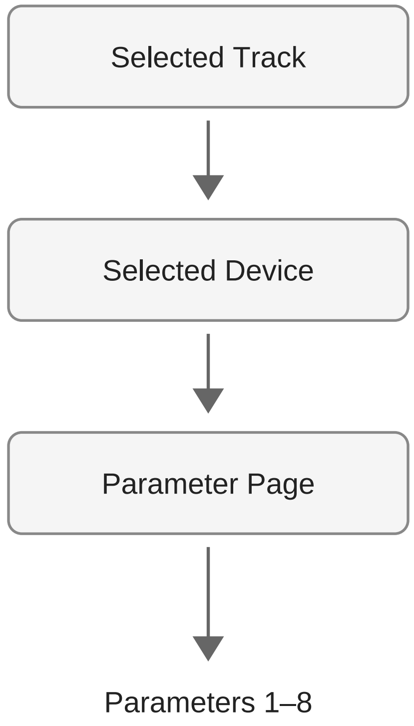


Figure 11.1 — Device Mode moves the controller's focus from the selected track, through its devices, to the parameters of the selected device.

SELECT Comes First

Chapter 6 introduced the idea of focus.

Device Mode is where that idea becomes particularly important.

Before controlling a device, the X-Touch needs to know which track contains it.

The first step is therefore familiar:

Select the track.

Once the track is selected, Device Mode can operate on the devices belonging to that track.

This gives us a simple hierarchy:

```
SELECT
   │
   ▼
Track
   │
   ▼
Device
   │
   ▼
Parameters
```

If Device Mode appears to be showing the wrong instrument or effect, work backwards through that hierarchy.

Start by checking the selected track.

> Reality Check
> When the X-Touch appears to be controlling the "wrong" device, do not immediately start pressing mode buttons.
>
> First check which track is selected.
>
> The controller may be doing exactly what it should — just on a different track.

A Track Can Contain Many Devices

A Bitwig track can contain an entire chain of devices.

For example:

`Polysynth → EQ+ → Compressor → Delay+`

Each device has a different purpose and therefore exposes different parameters.

Selecting the Polysynth might give you controls relating to synthesis.

Selecting the EQ might give you frequency and gain controls.

Selecting the delay might expose delay time, feedback and mix.

The same eight V-Pots can control all of them.

The hardware remains unchanged.

The context changes.

The Scribble Strips Become Essential

This is where the scribble strips really earn their keep.

In mixer operation, an encoder's purpose may be fairly predictable.

In Device Mode, it could represent almost anything.

One encoder might control:

- filter cutoff
- resonance
- attack
- release
- frequency
- gain
- feedback
- wet/dry mix

Trying to memorise those assignments would be hopeless.

Fortunately, you do not need to.

The scribble strips tell you what the controls currently represent.

This is precisely the behaviour we explored in Chapter 7.

Observe before you adjust.

In Device Mode, that principle becomes essential.

The V-Pots Become Parameter Controls

Chapter 8 introduced the V-Pots as context-sensitive rotary encoders.

Device Mode demonstrates why that design is so useful.

Suppose the currently selected device exposes eight parameters:

```
Cutoff   Reso   Attack   Decay   Sustain   Release   Drive   Mix
```

The eight V-Pots can immediately become physical controls for those parameters.

Turn an encoder and the corresponding value changes in Bitwig.

The LED ring provides visual feedback.

The scribble strip tells you what the parameter is.

Suddenly, a software device begins to feel rather more like a piece of hardware.

More Than Eight Parameters

Of course, most devices have considerably more than eight parameters.

The X-Touch only has eight V-Pots.

This creates a familiar problem.

Fortunately, we have already encountered the solution.

Banking.

Just as eight channel strips can provide access to many tracks, eight encoders can provide access to many parameters.

Think of the device parameters as pages:

```
Page 1

P1   P2   P3   P4   P5   P6   P7   P8


Page 2

P9   P10  P11  P12  P13  P14  P15  P16


Page 3

P17  P18  P19  P20  P21  P22  P23  P24
```

Move to another parameter page and the V-Pots acquire new assignments.

The scribble strips update.

The LED rings update.

Once again, the controller reconfigures itself around the current context.

> Field Note
> Parameter pages are really just another form of banking.
>
> You already understand the underlying idea from Chapter 4.
>
> Eight physical controls provide a window onto a much larger collection of software controls.

Devices, Parameters and Pages

It is useful to keep three separate concepts in mind:

```
Device
   │
   ▼
Parameter Page
   │
   ▼
Eight Parameters
```

First you choose what device you want to control.

Then you choose which page of its parameters you want to see.

The eight V-Pots then control the parameters on that page.

This structure may sound complicated when described in words.

In use, it quickly becomes natural because the X-Touch continually updates its displays as you navigate.

Follow the Feedback

Imagine moving from a synthesizer to a delay.

The controller may immediately change:

• parameter names on the scribble strips
• LED ring positions
• encoder functions

Nothing has physically moved except perhaps your hand.

Yet the entire row of V-Pots now represents a different piece of software.

This is exactly why Chapter 7 placed so much emphasis on feedback.

In Device Mode, the displays are not merely helpful.

They are part of the interface.

Editing by Ear

One of the most enjoyable aspects of Device Mode is that it can change how you interact with plug-ins and instruments.

Consider adjusting a filter cutoff with a mouse.

Your attention naturally goes to the computer screen.

You locate the graphical control.

You watch the pointer.

You watch the value change.

With the X-Touch, you can instead:

1. Read the scribble strip.
2. Place your hand on the appropriate V-Pot.
3. Turn it.
4. Listen.

Once you know which control you are adjusting, there may be very little reason to keep watching the screen.

Field Note
Device Mode is not necessarily about reproducing every control from a plug-in interface on the X-Touch.

Its real value is providing immediate physical access to the parameters you actually want to adjust.

Hardware Feel from Software Instruments

This becomes particularly interesting with software instruments.

A synthesizer may exist entirely inside the computer, but Device Mode gives some of its controls a physical presence.

Instead of dragging a virtual filter knob, you turn an encoder.

Instead of watching an envelope value, you adjust it while listening.

The software has not changed.

What has changed is your relationship with it.

This is one of the ways in which a control surface can make working with a DAW feel less like operating software and more like playing an instrument.

Device Chains

As projects become more complex, tracks often contain several devices.

For example:

```
Instrument
    │
    ▼
EQ
    │
    ▼
Compressor
    │
    ▼
Saturation
    │
    ▼
Delay
```

Device Mode allows you to move through that chain and concentrate on one device at a time.

The important thing is not to memorise where every device lives.

Instead, maintain the mental hierarchy:

Track → Device → Parameter Page → Parameter

Whenever you become lost, move back up that hierarchy.

Which track?

Which device?

Which page?

Which parameter?

The displays provide the answers.

Device Mode and Focus

We can now expand the idea introduced in Chapter 6.

SELECT establishes the track focus.

Device Mode establishes the device focus.

Parameter navigation establishes the parameter focus.

Conceptually:

```
Project
   │
   ▼
Selected Track
   │
   ▼
Selected Device
   │
   ▼
Parameter Page
   │
   ▼
Parameter
```

This is not just a description of Device Mode.

It is a useful mental model for navigating Bitwig itself.

Returning to the Mixer

Device Mode is temporary.

Entering it does not alter the structure of your project or permanently reassign the X-Touch.

When you have finished editing the device, you can return to normal mixer operation.

The V-Pots resume their mixer-related roles.

The scribble strips return to displaying the appropriate mixer information.

The controller has not changed identity.

It has simply changed context again.

Device Mode and the Mouse

Device Mode also represents another significant step towards our Mouse-Free — or Mouse-Lite — workflow.

Without a control surface, editing a device often involves:

- locating the device on screen
- locating the required parameter
- moving the pointer to it
- dragging the control
- moving to the next parameter
- repeating the process

With the X-Touch, once the correct device and parameter page are selected, eight physical controls are immediately available.

That does not mean the graphical interface becomes useless.

Some devices contain complex visual information that is much easier to understand on screen.

The point is not to eliminate the screen or the mouse.

The point is to stop depending upon them when a physical control provides a better way to work.

> Field Note
> Mouse-Lite does not mean "never touch the mouse".
>
> It means:
>
> Use the mouse when it is the best tool — not simply because it is the only tool you know.

The Bigger Picture

Device Mode brings together almost every concept we have encountered so far.

Banks and Channels taught us how a limited number of physical controls can represent a larger system.

Modes taught us that controls change meaning according to context.

SELECT taught us about focus.

Displays and Feedback taught us to read the controller.

V-Pots gave us context-sensitive parameter controls.

Device Mode combines all of those ideas.

That is why it can initially appear complicated.

It is also why, once the earlier concepts are understood, it becomes surprisingly logical.

You do not need to memorise the entire X-Touch.

You simply need to know where you are.

Exercise
Open a Bitwig project containing a track with several devices.

Select that track using the X-Touch.

Enter Device Mode and observe the scribble strips before touching any controls.

Identify the currently selected device.

Now:

1. Locate a parameter shown on one of the scribble strips.
2. Adjust it with the corresponding V-Pot.
3. Watch the LED ring respond.
4. Listen to the result.
5. Move to another parameter page if one is available.
6. Observe how all eight encoder assignments change.
7. Select another device in the chain.
8. Observe how the scribble strips and LED rings change again.

Finally, select a completely different track.

Notice how the available devices change with it.

Throughout the exercise, keep the hierarchy in mind:

Track → Device → Parameter Page → Parameter

If you become lost, do not start pressing buttons at random.

Work backwards through the hierarchy and read the feedback the X-Touch is already providing.

Chapter 13 - Browser Mode

Up to this point, most of our work with the X-Touch has involved controlling things that already exist in the project.

We have selected tracks.

We have adjusted levels.

We have navigated devices and changed their parameters.

But sooner or later we need something new.

Perhaps we want to add an instrument.

Perhaps we need an audio effect.

Perhaps we want to replace a preset or search for a particular sound.

This is where Browser Mode becomes particularly interesting.

Instead of reaching immediately for the mouse, we can begin exploring Bitwig's Browser directly from the X-Touch.

From Control to Creation

Device Mode allows us to move down through a hierarchy:

```
Track
   │
   ▼
Device
   │
   ▼
Parameter Page
   │
   ▼
Parameter
```

Browser Mode performs a different job.

It allows us to choose something that is not yet part of that hierarchy.

Conceptually:

```
Selected Track
      │
      ▼
```

```
    Browser
       │
       ▼
Search / Navigate
       │
       ▼
Choose an Item
       │
       ▼
Add to Project
```

This is an important change.

The X-Touch is no longer merely editing the project.

It is helping us build it.

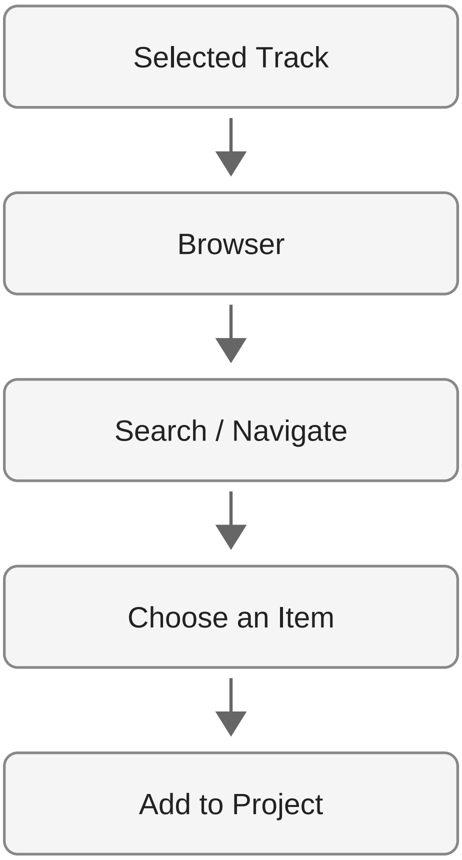


Figure 12.1 — Browser Mode extends the workflow from selecting a destination to finding and adding new material.

SELECT Still Comes First

Once again, the SELECT button plays a central role.

Before opening the Browser, ask:

"Where do I want the new item to go?"

The selected track establishes the context in which browsing takes place.

This is the same idea introduced in Chapter 6.

Selection is not simply about highlighting something.

It establishes focus.

A useful working sequence is therefore:

```
SELECT
   |
   ▼
Destination
   |
   ▼
Open Browser
   |
   ▼
Choose
```

That first step prevents a surprising amount of confusion.

> Reality Check
> Before browsing for a new device or sound, check the selected track.
>
> Finding exactly what you wanted and then discovering that you were working in the wrong place is considerably less entertaining than it sounds.

Opening the Browser

With DrivenByMoss, the X-Touch provides access to Bitwig's Browser without requiring you to begin the operation with the mouse.

When Browser Mode is entered, the role of the controller changes.

Controls that previously represented mixer or device parameters can now become navigation and selection controls.

The scribble strips change accordingly.

This should already feel familiar.

The hardware has not changed.

The context has.

Read the Scribble Strips

Browser Mode is another excellent demonstration of why the scribble strips matter.

When browsing, they may present information relating to the available choices rather than track or device parameters.

The labels on the controller therefore become your guide.

Do not assume that an encoder still performs the function it had a few seconds earlier.

Read first.

Then turn or press.

The principle remains:

Observe before you adjust.

Although in Browser Mode we might reasonably amend that to:

Observe before you choose.

Navigation Rather Than Parameter Control

In Device Mode, turning a V-Pot normally changes a value.

Browser Mode can give those same controls a different purpose.

Now an encoder may be involved in navigating choices rather than adjusting a continuous parameter.

This distinction is important.

In one context:

```
Turn V-Pot
     │
     ▼
Change Parameter
```

In another:

```
Turn V-Pot
     │
     ▼
Navigate Choices
```

The physical action is identical.

Its meaning is determined by the current mode.

This is precisely the mental model we established much earlier in the book.

Narrowing the Search

Bitwig's Browser can contain a huge amount of material.

Depending upon your installation, that may include:

- instruments
- audio effects
- note effects
- presets
- samples
- plug-ins
- other device content

Browsing everything at once would quickly become unwieldy.

The useful approach is to narrow the available choices until the required item becomes easy to locate.

Think of browsing as a funnel:

```
Everything
    │
    ▼
Category
    │
    ▼
Filtered Results
    │
    ▼
Selection
```

Each decision reduces the number of possibilities.

This is much more effective than treating the Browser as one enormous list.

Browsing Is Contextual

The contents of the Browser depend upon what you are doing.

If you are adding an instrument, the useful choices are different from those involved in adding an audio effect.

If you are choosing a preset, the available material depends upon the device.

If you are looking for a sample, you are dealing with another type of content again.

This means Browser Mode should not be thought of as:

"A list of everything Bitwig contains."

It is better understood as:

"A way of choosing something appropriate for the current context."

That makes the Browser much less intimidating.

Choosing an Item

Eventually, browsing leads to a choice.

Once the required item has been located, it can be selected and inserted or loaded in the appropriate context.

At this point the Browser has done its job.

The new item becomes part of the project.

Conceptually:

```
Browser
   |
   ▼
Choose
   |
   ▼
Insert
   |
   ▼
New Device
```

And something rather satisfying can happen next.

The workflow can move naturally back into Device Mode.

Browser Mode Meets Device Mode

Suppose you want to add a delay to a track.

The workflow might be:

1. SELECT the destination track.
2. Open the Browser.
3. Navigate to an appropriate delay.
4. Choose it.
5. Return to Device Mode.
6. Adjust its parameters with the V-Pots.

Notice what has happened.

You have gone from:

"I want a delay."

to:

"I am adjusting the delay."

without the mouse necessarily being the centre of the operation.

This is where the individual chapters of the book begin joining together into complete workflows.

Field Note
Browser Mode becomes much more powerful when you stop thinking of it as an isolated feature.

It is the bridge between deciding that you need something and then controlling what you have added.

Browsing for Instruments

The same principle applies when building an instrument track.

Imagine starting with an appropriate empty track.

You can:

1. Select the track.
2. Enter the Browser.
3. Locate an instrument.
4. Choose it.
5. Enter Device Mode.
6. Begin adjusting its parameters.

The Browser finds the instrument.

Device Mode controls it.

SELECT establishes where the whole operation takes place.

Three apparently separate features become a single workflow.

Browsing for Presets

Browser Mode is equally useful when searching for sounds rather than devices.

Suppose you already have a synthesizer loaded but want another preset.

Instead of thinking:

"I need to operate the Browser."

think:

"I want another sound for this device."

The Browser is simply the mechanism that helps you find it.

This distinction matters because it keeps your attention on the musical task rather than the software operation.

Browsing with Your Ears

Browsing sounds presents an interesting opportunity.

Computer-based browsing naturally encourages us to look.

We read names.

We study categories.

We watch lists move up and down the screen.

But ultimately, when choosing a sound, the important question is usually:

"Does this sound right?"

The more navigation you can perform from the controller, the easier it becomes to keep your attention on listening rather than pointing and clicking.

> Field Note
> Names and categories help you find sounds.
>
> Your ears decide whether they belong in the music.

The Screen Still Matters

A Mouse-Lite workflow does not mean pretending the computer display has suddenly become useless.

Bitwig's Browser can present detailed information that simply cannot fit onto the X-Touch's scribble strips.

There will be occasions when looking at the screen is clearly the most efficient way to understand what is available.

That is perfectly fine.

The aim is not:

"Never look at Bitwig."

The aim is:

"Do not reach for the mouse automatically."

If the X-Touch can perform the operation comfortably, use it.

If the graphical Browser is better suited to the task, use that.

The two approaches complement each other.

Do Not Memorise the Browser

As with Device Mode, there is little value in trying to memorise every possible Browser state.

The available choices will change.

Your installed devices may change.

Your plug-ins may change.

Your presets and samples may change.

The useful skill is not remembering where everything lives.

It is understanding how to navigate what is currently in front of you.

Read the feedback.

Make a choice.

Observe the result.

Continue.

When You Become Lost

Browser Mode can initially feel more complicated than the mixer because the controls are being used in a less familiar way.

If you become unsure what is happening, return to the questions we have used throughout the book:

- Which track is selected?
- Which mode am I in?
- What are the scribble strips showing?
- What does the current control represent?

Do not press controls randomly in an attempt to escape.

Read the controller first.

The information you need is often already there.

> Reality Check
> When a control appears to have stopped doing its usual job, check whether you are still in Browser Mode.
>
> A V-Pot that normally adjusts pan cannot adjust pan while it is busy helping you navigate the Browser.

Building a Workflow

We can now combine much of what we have learned into a single sequence.

Imagine adding and adjusting a new effect:

```
SELECT Track
      │
      ▼
Open Browser
      │
      ▼
Find Effect
      │
      ▼
Choose Effect
      │
      ▼
Device Mode
      │
      ▼
Choose Parameter Page
      │
      ▼
Adjust with V-Pots
      │
      ▼
Listen
```

This is more than a collection of X-Touch features.

It is a workflow.

That distinction is important.

The purpose of learning the controller is not to become good at pressing its buttons.

The purpose is to make the controller disappear into the process of making music.

Another Step Towards Mouse-Lite

Browser Mode brings us very close to the subject of the final chapter.

Earlier we used the X-Touch to manipulate a project.

Now we can also begin adding things to it.

That removes another common reason for immediately reaching for the mouse.

Again, the goal is not mouse avoidance for its own sake.

A mouse remains extremely useful for detailed editing, graphical operations and many other tasks.

But opening a Browser, finding something, selecting it and then adjusting it are operations that can increasingly take place from the control surface.

The mouse becomes an option rather than a reflex.

The Bigger Picture

Browser Mode completes an important chain of ideas.

SELECT establishes where we are working.

Browser Mode helps us find something to add.

Device Mode lets us control what we have added.

V-Pots provide the physical controls.

Displays and Feedback tell us what those controls currently mean.

Motor Faders and Transport allow the rest of the session to continue around us.

The individual features of the X-Touch are beginning to disappear.

In their place we are developing workflows.

And that brings us naturally to the final chapter.

Exercise

Open a Bitwig project containing a suitable track.

For this exercise, resist the temptation to begin with the mouse.

First:

1. Select the destination track from the X-Touch.
2. Enter Browser Mode.
3. Observe the scribble strips before touching anything else.
4. Navigate through the available choices.
5. Choose an instrument, effect or other suitable item.
6. Confirm that it has been added in the intended location.

Now move directly into Device Mode.

7. Identify the newly added device.
8. Locate a useful parameter.
9. Adjust it using a V-Pot.
10. Listen to the result.

Finally, return to normal mixer operation.

The purpose of this exercise is not to prove that every operation can be performed without a mouse.

It is to experience the complete sequence:

Select → Browse → Choose → Control → Listen

When that sequence begins to feel like a single operation rather than five separate features, the X-Touch is becoming part of your workflow.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 14 - Mixer Edit Modes

Mixer Edit Modes

By now, we have become used to an important characteristic of the X-Touch:

The physical controls stay in the same place, but what they control can change.

The eight channel strips normally give us a familiar view of the mixer.

We have faders for track levels, V-Pots for parameters, and buttons for operations such as SELECT, MUTE, SOLO and ARM.

DrivenByMoss can take this idea further.

Instead of thinking only in terms of eight complete channel strips, we can temporarily ask the X-Touch to concentrate on one particular aspect of the mixer.

That is the idea behind the mixer edit modes.

Looking Across the Mixer

Imagine eight tracks:

```
Track 1   Track 2   Track 3   Track 4   Track 5   Track 6   Track 7   Track 8
```

There are several different ways we might want to look across them.

We could ask:

```
What are their volumes?
```

or:

```
What are their panorama positions?
```

or:

```
How much of each track is being sent to Send 1?
```

The tracks have not changed.

What changes is the dimension of the mixer that we are looking at.

Conceptually:

```
                Tracks 1–8
                    │
     ┌──────────────┼──────────────┐
     │              │              │
     ▼              ▼              ▼
  Volume        Panorama         Sends
```

This is a useful way to understand the mixer edit modes.

Track Edit Mode

Press TRACK to enter Track Edit Mode.

Rather than treating each V-Pot simply as the normal control for its channel strip, Track Edit Mode exposes parameters associated with the selected track.

This gives the V-Pots another role:

```
Selected Track
      │
      ▼
Track Edit Mode
      │
      ▼
V-Pots → Track Parameters
```

The precise parameters available can depend on the current DrivenByMoss configuration.

The important point is the change in perspective.

We are no longer primarily looking across eight tracks.

We are looking more deeply at one selected track.

Volume Edit Mode

Press TRACK again and DrivenByMoss switches to Volume Edit Mode.

Now the V-Pots provide another way of controlling the volumes of the eight tracks in the current bank.

Conceptually:

```
V-Pot 1  → Track 1 Volume
V-Pot 2  → Track 2 Volume
V-Pot 3  → Track 3 Volume
...
V-Pot 8  → Track 8 Volume
```

Of course, the X-Touch already has eight motor faders.

So why would we want volume on the V-Pots as well?

One answer is FLIP, which we met earlier.

The ability to exchange the roles of faders and rotary controls becomes much more useful when the controller can present parameters in different ways.

The important lesson is not that one control is the "correct" way to adjust volume.

It is that DrivenByMoss allows the surface to be reorganised around the task.

Panorama Edit Mode

Press PAN to enter Panorama Edit Mode.

The eight V-Pots now control the panorama positions of the eight tracks in the current bank:

```
V-Pot 1  → Track 1 Pan
V-Pot 2  → Track 2 Pan
V-Pot 3  → Track 3 Pan
...
V-Pot 8  → Track 8 Pan
```

This is perhaps the clearest example of an edit mode.

Instead of thinking:

```
Channel 1
   ├── Volume
   ├── Pan
   ├── Mute
   └── Solo
```

we temporarily rotate our view through ninety degrees and think:

```
Pan
   ├── Track 1
   ├── Track 2
   ├── Track 3
   ├── ...
   └── Track 8
```

The information is the same.

The view onto it has changed.

Fine Adjustment with SHIFT

As we saw in Chapter 8, SHIFT frequently provides finer adjustment of continuous parameters.

That pattern applies here too.

Hold SHIFT while turning a V-Pot when you need more precise control.

Conceptually:

```
Turn V-Pot
```

```
     ↓
Normal adjustment

SHIFT + Turn V-Pot
     ↓
Finer adjustment
```

This is a good example of why we introduced modifiers before exploring the advanced modes.

The modifier itself is no longer something new to learn.

We simply apply an existing idea to a new task.

Send Edit Modes

The SEND control opens another particularly useful family of mixer views.

Press SEND and the V-Pots control a Send across the eight tracks.

For example, in the first Send mode:

```
             SEND 1

Track 1  ──────────────► Send 1
Track 2  ──────────────► Send 1
Track 3  ──────────────► Send 1
Track 4  ──────────────► Send 1
Track 5  ──────────────► Send 1
Track 6  ──────────────► Send 1
Track 7  ──────────────► Send 1
Track 8  ──────────────► Send 1
```

The eight V-Pots now answer one question:

How much of each track is being sent to this destination?

This is an especially natural way to work with effects sends.

If Send 1 feeds a reverb, for example, you can adjust the amount of reverb for eight tracks without changing the basic mixer bank.

Moving Through the Sends

A project may contain more than one Send.

Repeated presses of SEND move through the available Send modes.

Conceptually:

```
SEND
  ↓
Send 1
  ↓
SEND
  ↓
```

```
Send 2
  ↓
SEND
  ↓
Send 3
  ↓
...
```

DrivenByMoss supports Send modes 1–8.

Hold SHIFT while pressing SEND to move through the Sends in the opposite direction.

So:

```
SEND
   → next Send

SHIFT + SEND
   → previous Send
```

This is another modifier pattern we have already encountered:

SHIFT can provide the related or reverse operation.

Jumping Directly to a Send

Cycling is useful when moving between neighbouring Sends.

But if you already know which Send you want, DrivenByMoss provides a quicker route.

Hold SEND and use the channel SELECT buttons to choose the Send directly.

Conceptually:

```
SEND + SELECT 1  → Send 1
SEND + SELECT 2  → Send 2
SEND + SELECT 3  → Send 3
...
SEND + SELECT 8  → Send 8
```

This is an important example of a control acquiring a new meaning from context.

Normally:

```
SELECT 3
```

means:

Select Track 3.

But while SEND is being used as the context:

```
SEND + SELECT 3
```

means:

Select Send 3.

The physical SELECT button has not changed.

The context has.

Switching a Send On or Off

Send level and Send state are two different things.

Turning a V-Pot adjusts the amount being sent.

DrivenByMoss also allows the Send itself to be switched on or off.

In Send Mode:

```
OPTION + V-Pot press
```

toggles the corresponding Send.

So one rotary control can provide both:

```
Turn
   ↓
Send level

OPTION + Press
   ↓
Send on/off
```

This is exactly the kind of compact interaction for which modifiers are useful.

The common operation remains immediately available.

The related secondary operation is one modifier away.

Send Mode as a Mixing Tool

Send Mode is worth thinking about as more than a shortcut.

Suppose Send 1 contains a reverb and Send 2 contains a delay.

You can move between two complete views of your mix:

```
Send 1 — Reverb
────────────────────
Track 1 amount
Track 2 amount
Track 3 amount
...
Track 8 amount
```

then:

```
Send 2 — Delay
────────────────────
Track 1 amount
```

```
Track 2 amount
Track 3 amount
...
Track 8 amount
```

Rather than visiting each track and adjusting one Send at a time, you can work across the mix according to the effect you are shaping.

For dub-style mixing in particular, this way of thinking can be extremely useful.

ARM, MUTE and SOLO

The channel-strip buttons continue to provide immediate control over the tracks.

ARM

Press ARM on a channel strip to arm that track for recording.

The bank of ARM buttons allows record state to be managed directly from the surface.

MUTE

Press MUTE to mute the corresponding track.

SOLO

Press SOLO to solo the corresponding track.

These are familiar mixer operations, but DrivenByMoss also provides modified versions for useful bank-wide or related operations.

Clearing Mutes

When several tracks have been muted, clearing them individually can be tedious.

DrivenByMoss provides a global operation for clearing active mutes.

This is particularly useful after using mute creatively during playback.

Rather than searching the surface for every illuminated MUTE button, the controller can return the mixer to an unmuted state in one operation.

The exact global-control combination will be included in the Quick Reference once the final command set has been verified against the current DrivenByMoss version.

Clearing Solos

The same problem occurs with Solo.

During mixing it is easy to accumulate a state in which one or more tracks are soloed.

DrivenByMoss provides a global operation for clearing active solos.

Again, the important workflow idea is:

```
Experiment
    ↓
Mute / Solo tracks
    ↓
Return the mixer to a known state
```

The controller is not merely a way to activate states.

It also provides efficient ways to recover from them.

Monitoring

SHIFT-modified channel buttons provide access to monitoring-related functions.

These are useful when recording and will become more meaningful when we look at advanced recording workflows later in the guide.

For now, the important distinction is that ARM, MUTE and SOLO buttons are not necessarily limited to the words printed on their caps.

Like the rest of the X-Touch, they participate in the modifier system.

FLIP

FLIP changes the relationship between rotary and fader control.

This becomes especially useful in the mixer edit modes.

A parameter that would normally be controlled from a V-Pot can be placed onto the motor faders.

Why might we want that?

Because a fader gives us:

- a long physical travel;
- touch sensitivity;
- motorised feedback;
- a very different physical feel from a rotary encoder.

For some adjustments, the V-Pot is ideal.

For others, the fader is more expressive.

FLIP lets the controller adapt.

Conceptually:

```
Before FLIP

V-Pots   → parameter
Faders   → volume


After FLIP

V-Pots   → volume
Faders   → parameter
```

The exact result depends on the current mode, but the principle remains the same:

FLIP exchanges control roles.

Mixing by Dimension

The most useful way to think about this chapter is not as a collection of modes.

Think of it as several different ways to slice through the same mixer.

By channel

```
Track 1
   ├── Volume
   ├── Pan
   ├── Sends
   ├── Mute
   └── Solo
```

By parameter

```
Pan
   ├── Track 1
   ├── Track 2
   ├── Track 3
   └── ...
```

By Send

```
Reverb Send
   ├── Track 1
   ├── Track 2
   ├── Track 3
   └── ...
```

None of these is more correct than another.

They are different views of the same project.

Choosing the Useful View

A good control surface should reduce the distance between an intention and an action.

If your intention is:

Turn Track 4 down.

the motor fader is probably already exactly what you want.

If your intention is:

Spread these eight tracks across the stereo field.

Panorama Mode gives you eight related controls together.

If your intention is:

Decide which tracks should feed the delay.

Send Mode gives you a complete view of that Send across the bank.

The important skill is therefore not memorising modes.

It is recognising which view best matches the job you are doing.

The Important Idea

Mixer Edit Modes let you reorganise the X-Touch around a particular mixing task.

Instead of always seeing:

```
eight tracks
×
many different controls
```

you can temporarily see:

```
one kind of control
×
eight tracks
```

That change of perspective is what makes these modes useful.

The X-Touch is still an eight-channel control surface.

But DrivenByMoss lets those eight channels become eight simultaneous views of:

- volume;
- panorama;
- Sends;
- track parameters;

depending on what you need at that moment.

Once this way of thinking becomes natural, the controller begins to feel less like a fixed bank of knobs and faders and more like a surface that reorganises itself around the mix.

Coming Next

Mixer Edit Modes help us change what we are controlling.

The next chapter deals with another question:

Where are we in the project?

Markers give us named positions in the timeline, and DrivenByMoss gives the X-Touch several ways to create, display and navigate them.

In the next chapter we will look at Markers and Advanced Navigation.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 15 - Markers and Advanced Navigation

Markers and Advanced Navigation

Transport controls let us move through a project.

We can play, stop, rewind, fast-forward and use the Jog Wheel to position the cursor precisely.

But as a project grows, there is another way to think about navigation.

Instead of asking:

How far should I move?

we can ask:

Where do I want to go?

That is where markers become useful.

A marker gives a position in the project a meaningful identity.

Instead of remembering that the chorus begins somewhere around bar 33, we can create a marker called:

```
Chorus
```

and navigate directly in terms of the structure of the music.

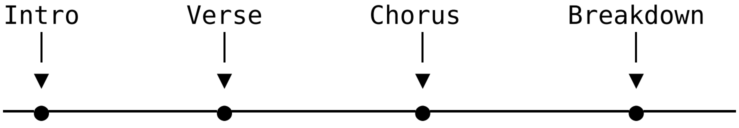
DrivenByMoss extends this idea onto the X-Touch.

From Position to Structure

Without markers, navigation is principally about position:

```
1       9       17      25      33      41
|-------|-------|-------|-------|-------|
```

With markers, the same timeline can acquire musical meaning:



Now the project is not merely a sequence of bars.

It has landmarks.

And once those landmarks exist, the X-Touch can use them for navigation.

The MARKER Button

The MARKER button is the centre of marker operations.

Pressing MARKER enters Marker Mode.

This changes the context of the controller so that markers can be accessed from the surface.

As with the other modes we have encountered:

The hardware has not changed. Its current meaning has.

Marker Mode turns the X-Touch from a device that moves through time into one that can navigate the structure of the project.

Creating a Marker

Hold OPTION and press MARKER:

```
OPTION + MARKER
```

to create a marker at the current position.

Conceptually:

```
Current Position
       │
       │ OPTION + MARKER
       ▼
    New Marker
```

This is particularly useful because creating the marker does not require breaking away from the control surface and reaching for the mouse.

Imagine playback reaching the beginning of an important section.

You stop at the required position and press:

```
OPTION + MARKER
```

That position is now represented by a marker in the project.

The operation that originally looked like an obscure modifier combination now makes sense as part of a workflow:

```
Navigate
   ↓
Find an important position
   ↓
```

```
OPTION + MARKER
   ↓
Create a landmark
```

Showing and Hiding Markers

Hold SHIFT and press MARKER:

```
SHIFT + MARKER
```

to show or hide the markers.

This provides another example of the modifier vocabulary introduced in Chapter 8.

MARKER performs the primary marker operation.

SHIFT + MARKER accesses a related secondary operation.

You do not need to remember this as an isolated shortcut.

Think:

```
MARKER
   → work with markers

SHIFT + MARKER
   → change their visibility
```

Previous and Next Marker

Once markers exist, REWIND and FORWARD gain useful alternative functions.

Normally these controls move through the timeline.

Hold OPTION, however, and they navigate between markers:

```
OPTION + REWIND
   → previous marker

OPTION + FORWARD
   → next marker
```

This gives us two distinct forms of navigation:

```
REWIND / FORWARD
        │
        ├── normally
        │      move through time
        │
        └── OPTION
               move through structure
```

That distinction is worth remembering.

Without OPTION, you are navigating distance.

With OPTION, you are navigating landmarks.

A Different Way to Move Through a Song

Suppose a project contains:

```
Intro
Verse 1
Chorus 1
Verse 2
Chorus 2
Breakdown
Final Chorus
Outro
```

With markers placed at those positions, moving through the project no longer requires repeatedly turning the Jog Wheel or holding FORWARD.

Instead:

```
OPTION + FORWARD
```

can step through the musical sections.

Likewise:

```
OPTION + REWIND
```

moves back through them.

So navigation begins to resemble:

```
Verse 1
   |
   ▼
Chorus 1
   |
   ▼
Verse 2
   |
   ▼
Chorus 2
```

rather than:

```
bar 17
   |
   ▼
bar 33
   |
   ▼
bar 49
```

Both views are useful.

Markers simply add another level of meaning.

Playing from a Marker

In Marker Mode, the V-Pots give us another way to interact with markers.

Press the V-Pot associated with a marker to start playback from that marker.

Conceptually:

```
Marker
   │
   │ press associated V-Pot
   ▼
Playback starts here
```

This turns the X-Touch into something closer to a set of structural navigation controls.

Instead of moving to a location and then pressing PLAY, the marker itself becomes the starting point.

Why Marker Mode Matters

Marker Mode may initially seem like a fairly specialised feature.

It becomes much more useful when we consider how often navigation interrupts a workflow.

Imagine repeatedly working on a transition between a verse and chorus.

With ordinary transport navigation, the cycle might be:

```
Stop
  ↓
Find the position
  ↓
Reposition
  ↓
Play
```

With a marker already placed at the start of the section:

```
Select marker
      ↓
Play from marker
```

The mechanical part of the operation becomes smaller.

That matters because every navigation operation is something happening between musical decisions.

Markers as a Project Map

There is a broader principle here.

A well-marked project effectively contains a map of itself.

For example:

```
SONG
 │
 ├── Intro
 │
 ├── Verse 1
 │
 ├── Chorus 1
 │
 ├── Verse 2
 │
 ├── Chorus 2
 │
 ├── Breakdown
 │
 ├── Final Chorus
 │
 └── Outro
```

The X-Touch can then navigate that map.

This is fundamentally different from using the controller merely as a remote transport.

The transport controls know about time.

Markers know about places.

Together they provide both.

Markers During Arrangement

Markers are particularly useful while arranging.

Suppose you are deciding whether the breakdown is too long.

You might repeatedly compare:

```
Chorus 2
    ↓
Breakdown
    ↓
Final Chorus
```

Marker navigation lets you move between those structural points quickly.

You can concentrate on questions such as:

- Does the transition work?
- Does the next section arrive soon enough?
- Does the energy drop too far?
- Does the final chorus have enough impact?

The X-Touch handles the navigation while your attention remains on the arrangement.

Markers During Mixing

Markers are equally useful during mixing.

A mix rarely needs attention everywhere at once.

You may want to revisit:

```
Vocal entrance
Bass drop
Busy chorus
Quiet breakdown
Final hit
```

Markers can turn these into repeatable destinations.

For example:

```
OPTION + FORWARD
      ↓
Busy Chorus
      ↓
listen
      ↓
OPTION + FORWARD
      ↓
Quiet Breakdown
      ↓
listen
```

This is much faster than repeatedly finding those positions visually.

It also encourages listening rather than screen-watching.

Markers During Performance

Markers can also provide a useful structural framework when Bitwig is being used more performatively.

The important point is not that markers replace clips, scenes or Launcher workflows.

They do not.

Markers describe positions in the Arranger timeline.

But where the Arranger itself forms part of a performance, marker navigation gives the X-Touch another way to interact with that structure.

As always, the appropriate tool depends on the job.

Marker Navigation and the Transport Controls

We first encountered REWIND and FORWARD as transport controls.

Now we can see that they have two related roles.

Ordinary transport

```
REWIND
FORWARD
```

move through the timeline.

Structural navigation

```
OPTION + REWIND
OPTION + FORWARD
```

move through markers.

This is why modifier commands are best learned in context.

If Chapter 8 had simply presented:

```
OPTION + FORWARD = next marker
```

as one line in a large table, it would have been another fact to memorise.

Here, its purpose is obvious.

Marker Mode and the Mental Model

Markers also fit neatly into the mental model developed throughout this guide.

The same physical controls can acquire meaning from context.

For example:

```
V-Pot
  +
Marker Mode
  =
Marker interaction
```

And:

```
FORWARD
   +
OPTION
   =
Next Marker
```

So our increasingly complete model is:

```
Physical Control
       +
Current Mode
       +
Modifier
       =
Current Function
```

The surface is not a collection of controls with permanently fixed meanings.

It is a system of contextual controls.

A Practical Marker Workflow

A simple workflow might look like this.

1. Find the beginning of a section

Use the normal transport controls or Jog Wheel.

```
Navigate
   ↓
Beginning of Chorus
```

2. Create the marker

```
OPTION + MARKER
```

3. Repeat for other important sections

For example:

```
Intro
Verse
Chorus
Breakdown
Outro
```

4. Navigate structurally

Use:

```
OPTION + REWIND
OPTION + FORWARD
```

to move between those landmarks.

5. Enter Marker Mode when appropriate

Use MARKER to expose marker-oriented control from the X-Touch.

6. Start playback from the required marker

Press the corresponding V-Pot.

The result is a project that can increasingly be navigated by musical structure rather than screen position.

You Don't Need a Marker Everywhere

Markers are useful because they simplify navigation.

Too many markers can have the opposite effect.

A project containing:

```
Intro
Verse
Chorus
Breakdown
Outro
```

provides obvious destinations.

A project containing a marker every few bars may simply replace one navigation problem with another.

Use markers for positions you expect to revisit.

Good candidates include:

- major song sections;
- important transitions;
- difficult edits;
- mix-check locations;
- unusual events;
- positions needed repeatedly during a session.

The aim is not to document every metre of the timeline.

It is to create useful landmarks.

Less Looking, More Listening

There is another benefit to marker navigation that fits the larger aim of Project XTC.

If you know that the next press of:

```
OPTION + FORWARD
```

will take you to the next important section, you do not need to locate that section visually.

Your eyes do not need to find the mouse pointer.

The pointer does not need to find the timeline.

The timeline does not need to be zoomed to the right scale.

You simply move to the next landmark.

This is a small example of the mouse-lite principle we will return to later:

The best control-surface operation is often the one that lets your attention remain on the music.

The Important Idea

Transport navigation answers:

How do I move through time?

Marker navigation answers:

How do I move through the structure of my project?

DrivenByMoss gives the X-Touch both.

The essential marker commands are:

```
MARKER
   → Marker Mode

OPTION + MARKER
   → Create marker

SHIFT + MARKER
   → Show/hide markers

OPTION + REWIND
   → Previous marker

OPTION + FORWARD
   → Next marker
```

And within Marker Mode, the V-Pots allow markers to become direct playback destinations.

Once markers are part of your normal project workflow, navigation stops being purely a matter of bars, beats and cursor positions.

The project acquires places you can go.

Coming Next

So far we have used the motor faders to control parameters and respond to changes from Bitwig.

But their touch sensitivity becomes particularly powerful when Bitwig is recording those movements.

That brings us to one of the areas where a motorised control surface can feel dramatically different from using a mouse.

Next:

Automation.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 16 - Automation

Automation

The motor faders on the X-Touch do something rather special.

Move a fader and Bitwig responds.

Change the same parameter in Bitwig and the physical fader moves.

We saw this two-way relationship earlier:

```
You move the fader
        │
        ▼
      Bitwig

      Bitwig
        │
        ▼
The fader moves
```

Automation adds another dimension.

Bitwig can remember the movement.

Instead of a fader representing one fixed value, it can become part of a performance that changes over time.

From Position to Movement

Suppose a track begins quietly and gradually becomes louder.

Without automation, we might set the fader to one compromise level.

With automation, we can perform the change:

```
quiet                                loud
  │                                    │
  ▼                                    ▼
 ────────/────────────────────────────
        /
       /
```

Bitwig records the changing value.

When the project plays again, that movement can be reproduced.

And because the X-Touch has motorised faders, the physical fader can move with it.

That is one of the moments when a control surface stops feeling like a remote control and starts feeling like part of the DAW.

Automation Is About Behaviour

The X-Touch provides buttons for several automation modes:

- READ/OFF
- WRITE
- TOUCH
- LATCH
- TRIM

The names can look like a list that needs to be memorised.

A more useful way to understand them is to ask three questions:

```
What happens before I touch the control?

What happens while I move it?

What happens when I release it?
```

For a touch-sensitive motor fader, that gives us a simple model:

```
Touch
  ↓
Move
  ↓
Release
  ↓
What happens now?
```

The answer depends on the automation mode.

READ/OFF

The READ/OFF button controls automation playback.

In normal automation reading, Bitwig follows automation that has already been recorded.

Conceptually:

```
Recorded automation
        │
        ▼
      Bitwig
        │
        ▼
   Motor fader
```

The physical fader follows the changing automated value.

This is useful feedback.

You can see and feel the automation happening rather than merely watching a line move on the screen.

Automation Playback

Imagine that a track contains this volume automation:

```
Volume

high          ──────
             /
            /
low   ──────
      ────────────────────► time
```

During playback, the X-Touch fader follows it:

```
Bitwig automation
       │
       ▼
Motor position
       │
       ▼
Fader moves
```

Do not fight the fader while simply reading automation.

Its movement is information.

The controller is showing you what the project is doing.

WRITE

WRITE is the most direct automation-writing mode.

When Write is active, movements of an automated parameter are written into the project.

Conceptually:

```
WRITE active
     │
     ▼
Move fader
     │
     ▼
New value written
     │
     ▼
Automation continues to be written
```

Write is powerful precisely because it is direct.

It also deserves care.

If you leave Write active while playing through a section, you may overwrite automation that you intended to keep.

Think of WRITE as:

Keep writing the current control state into the automation.

That makes it useful when deliberately replacing an automation passage.

TOUCH

TOUCH makes particular sense with the X-Touch because the faders are touch-sensitive.

The important event is not merely moving the fader.

It is touching it.

Conceptually:

```
Existing automation plays
          │
          ▼
     Touch fader
          │
          ▼
You take control
          │
          ▼
      Move fader
          │
          ▼
New movement is written
          │
          ▼
    Release fader
          │
          ▼
Existing automation resumes
```

This is one of the most natural ways to make corrections to an existing automation pass.

You can listen to the automated mix, reach for the fader when something needs changing, make the adjustment, and let go.

Bitwig then resumes control.

Touch as an Override

The relationship can be thought of as a temporary handover:

```
Bitwig
```

```
   │
   │ automation playing
   ▼
Fader
   │
   │ you touch it
   ▼
  YOU
   │
   │ you release it
   ▼
Bitwig
```

This is where touch sensitivity becomes much more than a hardware specification.

The fader knows when you have deliberately put your hand on it.

That lets Bitwig distinguish between:

The motor is moving the fader

and:

The user is moving the fader.

That distinction is fundamental to touch automation.

LATCH

LATCH begins similarly to Touch.

Existing automation can play until you touch and move the control.

The important difference appears when you release it.

Conceptually:

```
Existing automation
        │
        ▼
   Touch fader
        │
        ▼
   Take control
        │
        ▼
    Move fader
        │
        ▼
  Release fader
        │
        ▼
New value remains latched
```

Instead of immediately returning to the previously recorded automation, the new value continues.

That makes Latch useful when you want to establish a new level and keep it there.

TOUCH and LATCH Compared

These two modes are easier to understand side by side.

TOUCH

```
automation
    │
    ▼
  touch
    │
    ▼
your movement
    │
    ▼
 release
    │
    ▼
automation resumes
```

LATCH

```
automation
    │
    ▼
  touch
    │
    ▼
your movement
    │
    ▼
 release
    │
    ▼
new value continues
```

The key difference is therefore not what happens while you are holding the fader.

It is what happens after you let go.

Choosing Between TOUCH and LATCH

Suppose a vocal is slightly too loud for one phrase.

TOUCH is a natural choice:

```
normal level
    ↓
phrase arrives
    ↓
touch fader
    ↓
```

```
pull it down
    ↓
phrase ends
    ↓
release
    ↓
previous automation resumes
```

Now suppose a synth needs to become quieter from the second chorus onwards.

LATCH may make more sense:

```
second chorus
    ↓
touch fader
    ↓
lower level
    ↓
release
    ↓
new level continues
```

The mode follows the musical intention.

TRIM

TRIM is associated with automation adjustment rather than simply replacing the existing automation shape.

The useful concept is relative change.

Imagine that you already have a detailed automation performance:

```
        /\
  /────┘  \──┐
─┘           └────
```

You like its movement, but the whole passage needs to sit a little lower.

Conceptually, trimming means:

```
existing shape
      +
relative adjustment
      =
same general shape at a new level
```

In the current DrivenByMoss MCU mapping, the TRIM control maps to Bitwig's available automation behaviour rather than providing a separate traditional console-style trim system.

For that reason, it is best to think of the X-Touch's automation buttons as controls over the automation modes Bitwig actually provides, rather than assuming that every label on the MCU surface corresponds to an identically named DAW function.

The Labels Belong to the MCU

This is an important general point.

The X-Touch follows the Mackie Control layout.

Its buttons therefore carry labels such as:

```
READ/OFF
WRITE
TRIM
TOUCH
LATCH
```

But the controller is being used with Bitwig through DrivenByMoss.

The printed label tells us where the control came from.

DrivenByMoss determines what that control does in Bitwig.

So:

Trust the current DrivenByMoss mapping, not assumptions based solely on the button legend.

This principle applies elsewhere on the X-Touch too.

Resetting Automation Overrides

DrivenByMoss also provides a way to reset automation overrides.

Use:

```
OPTION + READ/OFF
```

to reset overrides.

This is useful when manual intervention has left parameters overriding their normal automated state.

Conceptually:

```
Automation
    │
    ▼
Manual override
    │
    ▼
OPTION + READ/OFF
    │
    ▼
Reset override
```

It gives you a quick route back to the automation-controlled state.

Performing Automation

Automation does not have to be drawn.

It can be performed.

That distinction is especially important with a control surface.

Suppose you want a delay return to rise dramatically into the end of a phrase.

With a mouse, you might create automation points and draw a curve.

With the X-Touch, you can approach the same problem musically:

```
Play
  ↓
Listen
  ↓
Touch control
  ↓
Ride the level
  ↓
Release
```

The automation becomes a recorded gesture.

That is much closer to the way levels were traditionally ridden on a mixing console.

The First Pass Does Not Have to Be Perfect

One advantage of automation modes such as Touch is that automation can be refined.

A workflow might be:

```
First pass
   │
   ▼
Capture the broad movement
   │
   ▼
Listen again
   │
   ▼
Touch only where needed
   │
   ▼
Correct the problem area
```

This is often more natural than trying to perform an entire complicated automation move perfectly in one pass.

Automation can be treated like any other performance:

record it, listen, and improve it.

Automation and the Motor Fader

The motor fader gives us a particularly clear feedback loop:

```
      You
       │
       ▼
  Motor Fader
       │
       ▼
     Bitwig
       │
       ▼
  Automation
       │
       ▼
     Bitwig
       │
       ▼
  Motor Fader
```

Information travels both ways.

When you perform a move, the X-Touch sends control information to Bitwig.

When Bitwig plays the automation back, it sends the resulting state back to the X-Touch.

The fader therefore becomes both:

- an input device;
- an output display.

That is one of the defining advantages of a motorised control surface.

Don't Grab a Moving Fader Casually

A moving motor fader is telling you that Bitwig is controlling that parameter.

Touching it may have meaning depending on the current automation mode.

So before grabbing a moving fader, know which automation mode is active.

In READ, you may simply be inspecting playback.

In TOUCH, touching the fader may deliberately hand control to you.

In WRITE, your actions may replace automation.

The same physical gesture can therefore have very different consequences.

The current mode matters.

Automation Is Not Just Volume

Faders make volume automation particularly obvious, but automation in Bitwig is much broader.

Parameters throughout the project can change over time.

The X-Touch may expose those parameters through:

- faders;
- V-Pots;
- Device Mode;
- mixer edit modes;
- FLIP.

So the same general automation ideas apply beyond channel volume.

Conceptually:

```
Physical control
      │
      ▼
Bitwig parameter
      │
      ▼
Automation
```

If the parameter is automatable and the current X-Touch context gives you control over it, the surface can become part of the automation workflow.

FLIP and Automation

FLIP becomes particularly interesting here.

Suppose a parameter is normally assigned to a V-Pot.

FLIP may allow that parameter to be placed on a fader.

That gives the parameter:

- longer physical travel;
- touch sensitivity;
- motorised position feedback.

So FLIP is not merely a convenience for mixing.

It can change the physical way in which an automation performance is made.

For expressive automation, that can be significant.

Watching Automation Come Back

There is something worth doing at least once simply to understand the system.

Record a deliberate fader movement.

Then:

1. stop;
2. return to a point before the movement;
3. enable the appropriate automation playback;
4. press PLAY;
5. take your hand away.

Watch the fader reproduce what you performed.

The movement you made has become part of the project.

That simple demonstration makes the feedback loop tangible:

```
You moved it
     ↓
Bitwig remembered it
     ↓
Bitwig plays it
     ↓
The X-Touch moves it
```

A Practical Automation Workflow

A straightforward automation pass might look like this.

1. Choose the parameter

For example, track volume.

2. Choose the automation mode

For an existing automated mix that needs correction, TOUCH may be appropriate.

3. Start playback

Listen rather than watching the screen.

4. Touch the fader

Take control when the musical moment arrives.

5. Perform the movement

Ride the level by ear.

6. Release

In TOUCH, Bitwig can return to the existing automation behaviour.

7. Play the section again

Listen to the result while the motor fader reproduces the automation.

8. Correct it if necessary

Automation is editable and repeatable.

You are not committing to a one-take performance.

Automation and Mouse-Lite Working

Automation provides another example of why a control surface can change the relationship with a DAW.

With a mouse, automation often encourages this:

```
Look
  ↓
Point
  ↓
Click
  ↓
Draw
  ↓
Adjust
```

With a touch-sensitive fader:

```
Listen
  ↓
Touch
  ↓
Move
  ↓
Release
```

Neither approach is universally better.

Drawing automation is invaluable when exact editing is required.

But for a musical gesture, the physical approach can be far more immediate.

The aim of Project XTC is not to prohibit the mouse.

It is to make reaching for it a choice rather than a reflex.

Automation as Performance

This leads to perhaps the most useful way to think about automation with the X-Touch.

Do not think only:

I am programming a parameter change.

Think:

I am performing a parameter change and asking Bitwig to remember it.

That shift in perspective matters.

A fade becomes something you play.

A send ride becomes something you play.

A filter movement becomes something you play.

And because the X-Touch can reproduce the resulting movement, the surface remains connected to that performance afterwards.

The Important Idea

The automation buttons determine what happens when control passes between Bitwig and you.

The key questions are:

```
Before I touch:
Who has control?

While I move:
What is being written?

When I release:
What happens next?
```

That makes the main modes easier to distinguish:

```
WRITE
   → write automation directly

TOUCH
   → take control while touched,
     then return to automation

LATCH
   → take control,
     then retain the new value

READ/OFF
   → control automation playback state

OPTION + READ/OFF
   → reset automation overrides
```

And the motor fader makes that relationship physical.

You move it.

Bitwig remembers.

Then Bitwig moves it back.

Coming Next

So far, most of our navigation has treated the eight visible channels as a relatively flat bank.

But Bitwig projects can contain structures within structures:

- groups;
- instruments;
- layers;
- drum pads.

DrivenByMoss allows the X-Touch to navigate those structures too.

Next we will look at:

Groups, Layers and Drum Pads.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 17 - Groups, Layers and Drum Pads

Groups, Layers and Drum Pads

So far, we have often treated the Bitwig project as though it were a relatively simple row of tracks:

```
Track 1   Track 2   Track 3   Track 4   Track 5   Track 6   Track 7   Track 8
```

The X-Touch gives us eight channel strips, and banking lets that eight-channel window move across a larger project.

But Bitwig projects are not necessarily flat.

A track can belong to a Group.

An instrument can contain layers.

A Drum Machine can contain pads.

So the project may look more like this:

```
Project
   │
   ├── Group
   │     │
   │     ├── Track
   │     ├── Track
   │     └── Track
   │
   ├── Instrument Track
   │     │
   │     └── Instrument
   │           │
   │           ├── Layer
   │           ├── Layer
   │           └── Layer
   │
   └── Drum Track
         │
         └── Drum Machine
               │
               ├── Pad
               ├── Pad
               ├── Pad
               └── ...
```

DrivenByMoss allows the X-Touch to navigate these structures.

That introduces another important idea:

The eight channel strips can show a level of a hierarchy.

A Window into the Project

Think back to banking.

If a project contains sixteen tracks, the X-Touch cannot show all sixteen simultaneously.

Instead, it shows a window:

```
Project

1   2   3   4   5   6   7   8   9   10  11  12  13  14  15  16
└───────────────┘
     X-Touch
```

Bank to the right and the window moves.

Hierarchical navigation adds another possibility.

The window can move not only:

```
← sideways →
```

but also:

```
↓ deeper
↑ outward
```

into and out of project structures.

Flat and Hierarchical Navigation

DrivenByMoss provides two approaches to track navigation:

- flat navigation;
- hierarchical navigation.

The distinction affects what happens when the project contains Groups.

Flat Navigation

In a flat view, tracks can be presented as part of a single navigable sequence.

Conceptually:

```
Group A
   ├── Bass
   ├── Guitar
   └── Keys

Drums
Vocals
```

may be approached more like:

```
Bass   Guitar   Keys   Drums   Vocals
```

The hierarchy is not the main navigational concern.

This can be convenient when you primarily want to move rapidly across tracks.

Hierarchical Navigation

Hierarchical navigation preserves the idea that some tracks exist inside other structures.

The same project is understood as:

```
Project
    │
    ├── Group A
    │       ├── Bass
    │       ├── Guitar
    │       └── Keys
    │
    ├── Drums
    │
    └── Vocals
```

The controller can then enter Group A and expose its contents.

Conceptually:

```
Project level

┌──────────┐  ┌──────────┐  ┌──────────┐
│ Group A  │  │ Drums    │  │ Vocals   │
└──────────┘  └──────────┘  └──────────┘
      │
      │
      │ enter
      ▼

Inside Group A

┌──────────┐  ┌──────────┐  ┌──────────┐
│ Bass     │  │ Guitar   │  │ Keys     │
└──────────┘  └──────────┘  └──────────┘
```

The eight physical channel strips have not changed.

What they represent has.

Going Down a Level

When hierarchical navigation is enabled, selecting and entering a Group lets the X-Touch descend into that Group.

The channel strips can then represent its child tracks.

Think of this as opening a folder:

```
Group
  |
  | enter
  ▼
Contents of Group
```

The important conceptual distinction is between:

```
SELECT
   → establish focus
```

and:

```
ENTER
   → descend into the focused structure
```

This builds naturally on the SELECT-button principle introduced earlier in the guide.

First establish which object you mean.

Then decide what to do with it.

Coming Back Out

If we can descend into a hierarchy, we also need a way back.

DrivenByMoss provides navigation back towards the parent level.

Conceptually:

```
Project
   |
   ▼
Group
   |
   ▼
Track
```

and then:

```
Track
   |
   ▲
Group
   |
   ▲
Project
```

The important mental model is:

You are moving the X-Touch's point of view through the project hierarchy.

You are not moving the tracks themselves.

Hierarchy Is Another Kind of Banking

There is a useful connection with Chapter 4.

Ordinary banking moves the X-Touch's view across a collection:

```
← BANK                                        BANK →
```

Hierarchical navigation moves the view between levels:

```
      Parent
        ▲
        │
        │
        ▼
       Child
```

So we can think of navigation as having two dimensions:

```
                 Parent
                   ▲
                   │
Previous  ◄───── Current ─────► Next
                   │
                   ▼
                  Child
```

That is a much more powerful model than thinking of the X-Touch as permanently attached to eight particular tracks.

Why Hierarchical Navigation Helps

Consider a large project containing:

```
Drums
Bass
Guitars
Keyboards
Vocals
FX
```

where several of those are Groups.

At the top level, you may want the X-Touch to behave almost like a stem mixer:

```
Drums   Bass   Guitars   Keys   Vocals   FX
```

That is an excellent high-level view for balancing the song.

But now suppose the snare is too loud.

Instead of abandoning the control surface, you can descend into the Drums Group:

```
Drums
  │
  ▼
```

```
Kick   Snare   Hats   Toms   Percussion   ...
```

Make the adjustment.

Then return to the project level:

```
Drums   Bass   Guitars   Keys   Vocals   FX
```

The surface has changed scale.

From Mixing Desk to Magnifying Glass

This gives the X-Touch two complementary roles.

At one moment it can behave like a broad mixing desk:

```
Drums   Bass   Guitars   Keys   Vocals
```

A moment later it can act almost like a magnifying glass:

```
Kick   Snare   Hat   Tom 1   Tom 2   Shaker
```

The hardware is identical.

Only the level of detail has changed.

That is one of the advantages of a context-sensitive control surface.

Instrument Layers

Hierarchy does not stop at track Groups.

Bitwig instruments can themselves contain multiple layers.

Conceptually:

```
Instrument Track
       │
       ▼
   Instrument
       │
       ├── Layer 1
       ├── Layer 2
       ├── Layer 3
       └── Layer 4
```

DrivenByMoss can expose these layers on the X-Touch.

When the appropriate layer view is active, the channel strips cease to represent ordinary project tracks and instead represent the layers within the selected instrument.

Again:

The channel strip represents whatever object the current context assigns to it.

Layer Mode

For a layered instrument, the controller can enter a Layer-oriented view.

The eight channel strips can then represent up to eight visible layers:

```
Layer 1   Layer 2   Layer 3   Layer 4   Layer 5   Layer 6   Layer 7   Layer 8
```

If more layers exist, banking provides access to the others.

This should already feel familiar.

We are applying the same eight-channel-window model to a different collection of objects.

Mixing Layers

Once layers are exposed on the channel strips, they can be treated in mixer-like ways.

Depending on the current layer mode, the surface provides access to properties including:

• Volume;
• Panorama;
• Sends;
• Mute;
• Solo.

So a layered sound can be mixed from the X-Touch much as a set of tracks can.

Conceptually:

```
                Layered Instrument

             ┌────────┬────────┬────────┐
             │Layer 1 │Layer 2 │Layer 3 │
             └───┬────┴───┬────┴───┬────┘
                 │        │        │
              Volume   Volume   Volume
                Pan      Pan      Pan
               Sends    Sends    Sends
```

This can be much more immediate than repeatedly opening device panels and adjusting the layers with a mouse.

Volume and Panorama

The same perspective we used in Chapter 14 applies here.

We can look at layers by object:

```
Layer 1
```

```
    ├── Volume
    ├── Pan
    └── Sends
```

or by dimension:

```
Volume
    ├── Layer 1
    ├── Layer 2
    ├── Layer 3
    └── ...
```

The X-Touch can reorganise its controls according to the job.

The principles do not change merely because the objects are now layers rather than tracks.

Sends from Layers

Layer Sends can be particularly useful for building complex instruments.

Different layers may need different amounts of:

- reverb;
- delay;
- modulation;
- other send effects.

Instead of treating the layered instrument as one indivisible sound, the X-Touch can help expose its internal mix.

Conceptually:

```
Layer 1 ──────────► Reverb
Layer 2 ───►        Reverb
Layer 3 ────────────────► Reverb
Layer 4 ───────►    Reverb
```

Once again, we are mixing inside something that previously appeared to be a single track.

Mute and Solo for Layers

Mute and Solo become particularly useful when investigating a layered sound.

Suppose an instrument contains four layers and something in the combined sound is muddy.

Solo the layers individually.

Or mute one layer and listen to what disappears.

The workflow becomes:

```
Listen
```

```
  ↓
Solo / Mute Layer
  ↓
Identify contribution
  ↓
Adjust
  ↓
Restore full sound
```

This is another example of the X-Touch helping you work by ear rather than requiring constant visual navigation through Bitwig's device interface.

Drum Pads

A Drum Machine introduces another hierarchy.

Conceptually:

```
Drum Track
    │
    ▼
Drum Machine
    │
    ├── Kick Pad
    ├── Snare Pad
    ├── Closed Hat Pad
    ├── Open Hat Pad
    ├── Clap Pad
    ├── Tom Pad
    └── ...
```

From the project's top level, this may appear to be one track.

Inside the Drum Machine, however, it contains an entire collection of sounds.

DrivenByMoss can expose those pads through the X-Touch.

Drum Pad Mode

In the appropriate Drum Pad view, the channel strips represent individual pads.

For example:

```
Ch 1      Ch 2      Ch 3      Ch 4      Ch 5      Ch 6      Ch 7      Ch 8

Kick      Snare     C.Hat     O.Hat     Clap      Tom       Rim       Perc
```

Now the X-Touch behaves almost like a mixer for the internal elements of the drum kit.

The same physical faders that previously controlled entire tracks may now control individual drum sounds.

Mixing a Drum Machine

This can be extremely useful.

Suppose the overall drum track is at the correct level, but the hi-hat is too loud.

At project level:

```
Drums
  │
  └── one fader
```

does not solve the problem.

Descend to the Drum Machine's pads and the view becomes:

```
Kick   Snare   C.Hat   O.Hat   Clap   Tom   ...
```

Now the problem is directly accessible.

Turn the hi-hat down.

Then return to the higher-level mix.

Drum Pad Volume

In a volume-oriented pad view:

```
Fader 1 → Kick
Fader 2 → Snare
Fader 3 → Closed Hat
Fader 4 → Open Hat
...
```

The X-Touch becomes a conventional-looking mixer for something that Bitwig represents internally as a device.

This is another reminder that the distinction between:

```
Track
Device
Layer
Pad
```

matters less to the hardware than you might initially expect.

What matters is the current collection of controllable objects.

Drum Pad Panorama

Pads can also be treated as individual mixer elements for panorama.

That makes operations such as spreading percussion across the stereo field much more tactile.

Instead of repeatedly selecting pads on screen:

```
select pad
   ↓
adjust pan
   ↓
select next pad
   ↓
adjust pan
```

the X-Touch can present several pad panorama controls together.

Drum Pad Sends

The same applies to Sends.

Perhaps:

• the snare needs plenty of reverb;
• the kick needs almost none;
• the clap needs delay;
• the percussion needs a little of both.

The pad-oriented Send view allows these internal relationships to be mixed from the surface.

This is exactly the same mixing by dimension principle introduced in Chapter 14, now operating one level deeper.

Mute and Solo for Drum Pads

Mute and Solo are especially useful with drums.

Want to hear the groove without the kick?

Mute it.

Want to find which percussion pad is producing an unwanted sound?

Solo pads until you identify it.

Want to audition the rhythm without the hats?

Mute them.

The channel-strip buttons make these operations immediate.

Again, the X-Touch is acting not merely as a track mixer, but as a mixer for the contents of a device.

The Same Eight Strips, Again and Again

At this point, the eight channel strips may have represented:

```
Project tracks
Group contents
```

```
Instrument layers

Drum pads
```

This may sound complicated until we notice that the interaction pattern remains remarkably consistent.

The surface repeatedly asks:

What eight things are we looking at now?

Then the familiar controls operate on those things.

Conceptually:

```
                X-Touch
                   │
                   ▼
         Eight channel strips
                   │
      ┌────────────┼────────────┐
      │            │            │
      ▼            ▼            ▼
    Tracks       Layers        Pads
```

Once you understand the window, you do not need to relearn the surface every time the contents change.

SELECT Still Means Focus

The SELECT-button principle remains valuable throughout hierarchical navigation.

At whatever level you are working:

SELECT establishes focus.

At project level, SELECT may focus a track or Group.

Inside a Group, it may focus one of its child tracks.

In a layer-oriented context, it may focus a layer.

In a pad-oriented context, it may focus a drum pad.

The object changes.

The principle does not.

That consistency is what makes a complex hierarchy manageable.

ENTER Means Go Deeper

A useful companion principle is:

ENTER descends into something that has contents.

Not every selected object has another meaningful level beneath it.

But when it does, ENTER provides a natural conceptual operation:

```
SELECT
   ↓
"This one"

ENTER
   ↓
"Show me inside it"
```

That pairing is worth remembering.

CANCEL Means Back Out

Likewise, navigation needs a way to retreat from the current level.

Think of CANCEL as the complementary idea:

```
ENTER
   → go in

CANCEL
   → come back out
```

The precise behaviour still depends on the current context, but this provides a useful mental model for hierarchical navigation.

Don't Confuse Navigation with Rearrangement

There is an important distinction here.

When you enter a Group or expose layers, you are changing the controller's view.

You are not moving objects in the Bitwig project.

This is similar to the distinction introduced in Chapter 4.

Compare:

```
Navigate hierarchy
      ↓
Move the X-Touch's view
```

with:

```
Move track
      ↓
Change the project structure
```

Those are fundamentally different operations.

If something unexpected happens, ask yourself:

Am I moving the view, or moving the object?

That question resolves many apparently confusing control-surface operations.

A Practical Group Workflow

Suppose the top-level mix contains:

```
Drums   Bass   Guitars   Keys   Vocals   FX
```

and the snare needs adjustment.

1. Select Drums

Use the appropriate channel SELECT button.

2. Enter the Group

Descend into its contents.

The surface now shows something like:

```
Kick   Snare   Hats   Toms   Percussion
```

3. Adjust Snare

Use the familiar channel controls.

4. Return to the parent

Back out of the Group.

The X-Touch returns to:

```
Drums   Bass   Guitars   Keys   Vocals   FX
```

No mouse was required simply to change the scale at which you were mixing.

A Practical Drum Workflow

Suppose a Drum Machine contains:

```
Kick   Snare   Hat   Clap   Rim   Shaker   Conga   Tamb
```

and you want to shape the internal mix.

You can:

1. enter the appropriate pad view;
2. balance the pad volumes;
3. adjust panorama;
4. work with Sends;
5. mute or solo individual sounds;
6. return to the higher-level track view.

The important point is that you have not left the X-Touch's basic interaction model.

You have simply moved deeper into the project.

Hierarchy and Mouse-Lite Working

Hierarchical navigation addresses one of the common reasons for reaching for a mouse:

I need to get at something inside that thing.

Without a control-surface hierarchy, the usual response is:

```
look at screen
    ↓
find Group or device
    ↓
expand it
    ↓
find child object
    ↓
select it
```

With a navigable hierarchy:

```
SELECT
   ↓
ENTER
   ↓
work
   ↓
CANCEL
```

The exact number of steps depends on the project, but the principle is powerful.

The controller can change its point of view without requiring you to manipulate the visual interface first.

A Project Is Not Flat

This is the larger lesson of the chapter.

At the beginning of the guide, it was convenient to imagine:

```
Track 1   Track 2   Track 3   Track 4   ...
```

That model remains useful.

But a real Bitwig project may be closer to:

```
Project
   │
   ├── Group
```

```
│       ├── Track
│       └── Track
│
├── Instrument Track
│       └── Instrument
│               ├── Layer
│               └── Layer
│
└── Drum Track
        └── Drum Machine
                ├── Pad
                ├── Pad
                └── Pad
```

DrivenByMoss allows the X-Touch to follow that structure.

The Important Idea

The eight channel strips are not permanently eight tracks.

They are an eight-object window.

Depending on context, those objects may be:

```
Tracks
Groups
Child tracks
Layers
Drum pads
```

Banking moves the window across objects.

Hierarchical navigation moves the window between levels.

So our mental model can now expand again:

```
                    PROJECT

                       │
             ┌─────────┴─────────┐
             │                   │
           Group             Instrument
             │                   │
       ┌─────┴─────┐       ┌─────┴─────┐
       │           │       │           │
     Track       Track   Layer       Layer

                      Drum Machine
                           │
                    ┌──────┼──────┐
                    │      │      │
                   Pad    Pad    Pad
```

The X-Touch does not need a separate physical surface for every level.

It changes what its existing surface represents.

Once that becomes intuitive, even a deeply structured Bitwig project can remain accessible from the same eight channel strips.

Coming Next

We have now moved:

- sideways through banks;
- between mixer dimensions;
- through markers;
- into Groups;
- into layers;
- into Drum Machine pads.

The next chapter moves somewhere different again:

upwards, to the project as a whole.

We will look at the Master channel and the project-level operations that DrivenByMoss places on the X-Touch.

Next:

Master Mode and Project Control.

Chapter 18 - Master Mode and Project Control

Master Mode and Project Control

Most of the X-Touch is concerned with the tracks that make up a project.

We select them.

We bank through them.

We mix them.

We control their devices and parameters.

But Bitwig also has a level above the individual tracks.

The Master channel belongs to the project as a whole.

DrivenByMoss gives the X-Touch a specialised Master Edit Mode for working at that level.

The Master Fader Is More Than a Fader

The obvious job of the Master fader is to control the project's overall output level.

That is already useful.

But with DrivenByMoss, touching the Master fader also establishes a different context:

```
Touch Master Fader
        │
        ▼
Select Master
        │
        ▼
Enter Master Edit Mode
```

So the Master fader performs two related jobs:

• it controls the Master level;
• it gives access to project-level controls.

This is another example of a familiar Project XTC principle:

An action can both change a value and establish focus.

From Track Focus to Project Focus

Earlier chapters concentrated on questions such as:

Which track am I controlling?

Master Mode asks a different question:

What if the thing I want to control belongs to the whole project?

Conceptually:

```
Track Mode
   │
   ▼
Track-specific controls


Master Mode
   │
   ▼
Project-level controls
```

The physical V-Pots are the same.

What they represent has changed.

Entering Master Edit Mode

Touch the Master fader.

DrivenByMoss selects the Master track and enters Master Edit Mode.

The V-Pots then acquire Master/project-related assignments.

At a high level:

```
Master Edit Mode

V-Pot 1   → Master Volume
V-Pot 2   → Master Panorama
V-Pots 3–5 → Project / audio-engine controls
V-Pot 7   → Previous Project
V-Pot 8   → Next Project
```

The important point is not merely the list.

The important point is that the X-Touch has moved from:

```
one track in the project
```

to:

```
the project itself
```

V-Pot 1 — Master Volume

V-Pot 1 controls the Master volume.

The Master fader already provides a large physical control for the same parameter, so this may seem redundant.

But redundancy is not necessarily wasteful on a control surface.

It gives the same parameter another form of access within the current mode.

Pressing V-Pot 1 resets the Master volume to its default value.

Conceptually:

```
Turn V-Pot 1
      ↓
Master Volume

Press V-Pot 1
      ↓
Reset Master Volume
```

This follows the V-Pot press behaviour introduced earlier in the guide.

V-Pot 2 — Master Panorama

V-Pot 2 controls the Master panorama.

Pressing it resets the panorama to its default position.

So:

```
Turn V-Pot 2
      ↓
Master Panorama

Press V-Pot 2
      ↓
Reset Panorama
```

This gives the Master channel the same sort of direct physical access we have already used on ordinary tracks.

Master Control and Caution

Master controls deserve a little more care than ordinary track controls.

Changing a single track affects one element of the mix.

Changing the Master affects everything downstream of it.

That means accidental adjustments can have much wider consequences.

A good habit is therefore:

Know that you are in Master Mode before turning controls.

The displays and current feedback should make that clear.

Again:

Observe before you adjust.

V-Pots 3–5 — Project-Level Controls

DrivenByMoss assigns V-Pots 3–5 to project-level audio-engine control functions.

These are operated by pressing the V-Pots rather than turning them.

The important conceptual point is that these controls affect the state of the project/audio engine, not an individual channel parameter.

So the surface now contains several different kinds of control at once:

```
V-Pot 1
   → continuous Master parameter

V-Pot 2
   → continuous Master parameter

V-Pots 3–5
   → project/audio-engine actions
```

This is another example of the controller being organised around context rather than around one fixed type of control.

Why Project-Level Controls Matter

At first glance, project-level operations may seem less important than faders or device parameters.

But they become useful precisely because they save another trip to the computer interface.

Instead of thinking:

```
find project control on screen
       ↓
move mouse
       ↓
click
```

the X-Touch can expose the relevant action directly when Master Mode is active.

That is a small but useful step towards a Mouse-Lite workflow.

V-Pot 7 — Previous Project

Press V-Pot 7 to move to the previous project.

Conceptually:

```
Current Project
      │
      │ press V-Pot 7
      ▼
Previous Project
```

This is a surprisingly powerful operation to have on the control surface.

It means project navigation itself can become part of the hardware workflow.

V-Pot 8 — Next Project

Press V-Pot 8 to move to the next project.

So V-Pots 7 and 8 form a natural pair:

```
V-Pot 7
   ← Previous Project

V-Pot 8
   Next Project →
```

This is easy to remember because the spatial relationship mirrors the navigation.

Left control: previous.

Right control: next.

Project Switching from the Surface

Imagine working through several related Bitwig projects.

Perhaps they are:

- different songs in a set;
- alternative versions;
- works in progress;
- test projects.

Instead of opening project-selection controls with the mouse, Master Mode can provide direct navigation.

The workflow becomes:

```
Finish with Project A
        │
        ▼
Touch Master Fader
```

```
        │
        ▼
Master Edit Mode
        │
        ▼
Press V-Pot 8
        │
        ▼
Project B
```

That is a good example of a command that is not musically glamorous but can make the overall workflow feel much more continuous.

The Master Fader and Metronome Volume

We have already encountered another useful Master-fader modifier:

```
SHIFT + Master Fader
```

controls metronome volume.

That is not part of Master Edit Mode itself, but it is worth remembering because it demonstrates how many roles the Master area can acquire:

```
Master Fader
     │
     ├── normal
     │      Master volume
     │
     ├── touch
     │      Master focus / Master Mode
     │
     └── SHIFT
            Metronome volume
```

One physical fader participates in several related workflows.

Master Mode and Context

Suppose you were just working in Device Mode.

The V-Pots represented device parameters.

Then you touch the Master fader.

Now those same V-Pots represent project-level functions.

Conceptually:

```
Device Mode
   │
   ▼
V-Pots = Device Parameters

Touch Master Fader

   │
```

```
   ▼

Master Mode
   │
   ▼
V-Pots = Master / Project Controls
```

Nothing about the hardware has changed.

The current focus has changed.

This is precisely the mental model Project XTC has been building throughout the guide.

The Master Channel Is a Different Scale

There is a useful way to think about this.

Ordinary track control is local:

```
Track
   ↓
its level
its pan
its sends
its devices
```

Master control is global:

```
Project
   ↓
overall output
overall panorama
project state
project navigation
```

The X-Touch lets you change scale.

You can work on:

```
one parameter
```

then:

```
one track
```

then:

```
one Group
```

then:

```
the entire project
```

without changing control surfaces.

Working from the Top Down

Master Mode is particularly useful when you want to start from the project level and then descend.

For example:

```
Master level
     ↓
Group balance
     ↓
Track balance
     ↓
Device parameter
```

Earlier chapters showed how the X-Touch can move deeper into a project.

Master Mode reminds us that we can also move upwards.

The controller's point of view can operate at several levels.

The Hierarchy Keeps Expanding

At this point, our project hierarchy looks something like:

```
Project
   │
   ├── Master
   │
   ├── Groups
   │     └── Tracks
   │
   ├── Tracks
   │     └── Devices
   │           └── Parameters
   │
   └── Instruments
         └── Layers / Drum Pads
```

The X-Touch does not expose all of this at once.

Instead, it exposes the level that is currently useful.

That is what keeps the surface manageable.

Project Control Is Still Feedback-Driven

Master Mode is another context where displays matter.

When the V-Pots no longer represent ordinary mixer or device parameters, the feedback tells you what they now mean.

That is especially important for project-level actions.

Do not assume that a V-Pot is still doing what it did moments ago.

Read the controller.

Check the context.

Then act.

The same rule continues to pay off:

Observe before you adjust.

A Practical Master Workflow

A simple Master Mode workflow might look like this.

1. Touch the Master fader

This establishes Master focus and enters Master Edit Mode.

2. Check the displays

Confirm that the controller is showing the Master/project context.

3. Adjust Master level if required

Use the Master fader or the appropriate V-Pot.

4. Adjust Master panorama if required

Use V-Pot 2.

5. Use project-level actions only deliberately

Treat V-Pots 3–5 as actions rather than ordinary continuous controls.

6. Navigate projects if required

Use V-Pots 7 and 8.

7. Return to normal track work

Select the required track or mode and continue.

The important thing is the shift in scale.

You temporarily step out of the track-level view, work at the project level, then return.

Project Navigation and Mouse-Lite Working

Project switching is another good example of Mouse-Lite philosophy.

The goal is not:

Never use Bitwig's project interface.

The goal is:

If you already know you want the previous or next project, why should that necessarily require the mouse?

The X-Touch gives you a direct physical route.

That is the recurring theme of this guide:

use the physical surface when it shortens the path between intention and action.

A Note on Verification

Some project-level assignments are less obvious than ordinary mixer functions.

For that reason, Project XTC should verify the exact current DrivenByMoss behaviour of the Master Mode action controls before final publication.

In particular, the functions assigned to V-Pots 3–5 should be confirmed against the current DrivenByMoss version rather than inferred from the labels alone.

The general Master Mode structure is clear.

The final reference tables should contain only verified current behaviour.

The Important Idea

Master Mode changes the X-Touch's point of view from:

one part of the project

to:

the project as a whole.

Touching the Master fader establishes that context.

The surface can then provide access to:

- Master volume;
- Master panorama;
- project/audio-engine controls;
- previous project;
- next project.

So our mental model expands once more:

```
Physical Control
      +
Current Focus
      +
```

```
Current Mode
      =
Current Function
```

At track level, the focus is a track.

At Device level, it is a device.

Inside a Group, it may be a child track.

In Master Mode, it is the project-level Master context.

The same surface moves between all of them.

Coming Next

So far, we have looked at how to navigate, mix, automate and control increasingly complex structures.

The next chapter turns to another fundamental workflow:

recording.

DrivenByMoss gives the X-Touch several ways to work with:

• Arranger recording;
• Launcher overdub;
• clip creation;
• New Clip Length;
• overdub controls.

Next:

Advanced Recording and Overdub.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 19 - Advanced Recording and Overdub

Advanced Recording and Overdub

The RECORD button looks simple enough.

Press it, and Bitwig records.

But Bitwig has more than one recording environment.

It has the linear Arranger.

It has the clip-based Launcher.

It can record new material.

It can overdub onto existing material.

And DrivenByMoss gives the X-Touch access to several of these operations without requiring us to reach for the mouse.

The important question is therefore not simply:

How do I record?

It is:

What do I want Bitwig to record, and where?

Two Recording Worlds

Bitwig gives us two related but distinct ways of organising music.

The Arranger

The Arranger is based around a linear timeline:

Music has a position in time.

Recording into the Arranger creates material along that timeline.

The Launcher

The Launcher is based around clips that can be triggered independently:

```
          Scene 1    Scene 2    Scene 3

Track 1   [ Clip ]   [ Clip ]   [ Clip ]
Track 2   [ Clip ]   [ Clip ]   [ Clip ]
Track 3   [ Clip ]   [ Clip ]   [ Clip ]
```

Here the important object is not merely a timeline position.

It is a clip.

That difference matters when we start talking about recording and overdubbing.

The RECORD Button

The X-Touch's RECORD button provides the main recording operation.

At its simplest:

```
RECORD
   ↓
Recording
```

But, as with so many X-Touch controls, modifiers give the button additional meanings.

DrivenByMoss can use the same physical control for related recording operations.

The button therefore belongs to a family of actions rather than one isolated command.

Recording Starts with Track ARM

Before recording musical input, the appropriate track normally needs to be armed.

The channel-strip ARM buttons provide direct access to that state.

Conceptually:

```
Choose track
    ↓
ARM
    ↓
Track ready to record
    ↓
RECORD
```

This is an important distinction.

ARM answers:

Which track should receive the input?

RECORD answers:

Should Bitwig record now?

Keeping those two ideas separate makes recording behaviour much easier to understand.

Arranger Recording

For conventional linear recording, the workflow is straightforward.

1. Select the track

Establish focus on the track you want to record.

2. Arm it

Press the corresponding ARM button.

3. Position the transport

Use the transport controls, Jog Wheel or markers.

4. Start recording

Press RECORD.

5. Perform

Bitwig records into the Arranger.

Conceptually:

```
Position
   ↓
ARM
   ↓
RECORD
   ↓
Perform
   ↓
Arranger material
```

The X-Touch therefore provides the essential physical controls for a conventional recording pass.

Recording and Playback Are Related

Recording does not exist separately from transport.

Before recording, we may need to:

- locate the start position;
- set the loop;
- enable the metronome;
- configure a count-in;
- arm a track.

During recording, we may need to:

- monitor the transport;
- adjust a performance;
- stop;
- restart.

Afterwards, we need to:

- return;
- listen;
- record another pass.

So the recording workflow combines several ideas we have already learned:

```
Navigation
    +
Transport
    +
Track focus
    +
ARM
    +
RECORD
    =
Recording workflow
```

This is why learning the controller as a system is more useful than memorising individual buttons.

Overdub Is Not the Same as Record

The word overdub means that existing material remains while new material is added.

Conceptually:

```
Existing material
       +
New performance
       =
Combined material
```

That is different from simply recording a new passage.

The distinction becomes especially important with MIDI and Launcher clips.

Arranger Overdub

Bitwig can overdub note data into existing Arranger material.

Instead of replacing the existing notes, the new performance is added.

For example, suppose a MIDI clip already contains:

```
Kick        ●       ●       ●       ●
Snare           ●               ●
```

An overdub pass might add:

```
Hat         x   x   x   x   x   x   x   x
```

giving:

```
Kick        ●       ●       ●       ●
Snare           ●               ●
Hat         x   x   x   x   x   x   x   x
```

The original performance remains.

The new one becomes part of it.

Why Overdub Is Useful

Overdub encourages a layered way of working.

A drum pattern might be built as:

```
Pass 1
   ↓
Kick and Snare

Pass 2
   ↓
Hi-Hats

Pass 3
   ↓
Percussion

Pass 4
   ↓
Extra hits
```

Likewise, a MIDI instrument performance might begin with chords and later acquire additional notes.

The controller allows recording to become an iterative process rather than a one-shot operation.

Launcher Overdub

The Launcher introduces its own overdub workflow.

Instead of thinking about the Arranger timeline, we are working with a clip.

Conceptually:

```
Launcher Clip
      │
      ▼
Existing Notes
      │
      │ overdub
      ▼
Existing Notes
      +
New Notes
```

This is particularly useful for loop-based composition.

Start with a simple idea.

Let it repeat.

Add another part.

Let it repeat again.

Add another.

The clip develops while playback continues.

OVR

The X-Touch's OVR control participates in the overdub workflow.

DrivenByMoss maps the MCU controls onto Bitwig's available recording and overdub functions.

As elsewhere in this guide, the printed MCU label should not be treated as a guarantee that Bitwig has an identically named internal function.

The useful question is always:

What Bitwig recording state is this control changing?

The final Quick Reference will give the verified mapping for the DrivenByMoss version covered by Project XTC.

Arranger and Launcher Overdub Are Different States

It is worth keeping these conceptually separate.

```
Arranger Overdub
       │
       ▼
Add material to existing
Arranger note content
```

versus:

```
Launcher Overdub
       │
       ▼
Add material to an
existing Launcher clip
```

Both involve adding rather than replacing.

But they act in different recording environments.

When something appears not to be overdubbing as expected, one of the first questions should therefore be:

Am I working in the Arranger or the Launcher?

OPTION + RECORD

DrivenByMoss gives OPTION + RECORD an additional recording-related function.

This is one of the advanced combinations identified during the MCU feature audit.

Rather than treating it as an isolated shortcut, place it in the broader model:

```
RECORD
    │
    ├── normal
    │      primary recording operation
    │
    └── OPTION
           related advanced
           recording operation
```

The exact current behaviour should be verified against the DrivenByMoss version used for final publication before it is placed in the Quick Reference.

This is preferable to inferring behaviour merely from older MCU conventions or the label printed on the controller.

Creating a New Clip

Launcher recording introduces another practical problem.

What happens if there is no clip yet?

Before we can overdub into a clip, there needs to be a clip to receive the material.

DrivenByMoss provides control-surface operations for creating clips.

Conceptually:

```
Empty slot
    │
    ▼
Create Clip
    │
    ▼
New empty clip
    │
    ▼
Record / Overdub
```

This allows the Launcher workflow to begin from the X-Touch rather than requiring the initial clip to be created with the mouse.

New Clip Length

If the controller creates a new clip, another question immediately appears:

How long should it be?

DrivenByMoss provides a New Clip Length setting.

For example, a new clip might be:

```
1 bar
2 bars
4 bars
8 bars
```

depending on the chosen configuration.

Conceptually:

```
New Clip Length
       │
       ▼
Create Clip
       │
       ▼
Clip of configured length
```

This is an important example of the relationship between configuration and performance.

During a session, you want clip creation to be immediate.

Before the session, you decide what behaviour will usually be most useful.

Configuration Removes Decisions from Performance

Suppose you normally build four-bar loops.

If New Clip Length is already configured appropriately, the creative workflow can become:

```
Create
```

```
   ↓
Record
   ↓
Loop
   ↓
Overdub
```

rather than:

```
Create
   ↓
Specify length
   ↓
Confirm
   ↓
Record
```

A good configuration removes repetitive decisions from the musical moment.

We will return to this principle in Chapter 21.

Building a Clip in Layers

A Launcher workflow might look like this:

Pass 1 — Foundation

Record the basic idea.

```
Kick + Snare
```

Pass 2 — Add movement

Enable the appropriate overdub state and add:

```
Hi-Hats
```

Pass 3 — Add detail

Add:

```
Percussion
```

Pass 4 — Listen

Let the clip repeat without playing.

Pass 5 — Correct or extend

Add only what the pattern needs.

This turns recording into a conversation with the loop.

Recording Without Stopping the Idea

One of the attractions of overdub-based working is continuity.

Instead of:

```
record
  ↓
stop
  ↓
edit
  ↓
record
  ↓
stop
```

the process can become:

```
play
  ↓
record
  ↓
overdub
  ↓
listen
  ↓
overdub
  ↓
listen
```

The musical idea keeps moving.

For some styles of composition, that can be much more productive.

Looping and Recording

The loop controls introduced earlier become especially useful during recording.

Set a region.

Let it repeat.

Perform another pass.

Conceptually:

```
Loop Region
┌─────────────────────┐
│                     │
└─────────────────────┘

        ▲
        │
    repeats

Pass 1 → Pass 2 → Pass 3 → ...
```

Combined with overdub, the X-Touch can support a workflow in which a section develops over successive repetitions.

Recording by Ear

There is a recurring theme here.

A screen-based recording workflow can easily become:

```
find control
    ↓
click
    ↓
watch cursor
    ↓
find another control
```

A well-configured control surface allows more of the process to become:

```
listen
   ↓
ARM
   ↓
RECORD
   ↓
perform
   ↓
listen
```

The screen remains useful.

But it does not have to mediate every decision.

Recording and Markers

Markers from Chapter 15 can also become part of the recording workflow.

Suppose a vocal needs replacing in the second chorus.

Instead of searching visually for the section:

```
OPTION + FORWARD
       ↓
Second Chorus marker
       ↓
ARM
       ↓
RECORD
```

Navigation and recording now form one physical workflow.

This is where apparently separate controller features begin to reinforce one another.

Recording and Groups

The hierarchy from Chapter 17 can also matter.

A large project may contain a Group of recording tracks.

You can navigate into the Group, select the required track, arm it and record.

Conceptually:

```
Project
   ↓
Recording Group
   ↓
Track
   ↓
ARM
   ↓
RECORD
```

Again, the individual commands matter less than the fact that they connect into a sequence.

Recording and the Metronome

The Master fader also participates in the recording environment.

As noted earlier:

```
SHIFT + Master Fader
```

controls metronome volume.

This is a particularly good example of a control-surface feature whose usefulness becomes obvious in context.

During recording you may want the click:

- loud enough to perform accurately;
- quiet enough not to dominate;
- changed without opening another Bitwig panel.

The modified Master fader gives you a direct physical control.

Undo Is Part of Recording

Not every take is worth keeping.

That is not a failure of the workflow.

It is part of recording.

A practical control-surface workflow should therefore include the ability to:

```
Record
   ↓
Listen
   ↓
No
   ↓
UNDO
   ↓
```

```
Try again
```

Undo is not merely an editing command.

During performance-oriented recording, it can become part of the creative cycle.

The Value of Immediate Recovery

The faster you can recover from a bad take, the less disruptive the mistake becomes.

Compare:

```
bad take
   ↓
stop
   ↓
reach for mouse
   ↓
find edit
   ↓
undo
   ↓
reposition
   ↓
start again
```

with:

```
bad take
   ↓
stop
   ↓
UNDO
   ↓
start again
```

That difference may amount to only a few seconds.

Musically, it can be much larger.

The second workflow is more likely to preserve momentum.

Clip Based Looper

DrivenByMoss also includes Clip Based Looper functionality.

This extends the idea of creating and building clips from the control surface into a more specialised looping workflow.

It is worth knowing that the functionality exists.

However, it should not be confused with the basic recording and overdub concepts in this chapter.

The core progression remains:

```
Create Clip
    ↓
Record
    ↓
Loop
    ↓
Overdub
```

Clip Based Looper builds a more specialised workflow on top of those ideas.

Because it is configurable and goes beyond the normal everyday MCU workflow, we will return to it in Chapter 22.

Don't Learn Recording as a Button Table

The X-Touch provides several recording-related controls.

It would be possible to summarise them immediately as a table of button combinations.

But that would hide the important distinctions.

Instead, ask:

Where am I recording?

```
Arranger
or
Launcher?
```

What am I doing?

```
New recording
or
Overdub?
```

What is the destination?

```
Timeline
or
Clip?
```

Does the destination already exist?

```
Existing clip
or
New clip?
```

Once those questions are answered, the relevant control becomes much easier to understand.

A Recording Decision Tree

A useful mental model is:

```
                RECORD SOMETHING
                       │
           ┌───────────┴───────────┐
           │                       │
       Arranger                Launcher
           │                       │
      ┌────┴────┐             ┌────┴────┐
      │         │             │         │
    New      Overdub       New Clip  Existing Clip
                                │         │
                                ▼         ▼
                             Record    Overdub
```

You do not need to consciously follow this diagram every time you press RECORD.

Its purpose is to separate several operations that otherwise all look like "recording".

A Practical Arranger Workflow

Suppose you want to record a guitar part.

1. Navigate to the section

Use transport or markers.

2. Select the guitar track

Establish focus.

3. Arm the track

Press ARM.

4. Set the metronome level if necessary

Use the available X-Touch control.

5. Press RECORD

Perform the part.

6. Stop and listen

Use the transport controls.

7. Undo if necessary

Then try again.

This is conventional recording, but most of the mechanical process can remain on the control surface.

A Practical Launcher Workflow

Suppose instead that you are building a rhythmic idea in the Launcher.

1. Choose the track

Select and arm it.

2. Create or select the destination clip

Use the appropriate clip workflow.

3. Record the first layer

Establish the foundation.

4. Enable the appropriate overdub state

Keep the existing material.

5. Add another layer

Let the clip continue looping.

6. Listen

Do nothing for a pass if necessary.

7. Add only what is needed

The controller becomes part of an iterative composition process.

A Practical Dub-Oriented Example

There is another interesting possibility.

Imagine building a rhythmic foundation in the Launcher:

```
Drums
Bass
Skank
Percussion
```

Once the clips are established, the X-Touch can move naturally from recording surface to mixing surface.

The same hardware that helped create the material can then provide:

- fader rides;
- mutes;
- Send levels;
- delay throws;
- reverb changes.

In other words:

```
Build the material
        ↓
Arrange the controls
        ↓
Perform the mix
```

We are going to return to this idea properly in Chapter 20.

There, the X-Touch gets to behave rather more like a traditional dub mixing desk.

The Important Idea

Advanced recording becomes much easier to understand once we stop treating RECORD as one universal operation.

Ask four questions:

```
WHERE?
   Arranger or Launcher?

WHAT?
   New recording or overdub?

INTO WHAT?
   Timeline or clip?

WHAT ALREADY EXISTS?
   Nothing or existing material?
```

Then the X-Touch controls fit into a meaningful workflow.

The important concepts are:

```
ARM
   → choose what receives input

RECORD
   → perform the primary recording operation

Overdub
   → add to existing material

New Clip Length
   → determine the size of newly created clips

Clip creation
   → establish a Launcher destination
```

```
OPTION + RECORD
   → advanced recording-related operation
     to be verified for the target version
```

The aim is not to memorise a larger collection of recording commands.

It is to make recording from the X-Touch feel like a connected process:

```
Navigate
   ↓
Arm
   ↓
Record
   ↓
Perform
   ↓
Listen
   ↓
Overdub or Undo
   ↓
Continue
```

The less machinery you have to think about, the more attention remains available for the performance.

End of Part III

We have now gone considerably deeper into what DrivenByMoss makes possible on the X-Touch.

We have explored:

- Mixer Edit Modes;
- Markers and structural navigation;
- Automation;
- Groups and hierarchical navigation;
- Instrument Layers;
- Drum Pads;
- Master Mode;
- project-level control;
- advanced recording;
- overdubbing.

Individually, these are useful features.

But Project XTC was never intended to become merely a better-organised list of features.

The real question is:

What happens when we put them together?

That is the purpose of Part IV.

Part IV — Building the Workflow

The next chapter brings together everything we have learned so far.

Transport.

Faders.

V-Pots.

Modes.

Modifiers.

Devices.

Browser.

Markers.

Automation.

Recording.

And, yes, Sends.

It is time to stop thinking about what each individual control can do and start thinking about what we can do with the whole surface.

Next:

Towards a Mouse-Free (or Mouse-Lite) Workflow.

Chapter 20 - Towards a Mouse-Free (or Mouse-Lite) Workflow

Towards a Mouse-Free (or Mouse-Lite) Workflow

We now know that the X-Touch can do rather a lot.

It can:

- navigate tracks and banks;
- select and control devices;
- browse for devices and presets;
- control volume, panorama and Sends;
- navigate markers;
- enter Groups;
- work with layers and drum pads;
- control automation;
- record and overdub;
- operate at Master and project level.

It would therefore be tempting to turn all of this into a challenge:

Can we use Bitwig without touching the mouse at all?

Perhaps.

But that is not really the most useful question.

A better one is:

When is the X-Touch a better way of expressing what I want to do?

That leads us away from the idea of a strictly mouse-free workflow and towards something more practical:

a Mouse-Lite workflow.

The Mouse Is Not the Enemy

There are things a mouse does extremely well.

If you need to:

- draw a detailed automation curve;
- edit individual MIDI notes;
- make a precise graphical selection;
- rearrange complicated regions;
- configure an unfamiliar device;
- perform detailed housekeeping;

Bitwig's graphical interface may be exactly the right tool.

There is no prize for refusing to use it.

The problem arises when reaching for the mouse becomes automatic.

```
Want to change something
        │
        ▼
Reach for mouse
        │
        ▼
Find pointer
        │
        ▼
Find object
        │
        ▼
Click
```

Sometimes that is appropriate.

Sometimes the X-Touch already has the answer beneath your fingers.

From Commands to Intentions

At the beginning of this guide, we learned individual operations.

```
Move fader
Press SELECT
Turn V-Pot
Press PLAY
```

Then we learned modes and modifiers.

Now we can stop thinking primarily in terms of commands.

Instead, start with the intention:

What am I trying to accomplish?

For example:

```
"The vocal is too loud."
```

That intention may translate directly into:

```
Find vocal channel
       ↓
Move fader
```

Or:

```
"The delay needs more feedback."
```

may become:

```
Select delay track/device
       ↓
Device Mode
       ↓
Find parameter
       ↓
Turn V-Pot
```

The controller becomes useful when the physical operation follows naturally from the musical thought.

The Shortest Useful Path

A Mouse-Lite workflow is not necessarily the workflow with the fewest button presses.

It is the workflow with the least unnecessary interruption.

Compare:

```
Hear problem
    ↓
Look at screen
    ↓
Find pointer
    ↓
Find track
    ↓
Find parameter
    ↓
Adjust
```

with:

```
Hear problem
    ↓
Touch control
    ↓
Adjust
```

The second route may allow your attention to remain on the sound.

That is the real advantage.

Learn Locations, Not Lists

One reason hardware becomes fast is that the body learns where things are.

After enough use, you do not think:

The PLAY button is located at this coordinate on the controller.

Your hand simply goes there.

The same can become true of:

- MUTE;
- SOLO;
- SELECT;
- ARM;
- SEND;
- DEVICE;
- BANK;
- CHANNEL;
- modifiers;
- transport controls.

This is different from using a graphical interface, where controls may move, disappear, scroll out of view or belong to another window.

Physical controls have geography.

That geography can become muscle memory.

Keep Your Eyes on the Feedback

Mouse-Lite does not mean screen-blind.

The X-Touch itself provides displays, LEDs, motor positions and V-Pot rings.

Bitwig provides additional feedback on the computer.

Use both.

The important change is that looking does not always have to be followed by pointing.

A useful pattern is:

```
Look
  ↓
Understand context
  ↓
Operate physically
  ↓
Listen
```

rather than:

```
Look
  ↓
Find pointer
  ↓
Point
```

```
  ↓
Click
  ↓
Look again
```

Build Workflows from Small Habits

Trying to become "mouse-free" overnight would be frustrating.

Instead, identify operations that you perform repeatedly.

Perhaps:

```
Play / Stop

Track selection

Volume

Pan

Mute / Solo

Banking
```

Use the X-Touch for those until they become automatic.

Then add:

```
Sends

Device Mode

Markers

Automation

Browser
```

Eventually, several small habits connect into complete workflows.

Workflow 1 — Navigating a Project

Suppose you want to listen to the chorus.

A screen-oriented workflow might involve finding the chorus visually and clicking the timeline.

But if the project has useful markers:

```
OPTION + FORWARD
       ↓
Next Marker
       ↓
OPTION + FORWARD
       ↓
Chorus
       ↓
```

```
PLAY
```

You are navigating the structure of the music, not pixels on a timeline.

That distinction becomes increasingly valuable as projects grow.

Add Markers While You Work

If you repeatedly visit a position, give it a marker.

For example:

```
Intro
Verse
Chorus
Breakdown
Outro
```

or more task-specific landmarks:

```
Vocal problem
Bass edit
Big transition
Mix check
```

The project gradually acquires a map that the X-Touch can navigate.

The mouse is not being eliminated.

The need to search visually is being reduced.

Workflow 2 — Moving from the Mix into Detail

Suppose you are balancing the top-level project:

```
Drums   Bass   Guitars   Keys   Vocals   FX
```

Something is wrong with the snare.

Instead of opening the Group on screen:

```
SELECT Drums
      ↓
ENTER
      ↓
Kick   Snare   Hats   Toms   Percussion
      ↓
Adjust Snare
      ↓
CANCEL
      ↓
Drums   Bass   Guitars   Keys   Vocals   FX
```

The X-Touch has effectively changed magnification.

You moved:

```
Project
   ↓
```

```
Group
   ↓
Child Track
```

made the adjustment, and came back out.

This is hierarchical navigation becoming a workflow rather than an abstract feature.

Workflow 3 — Working with a Device

Suppose a synth needs adjustment.

The process might be:

```
SELECT Track
      ↓
DEVICE
      ↓
Choose Device
      ↓
Choose Parameter Page
      ↓
Turn V-Pot
```

If the parameter would be more naturally performed with a fader, FLIP may give you another physical approach.

The important thing is that the X-Touch can move from:

```
Track
```

to:

```
Device
```

to:

```
Parameter
```

without requiring every stage to be performed graphically.

Browser When You Need Something New

Sometimes the required device is not already there.

That is where Browser Mode enters the workflow.

Conceptually:

```
Need something
     ↓
Browser Mode
     ↓
Navigate choices
     ↓
Select
     ↓
```

```
Return to work
```

Again, the goal is not to prove that the Bitwig Browser can be avoided.

It is to prevent the creative process from unnecessarily turning into a mouse-navigation exercise.

Workflow 4 — Recording a Performance

A recording workflow can connect many of the controls we have already learned.

```
Navigate to position
       ↓
SELECT Track
       ↓
ARM
       ↓
Set metronome if needed
       ↓
RECORD
       ↓
Perform
       ↓
STOP
       ↓
Listen
```

If the take is wrong:

```
UNDO
  ↓
Try again
```

If it is nearly right, another pass or overdub may be more appropriate.

The controller begins to support the cycle of recording, rather than merely the moment when RECORD is pressed.

Workflow 5 — Performing Automation

Suppose a track needs a level ride.

Rather than drawing the automation first:

```
Choose automation mode
       ↓
PLAY
       ↓
Listen
       ↓
Touch fader
       ↓
Perform movement
       ↓
Release
       ↓
Listen again
```

The automation is created as a musical gesture.

If it needs correction, perform another pass.

This is one of the clearest examples of the X-Touch offering something fundamentally different from a mouse.

A mouse edits a line extremely well.

A fader performs a movement extremely well.

Use each where it makes sense.

Workflow 6 — The X-Touch as a Dub Mixing Desk

Now for an example that brings many of these ideas together.

A traditional dub mix is not simply a set of static mixer settings.

The mix itself becomes a performance.

Faders move.

Channels disappear and return.

Sounds are thrown into delays.

Reverbs bloom and vanish.

Effects become rhythmic events.

That makes dub an unusually good demonstration of what a physical control surface can offer.

Set Up the Project

Imagine a simplified arrangement containing:

```
Drums
Bass
Skank
Organ
Percussion
Vocal
FX 1
FX 2
```

And suppose the project has two Sends:

```
Send 1 → Reverb
```

```
Send 2 → Delay
```

The exact effects do not matter.

The important point is that the tracks can be fed into them independently.

Start with the Faders

In the normal mixer view, the eight motor faders give us:

```
Drums    Bass   Skank    Organ   Perc   Vocal    FX 1   FX 2
  │        │      │        │       │      │        │      │
  ▼        ▼      ▼        ▼       ▼      ▼        ▼      ▼
█████    █████  █████    █████  █████  █████    █████  █████
```

Now the mix is physical.

You can ride several elements at once.

You can bring the organ forward.

Pull the vocal back.

Drop the percussion.

Return the drums.

These are not eight separate editing operations.

They can be part of one continuous performance.

Mutes Become Musical Controls

MUTE does not have to be merely corrective.

In a dub mix it can become rhythmic.

For example:

```
Full Mix
   ↓
Mute Drums
   ↓
Bass + Skank + Vocal
   ↓
Return Drums
   ↓
Mute Vocal
   ↓
Instrumental passage
```

The mixer becomes part of the arrangement.

You are deciding in real time what the listener hears.

Move to the Reverb Send

Press SEND to expose the first Send.

Suppose Send 1 is the reverb.

Now the V-Pots provide something like:

```
                 REVERB

Drums    Bass    Skank    Organ    Perc    Vocal    FX 1    FX 2
  |        |       |        |       |        |        |       |
  ▼        ▼       ▼        ▼       ▼        ▼        ▼       ▼
 Send     Send    Send     Send    Send    Send     Send    Send
```

The controller is no longer primarily asking:

How loud is each track?

It is asking:

How much of each track should enter the reverb?

That is a completely different view of the same mix.

Perform the Reverb

Perhaps the vocal reaches the end of a phrase.

Turn its Send up.

Let the phrase bloom into the reverb.

Then bring the Send back down.

Or send a snare hit heavily into the reverb while leaving the kick comparatively dry.

The effect is not merely configured.

It is played.

Move to the Delay

Press SEND again to reach the delay Send.

Now the same eight V-Pots answer:

How much of each track should enter the delay?

This is where things become particularly entertaining.

A vocal phrase ends:

```
"...into the night"
          |
          ▼
Raise Vocal Send
          |
          ▼
"...night... night... night..."
```

Then pull the Send back.

The delay throw becomes a performance gesture.

Send the Unexpected Things

Dub mixing becomes interesting when effects are not treated merely as polite ambience.

Try sending:

```
Snare       → Delay

Organ stab  → Delay

Percussion  → Reverb

Vocal word  → Delay
```

then pulling the Send away again.

The effect becomes an event.

And because the X-Touch exposes several Send levels simultaneously, you can react to the music rather than repeatedly opening individual track controls.

Faders + Mutes + Sends

Now combine the operations.

For example:

```
Drop Organ fader
       ↓
Mute Vocal
       ↓
Throw Snare into Delay
       ↓
Bring Organ back
       ↓
Increase Percussion Reverb
       ↓
Return Vocal
       ↓
Drop Drums
       ↓
Bass continues alone
       ↓
Drums return
```

At this point you are no longer merely "operating Bitwig".

You are performing the mix.

Use the Surface Like an Instrument

This is the important change of perspective.

A conventional editing mindset says:

```
Set this value.

Now set that value.

Now change another value.
```

A performance mindset says:

```
Listen.

Move.

React.

Anticipate.

Return.
```

The X-Touch is particularly good at the second kind of interaction because several controls are physically available at once.

You have two hands.

Use them.

Capture the Dub Performance with Automation

There is another possibility.

The performance does not have to disappear when playback stops.

Enable the appropriate automation-writing mode and Bitwig can capture the movements.

Conceptually:

```
PLAY
  ↓
Perform Mix
  │
  ├── Fader rides
  ├── Send movements
  └── other automated parameters
  ↓
STOP
  ↓
Bitwig remembers
```

Now play the section again.

The motor faders reproduce the movements you performed.

The live mix has become automation.

Perform First, Edit Later

The first pass does not need to be perfect.

That is important.

Try:

```
Perform
   ↓
Listen
   ↓
Keep what works
   ↓
Correct what doesn't
```

If one fader move is late, correct it.

If one Send stays open too long, fix it.

If a particular transition works brilliantly, leave it alone.

The control surface and graphical editor complement one another.

The surface captures the gesture.

The GUI can provide surgical correction afterwards.

That is Mouse-Lite working at its best.

A Dub Pass Might Look Like This

Imagine an eight-bar passage.

Bars 1–2

Full rhythm section.

Vocal present.

Moderate reverb.

Bar 3

Mute vocal after the final word.

Throw that word into the delay.

Bar 4

Drop drums.

Leave bass and delay repeats.

Bar 5

Return drums.

Bring organ up.

Bar 6

Push percussion into reverb.

Bar 7

Drop organ and percussion.

Return vocal.

Bar 8

Delay the final vocal phrase and pull several faders down into the transition.

None of this requires a complicated theoretical model.

It requires listening and reacting.

The X-Touch provides the physical controls.

Why Dub Demonstrates the Point So Well

The lesson is not really about dub.

The same principles apply to:

- electronic music;
- ambient;
- techno;
- live remixing;
- soundtrack work;
- conventional mixing.

Dub simply makes the idea unusually obvious.

The mixer is not merely a place where levels are set.

It can be a performance environment.

And a physical control surface makes that idea tangible.

Workflow 7 — From Broad Mix to Tiny Detail

Now imagine something happens during that mix.

A percussion sound is too loud.

The broad view is:

```
Drums   Bass   Skank   Organ   Percussion   Vocal
```

But the percussion track contains a Drum Machine.

Navigate deeper:

```
SELECT Percussion
       ↓
ENTER / appropriate Pad context
       ↓
Shaker   Conga   Rim   Bell   Tambourine   ...
```

Adjust the offending pad.

Then return.

This demonstrates something important.

A Mouse-Lite workflow does not mean remaining at one level of abstraction.

The X-Touch can move from:

```
whole project
```

to:

```
track
```

to:

```
device
```

to:

```
pad or layer
```

and back again.

Workflow 8 — Moving Between Songs or Projects

At the other extreme, suppose you have finished working on the current project.

Touch the Master fader to enter the Master context.

Use the project-navigation controls to move to another project.

The same surface that moments ago was adjusting one drum pad can now operate at the level of:

```
the entire project
```

That change in scale is one of the most remarkable aspects of the DrivenByMoss mapping.

Think in Layers of Attention

A useful way to summarise the whole controller is by thinking about the level at which your attention currently sits.

```
PROJECT
   │
   ▼
GROUP
   │
   ▼
TRACK
   │
   ▼
DEVICE
   │
   ▼
PARAMETER
```

or:

```
TRACK
   │
   ▼
INSTRUMENT
   │
   ▼
LAYER / PAD
```

The X-Touch can move its focus through these levels.

The question becomes:

Where is my attention right now?

Then put the controller there too.

The Surface as a Moving Window

We began this guide with eight channel strips.

They looked fixed.

Now we know better.

Those eight strips can be a window onto:

```
Tracks

Group contents

Layers

Drum pads

Sends

Parameters
```

The V-Pots can represent:

```
Pan
```

```
Sends
```

```
Device parameters
```

```
Track parameters
```

```
Project controls
```

The faders can represent:

```
Track volume
```

```
Other parameters through FLIP
```

```
Automation performances
```

The physical surface stays put.

Its meaning moves.

Modes Are Views, Not Obstacles

When first encountering a controller like the X-Touch, modes can seem like a complication.

Why can't every control simply do one thing?

Because then we would need an enormous controller.

Instead, think:

A mode gives me the view appropriate to the job I am doing.

If you want to mix:

```
Mixer view
```

If you want to edit a device:

```
Device view
```

If you want to browse:

```
Browser view
```

If you want to navigate markers:

```
Marker view
```

Modes are not hiding functionality from you.

They are organising it.

Modifiers Are Temporary Questions

Likewise, modifiers become easier to remember when treated as questions.

Instead of thinking:

```
I must memorise OPTION + this,
SHIFT + that,
CONTROL + something else...
```

think:

Is there a related operation available here?

For example:

```
MARKER
```

works with markers.

Then:

```
OPTION + MARKER
```

provides a related marker operation: create one.

The combination makes sense because it belongs to the task.

Let Repetition Build Muscle Memory

At first, you will still think:

Which button was that?

That is normal.

Then you will think:

Ah yes — OPTION + MARKER.

Eventually, your hand may simply do it.

The progression is:

```
Remember
   ↓
Recognise
   ↓
Repeat
   ↓
Muscle memory
```

That is when a hardware controller begins to disappear from conscious attention.

You stop operating the X-Touch.

You simply use it.

Don't Try to Use Everything

DrivenByMoss exposes a remarkable amount of functionality.

That does not mean every function needs to become part of your workflow.

If you never use a particular mode, that is fine.

If you prefer editing MIDI with the mouse, keep doing it.

If you love using the physical faders but prefer the Bitwig Browser on screen, that is also fine.

The goal is not completeness.

The goal is fluency.

A small set of operations that you can perform without thinking may be more valuable than fifty shortcuts you can barely remember.

Build Your Own Core Workflow

A useful personal core might be:

```
Transport

Track selection

Banking

Volume

Mute / Solo

Sends

Device control

Markers
```

Someone else might prioritise:

```
Transport

Recording

Overdub

Browser

Drum pads

Automation
```

There is no universally correct set.

The X-Touch should adapt to the work rather than forcing the work to adapt to the controller.

Know When to Reach for the Mouse

This may be the most important Mouse-Lite skill of all.

Do not turn a simple graphical operation into a twenty-button hardware puzzle merely because the controller technically permits it.

If the mouse is clearly faster:

use the mouse.

If the X-Touch keeps you closer to the music:

use the X-Touch.

If the keyboard shortcut is better than either:

use the keyboard shortcut.

The objective is not ideological purity.

It is a smoother path between:

```
Intention
    ↓
Action
    ↓
Result
```

Mouse-Free as an Experiment

There is still value in occasionally attempting a genuinely mouse-free session.

Not because it is necessarily the best way to work.

Because it reveals habits.

Try working for fifteen minutes without reaching for the mouse.

Every time you feel the urge, ask:

Can the X-Touch already do this?

Sometimes the answer will be no.

Sometimes the answer will be yes, but awkwardly.

And sometimes you will discover:

Yes — and actually this is better.

Those discoveries are how a personal Mouse-Lite workflow develops.

From Control Surface to Musical Surface

At the beginning of Project XTC, the X-Touch could easily be seen as:

a box containing faders and buttons that remotely control Bitwig.

That description is technically true.

But it misses something.

Once the surface becomes familiar, it can participate in the musical process.

You can:

```
navigate by song structure;
```

```
reach into a Group;
```

```
balance a Drum Machine;
```

```
shape a device;
```

```
perform automation;
```

```
record a part;
```

```
build a clip;
```

```
ride a mix;
```

```
throw a vocal into delay.
```

Those are not merely software commands.

They are actions within a musical workflow.

The Important Idea

A Mouse-Lite workflow does not begin by banning the mouse.

It begins by noticing where the X-Touch offers a more immediate relationship with the music.

The aim is not:

```
NO MOUSE
```

It is:

```
Hear intention
      ↓
Choose the most natural control
      ↓
Act
      ↓
Keep listening
```

Sometimes that control will be the mouse.

Sometimes the keyboard.

Very often, it can be the X-Touch.

And the more familiar the surface becomes, the less often you need to stop and think about the controller itself.

The modes stop feeling like modes.

The modifiers stop feeling like shortcuts.

The eight channel strips stop feeling like only eight channels.

They become a moving window onto the project.

And eventually the question changes from:

What can this button do?

to:

What do I want to do?

That is the point at which the X-Touch stops merely controlling the DAW.

It becomes part of the way you work.

End of Part IV

We have now reached the end of the main workflow section of Project XTC.

The final part turns from using the system to shaping it.

DrivenByMoss provides configuration choices that determine how the X-Touch behaves, and some of the more advanced functions only make sense once those choices are understood.

Next:

Part V — Configuration and Reference

Chapter 21 — Configuring DrivenByMoss for the X-Touch

Chapter 21 - Configuring DrivenByMoss for the X-Touch

Configuring DrivenByMoss for the X-Touch

For most of this guide, we have concentrated on the X-Touch itself.

Press this button.

Turn this V-Pot.

Move this fader.

Enter this mode.

But there is another part of the system that we have deliberately kept in the background:

DrivenByMoss configuration.

The X-Touch does not communicate with Bitwig in isolation.

The complete system is:

```
X-Touch
   │
   ▼
Mackie Control protocol
   │
   ▼
DrivenByMoss
   │
   ▼
Bitwig Studio
```

DrivenByMoss sits between the hardware and the DAW.

Its configuration therefore determines some important aspects of how the surface behaves.

Configuration Is Part of the Instrument

It is tempting to think of configuration as something separate from actually making music.

Open Settings.

Choose some options.

Close Settings.

Forget about them.

But configuration can have a direct effect on the musical workflow.

Consider New Clip Length.

If you regularly create four-bar clips, configuring that behaviour in advance removes a decision from the recording process.

Instead of:

```
Create clip
    ↓
Choose length
    ↓
Confirm
    ↓
Record
```

you can move towards:

```
Create
   ↓
Record
```

The configuration has become part of the workflow.

A useful principle is:

Configure beforehand so that you have less to configure while making music.

The Configuration Page

DrivenByMoss settings are accessed from Bitwig's controller configuration.

The precise appearance may vary slightly between Bitwig and DrivenByMoss versions, but the basic idea is the same.

The controller entry represents the connection between:

```
Bitwig
   ↕
DrivenByMoss
   ↕
X-Touch
```

This is where the controller's MIDI ports and behaviour are configured.

Choose the Correct Controller

The X-Touch should be configured using the appropriate Mackie MCU controller implementation supplied by DrivenByMoss.

This matters because the X-Touch can operate in several hardware protocols.

Project XTC assumes that the unit is operating in:

MC — Mackie Control mode.

If the X-Touch is operating in another protocol, the behaviour described throughout this guide should not be expected to match.

Confirm MC Mode on the X-Touch

The X-Touch's operating mode is selected on the hardware.

For use with this guide, confirm:

```
Mode
  ↓
MC
```

rather than one of the alternative controller protocols.

Once selected and confirmed, the X-Touch communicates using the Mackie Control protocol expected by DrivenByMoss.

This is the foundation on which everything else depends.

MIDI Ports

The X-Touch exposes MIDI ports to the computer.

On a typical system these may include names such as:

`X-Touch INT`

`X-Touch EXT`

The INT connection is the one associated with the main internal X-Touch control surface.

The EXT connection exists for expansion-related communication.

For a single X-Touch used as the main controller, make sure the DrivenByMoss controller entry is connected to the appropriate X-Touch MIDI input and output.

The important relationship is:

```
X-Touch MIDI OUT
        │
        ▼
DrivenByMoss Input


DrivenByMoss Output
        │
        ▼
```

```
X-Touch MIDI IN
```

Communication must work in both directions.

Why Both Directions Matter

The X-Touch is not merely sending commands to Bitwig.

Bitwig is also sending information back.

For example:

```
Move fader
    ↓
Bitwig parameter changes
```

but also:

```
Bitwig parameter changes
        ↓
Motor fader moves
```

Likewise, DrivenByMoss sends information for:

- displays;
- LEDs;
- V-Pot rings;
- motor faders.

So an apparently half-working controller can be a useful diagnostic clue.

If Bitwig responds to the X-Touch but the X-Touch does not update correctly, check the return path.

The control surface needs a conversation, not a monologue.

A Simple Connection Test

After configuring the ports, perform a few basic tests.

Test 1 — Fader to Bitwig

Move a channel fader.

Does the corresponding Bitwig value move?

Test 2 — Bitwig to Fader

Change that value in Bitwig.

Does the physical fader move?

Test 3 — Track Selection

Press SELECT.

Does Bitwig select the expected track?

Test 4 — Display Feedback

Does the X-Touch display meaningful track or parameter information?

Test 5 — Transport

Press PLAY and STOP.

Does Bitwig respond correctly?

If all five work, the basic two-way connection is in good shape.

Don't Troubleshoot Advanced Features First

If Device Mode or Browser Mode appears not to work, it can be tempting to start changing advanced settings.

Before doing that, verify the basics.

A useful troubleshooting order is:

```
Hardware mode
     ↓
MIDI ports
     ↓
Basic transport
     ↓
Track control
     ↓
Feedback
     ↓
Advanced modes
```

There is little value debugging an advanced feature while the underlying controller connection is incorrect.

DrivenByMoss Behaviour Settings

Once the basic connection works, the more interesting configuration begins.

DrivenByMoss provides settings that influence how the MCU surface behaves.

Not every user will want the same choices.

That is intentional.

The controller should adapt to the workflow.

Flat or Hierarchical Track Navigation

Chapter 17 introduced two ways of thinking about project tracks:

```
Flat
```

and:

```
Hierarchical
```

The choice affects how Groups are represented and navigated.

Flat Navigation

A flatter approach prioritises moving through tracks as a sequence.

Conceptually:

```
Track 1
Track 2
Track 3
Track 4
Track 5
...
```

This can be convenient if you mostly want rapid access to individual tracks regardless of Group structure.

Hierarchical Navigation

A hierarchical approach preserves the structure of Groups.

Conceptually:

```
Project
   │
   ├── Drums
   │     ├── Kick
   │     ├── Snare
   │     └── Hats
   │
   ├── Bass
   │
   └── Vocals
```

This allows the controller's view to move into and out of Groups.

Neither approach is inherently better.

They answer different needs.

Choosing the Navigation Style

Ask how you think about large projects.

If your mental model is:

Give me a long mixer and let me move along it.

a flatter approach may feel natural.

If your mental model is:

Let me see the broad structure and then drill into what I need.

hierarchical navigation may be more useful.

The second approach fits particularly well with the workflow developed in Chapters 17 and 20:

```
Project
   ↓
Group
   ↓
Track
   ↓
work
   ↓
back out
```

But the correct choice is the one that matches the way you actually work.

New Clip Length

Chapter 19 introduced New Clip Length.

This determines the length used when the controller creates a new Launcher clip.

For example:

```
1 bar
```

```
2 bars
```

```
4 bars
```

```
8 bars
```

The useful setting depends on the music.

If you frequently build four-bar patterns, a four-bar default may reduce unnecessary interaction.

If you normally work with longer phrases, another value may make more sense.

This is a good example of configuration serving muscle memory.

Once the setting matches your normal workflow, you stop thinking about it.

Configure for the Common Case

A useful general principle is:

Set defaults for what you do most often, not for every theoretical possibility.

Suppose:

```
70% of new clips → 4 bars
20%              → 8 bars
10%              → something else
```

A four-bar default is probably sensible.

The fact that some clips require another length does not make the default wrong.

A good default reduces work most of the time.

V-Pot Behaviour

Rotary encoders can behave differently depending on the type of parameter and the controller configuration.

DrivenByMoss translates the MCU encoder messages into Bitwig parameter changes and returns feedback to the V-Pot rings and displays.

The practical goal is simple:

```
Turn
  ↓
Predictable change
  ↓
Useful visual feedback
```

If V-Pots seem excessively fast, slow or otherwise unintuitive, check the available DrivenByMoss configuration before assuming that the hardware itself is at fault.

Touch Sensitivity and Motor Faders

The X-Touch's touch-sensitive motor faders are central to:

- automation;
- parameter feedback;
- FLIP;
- track volume control.

They depend on correct two-way communication.

A useful diagnostic distinction is:

Fader sends values but does not move

Check the controller's output path back to the X-Touch.

Fader moves but touch behaviour is unexpected

Check:

- the current automation mode;
- the current controller context;
- relevant DrivenByMoss behaviour.

Do not immediately assume a mechanical fault.

Context matters.

Display Configuration

The scribble strips are not decoration.

As Chapter 7 established, they are part of the control system.

They tell you what the physical controls currently represent.

A useful configuration should therefore favour readable, meaningful feedback.

Remember the relationship:

```
Mode changes
     ↓
Assignments change
     ↓
Displays change
     ↓
You know what the controls mean
```

Without the last step, a deeply contextual control surface becomes much harder to use.

Parameter Names and Values

When Device Mode exposes parameters, the X-Touch has limited display space.

DrivenByMoss must compress information that Bitwig may normally display in a much larger graphical interface.

So do not expect the scribble strip to reproduce the entire Bitwig parameter description.

The aim is:

enough information to identify the control confidently.

That is another reason familiarity matters.

A short parameter label that initially looks cryptic may become immediately recognisable after repeated use.

Configuration for Particular Workflows

Rather than asking which settings are "best", it is often more useful to ask:

Best for what?

Configuration for Conventional Mixing

If the X-Touch is primarily being used as a mixer, priorities may include:

- predictable track banking;
- clear track names;
- immediate fader control;
- panorama access;
- Sends;
- Mute and Solo;
- automation.

The desired experience is:

```
Select
   ↓
Mix
   ↓
Bank
   ↓
Continue mixing
```

Complex hierarchical navigation may be less important if the project structure is relatively simple.

Configuration for Large Projects

For projects containing many Groups, hierarchical navigation becomes more attractive.

For example:

```
Drums
Bass
Guitars
Keys
Vocals
FX
```

can provide a compact top-level view.

Then:

```
SELECT Drums
      ↓
ENTER
      ↓
Kick
Snare
Hats
Toms
Percussion
```

gives access to detail only when needed.

The configuration is helping the eight channel strips scale to a much larger project.

Configuration for Launcher-Based Work

For Launcher-oriented composition, priorities may include:

- New Clip Length;
- overdub behaviour;
- clip creation;
- navigation;
- Clip Based Looper options.

The goal is to reduce interruptions to the loop-building cycle:

```
Create
   ↓
Record
   ↓
Loop
   ↓
Overdub
   ↓
Listen
```

A well-chosen default clip length can be more valuable here than an advanced option that is rarely touched.

Configuration for Device-Heavy Work

If much of your work involves instruments and effects, Device Mode becomes especially important.

Priorities include:

- useful parameter feedback;
- predictable page navigation;
- easy device selection;
- effective V-Pot behaviour;
- FLIP where appropriate.

The desired path is:

```
Track
   ↓
Device
   ↓
Parameter Page
   ↓
Parameter
```

with as little unnecessary navigation as possible.

Configuration for Dub and Performance Mixing

Our Chapter 20 example suggests another set of priorities.

For performance-oriented mixing, useful behaviour includes:

- immediate fader access;
- fast Send selection;
- reliable Mute state;
- clear feedback;
- responsive automation;
- predictable banking.

The workflow might move rapidly between:

```
Faders
   ↓
Mutes
   ↓
Send 1 — Reverb
   ↓
Send 2 — Delay
   ↓
Faders
```

Here, predictability matters more than novelty.

When performing a mix, you do not want to stop and wonder what the surface is going to do.

The DrivenByMoss Settings Are Not a Test

There is a temptation with sophisticated software to assume that there must be one expert configuration.

There isn't.

A setting is not more advanced merely because it is more complicated.

The best configuration is the one that makes the controller behave predictably for your work.

That may be surprisingly simple.

Change One Thing at a Time

When experimenting with configuration, avoid changing many settings simultaneously.

Use:

```
Change one setting
      ↓
Test
      ↓
Understand the effect
      ↓
Keep or revert
```

rather than:

```
Change six settings
      ↓
Something is different
      ↓
Which setting did that?
```

This is particularly important with a contextual controller where one configuration choice may affect several workflows.

Test Changes Musically

A setting can look sensible in a preferences panel and still feel awkward in practice.

After changing something, use the controller for a real task.

If you change track navigation, navigate a real project.

If you change clip behaviour, create some clips.

If you change an encoder-related option, adjust real parameters.

The question is not:

Does the setting work?

It is:

Does the setting improve the workflow?

Keep a Known-Good Configuration

Once the X-Touch is behaving reliably, it is worth knowing what that working configuration looks like.

If a future update changes behaviour, a known-good setup gives you something to compare against.

Useful things to record include:

```
Bitwig version

DrivenByMoss version

X-Touch firmware version

X-Touch operating mode

MIDI port assignments

important DrivenByMoss options
```

This turns:

Something has changed.

into the much more useful:

What changed between these two known configurations?

Version Matters

DrivenByMoss continues to develop.

Bitwig continues to develop.

Behaviour may therefore change after this guide is published.

Project XTC should always state the versions against which its instructions were verified.

That does not mean the guide instantly becomes useless when a new version appears.

It means the reader knows the reference point.

For example:

```
Verified with:

Bitwig Studio 6.1 beta 4
DrivenByMoss 26.3.3
X-Touch firmware 1.22
```

A later version can then be compared with something concrete.

Updates Can Add Functionality

This guide itself grew because we discovered functionality that had not yet been represented adequately.

Features such as:

```
OPTION + MARKER
```

were easy to miss if we looked only at the most obvious controls.

DrivenByMoss development may introduce further capabilities.

So configuration and documentation should both be treated as living things.

The controller you know today may be able to do more tomorrow.

After an Update

After updating Bitwig or DrivenByMoss, do not immediately assume that every existing workflow remains identical.

Perform a quick sanity check.

For example:

```
Transport
   ✓

Faders
   ✓

SELECT
   ✓

Banking
   ✓

Displays
   ✓

Device Mode
   ✓

Sends
   ✓
```

If those fundamentals behave normally, continue to the more specialised functions you use.

This takes very little time and can prevent considerable confusion later.

Troubleshooting by Layer

Remember the complete system:

```
X-Touch Hardware
       │
       ▼
MC Protocol
       │
```

```
      ▼
MIDI Connection
      │
      ▼
DrivenByMoss
      │
      ▼
Bitwig
```

When something fails, work through those layers.

Is the X-Touch in MC mode?

If not, fix that first.

Are the MIDI ports correct?

If not, DrivenByMoss cannot communicate properly.

Does basic transport work?

If not, the problem is probably more fundamental than Device Mode.

Does feedback return to the X-Touch?

If commands work but displays or motors do not, investigate the return path.

Does only one advanced feature fail?

Now look at the relevant mode or configuration.

Troubleshooting from the bottom upwards is much more efficient than randomly changing settings.

Don't Forget the Obvious Things

Control-surface problems can sometimes have wonderfully uninteresting causes.

Before investigating obscure protocol behaviour, check:

- the X-Touch is powered on;
- the correct mode is selected;
- the correct MIDI ports are assigned;
- the controller entry is active;
- the expected track or mode is selected;
- the current project actually contains the object you are trying to control.

The more sophisticated the system becomes, the easier it is to overlook something simple.

A Sensible Starting Configuration

Project XTC is not going to prescribe one mandatory configuration.

But for someone following the workflows in this guide, a sensible starting point is:

```
X-Touch Mode
   → MC

MIDI
   → X-Touch internal ports

Track Navigation
   → choose deliberately:
      flat or hierarchical

New Clip Length
   → set to your most common loop length

Feedback
   → verify displays, LEDs,
      V-Pots and motor faders

Advanced options
   → leave at known defaults
      until you have a reason
      to change them
```

Then use the controller.

Let actual workflow problems tell you which configuration deserves attention.

Configuration Should Eventually Disappear

This may sound strange in a chapter about configuration, but the goal of good configuration is to stop thinking about configuration.

When the system is set up well:

```
Turn on X-Touch
      ↓
Open Bitwig
      ↓
Work
```

You should not need to reconsider the controller architecture every session.

The configuration has done its job when it becomes invisible.

The Important Idea

DrivenByMoss configuration determines the environment in which all the controls described in this guide operate.

The aim is not to discover the most complicated setup.

It is to create a predictable one.

Configure:

- the correct MCU mode;
- the correct MIDI ports;
- the navigation style that suits your projects;
- useful defaults such as New Clip Length;
- the behaviour required by your normal workflow.

Then verify the two-way conversation:

```
You
  ↓
X-Touch
  ↓
DrivenByMoss
  ↓
Bitwig

and

Bitwig
  ↓
DrivenByMoss
  ↓
X-Touch
  ↓
You
```

When that loop works reliably, the technology begins to recede.

And the controller becomes what we wanted from the beginning:

a predictable physical route into the project.

Coming Next

We now have a working controller, a mental model for understanding it, and a configuration that supports the way we want to work.

But DrivenByMoss goes further.

There are specialised options and expansion possibilities that not every user will need immediately.

The final chapter looks beyond the core workflow at:

- customisation;
- specialised options;
- Clip Based Looper;
- expansion;
- extending the surface;

• adapting the system as DrivenByMoss evolves.

Next:

Customisation and Expansion.

Project XTC

X-Touch Companion

The Unofficial Guide to the Behringer X-Touch, Bitwig Studio and DrivenByMoss

Chapter 22 - Customisation and Expansion

Customisation and Expansion

We began Project XTC with a fixed piece of hardware.

Eight channel strips.

Eight V-Pots.

Motor faders.

Transport controls.

Mode buttons.

A Jog Wheel.

It would be easy to assume that this defines the limits of the system.

But DrivenByMoss provides another layer:

some parts of the X-Touch can be adapted to the way you work.

And if one X-Touch is not enough, the Mackie MCU implementation can also support additional control surfaces.

So this final chapter is about two related ideas:

```
Customisation
     │
     ▼
Make the existing surface suit you


Expansion
     │
     ▼
Make the physical surface larger
```

Neither is required.

A single X-Touch with its normal mappings is already a very capable controller.

But once the basic workflow is familiar, these options allow the system to become more personal.

Customise After You Understand

There is a temptation to customise a new controller immediately.

Change the function buttons.

Reassign the footswitches.

Modify preferences.

Create the perfect setup before making any music.

That is usually backwards.

A better progression is:

```
Use the standard workflow
        ↓
Notice repeated friction
        ↓
Identify the missing action
        ↓
Customise deliberately
```

In other words:

Solve problems you actually have.

Do not solve theoretical problems merely because a configuration option exists.

The Value of an Unused Button

A programmable button is valuable because it can shorten a repeated workflow.

Suppose you perform an operation twenty times during a session.

If that operation normally requires:

```
open menu
   ↓
find command
   ↓
click
```

but can instead become:

```
press button
```

that assignment may be worth making.

The best custom controls often feel surprisingly mundane.

They remove tiny interruptions.

And tiny interruptions repeated many times become significant.

Function Buttons

The X-Touch provides a row of function buttons labelled:

```
F1  F2  F3  F4  F5  F6  F7  F8
```

DrivenByMoss allows function-button behaviour to be assigned through its settings.

This makes the F-buttons a useful area for personal workflow commands.

Instead of asking:

What are F1–F8 supposed to do?

a better question is:

What would I benefit from having one button away?

Choose Functions by Frequency

A useful assignment is normally something that is:

- performed frequently;
- slightly awkward to reach otherwise;
- unambiguous;
- safe to trigger accidentally;
- easy to remember.

Suppose you repeatedly use a particular Bitwig operation.

Giving it an F-button may turn:

```
intention
   ↓
search
   ↓
mouse
   ↓
command
```

into:

```
intention
   ↓
F-button
```

That is exactly the kind of compression a control surface is good at.

Don't Fill Every Button

Eight labelled buttons can create a strange psychological pressure:

I must find eight things to assign.

You don't.

An unused button is not wasted.

A button assigned to something you never remember is more useless than an intentionally empty one.

Start with one or two functions that genuinely improve the workflow.

Then let experience tell you whether more are needed.

A Documentation Detail to Verify

There is currently a small discrepancy in the DrivenByMoss MCU documentation.

The general Functions section describes:

```
F1–F8
```

as assignable function buttons.

The Preferences section specifically lists configurable assignments for:

```
F1–F5
```

Project XTC should therefore not claim that all eight buttons are independently assignable in the current configuration interface until this has been verified against the exact DrivenByMoss version covered by the final guide.

This is a useful example of an important documentation principle:

When two sources inside the same reference disagree, test rather than guess.

The Quick Reference should contain only the verified behaviour.

Actions

One of the assignable function types in DrivenByMoss is an Action.

This allows an available Bitwig action to be selected and triggered from the controller.

Conceptually:

```
Physical Button
       │
       ▼
DrivenByMoss
       │
       ▼
Selected Bitwig Action
```

This opens the door to a much more personal controller layout.

Actions Turn Workflow into Hardware

Suppose Bitwig provides an action that you use constantly.

Without customisation:

```
remember shortcut
       or
find command
```

With an assigned controller button:

```
press
```

The software command has acquired a physical location.

And once an action has a physical location, muscle memory can develop around it.

This is one of the subtle advantages of programmable hardware.

Pick Actions You Can Explain in One Sentence

A good custom assignment should normally have an obvious purpose.

For example:

This button does X.

If you need a paragraph to remember why you assigned it, the mapping may be too clever.

Simple mappings survive.

Complicated mappings get forgotten.

A useful test is:

If I return to this project after a month, will I still know what this button does?

If not, either simplify the assignment or document it.

Label Your Customisations

The X-Touch's printed F-button labels obviously cannot change.

If you create important custom assignments, keep a record.

For example:

```
F1   → [your chosen action]
F2   → [your chosen action]
F3   → unused
F4   → [your chosen action]
```

This record could live:

- in your Project XTC notes;
- beside the controller;
- in a small text file;
- in the project documentation.

The important thing is that a personal mapping should not become a personal mystery.

Footswitches

DrivenByMoss exposes two MCU footswitch functions:

```
Footswitch 1 — USER A
```

```
Footswitch 2 — USER B
```

Their functions can be selected in the DrivenByMoss settings.

This adds something fundamentally different from another button on the control surface.

A footswitch can be operated while both hands are busy.

Why Feet Matter

Consider a recording situation.

Your left hand may be on an instrument.

Your right hand may also be on the instrument.

Reaching for the X-Touch interrupts the performance.

A footswitch gives us another input channel:

```
Hands
  │
  └── perform music
```

```
Foot
  │
  └── control recording function
```

That separation can be extremely useful.

Good Footswitch Jobs

A footswitch is particularly suitable for actions that are:

- performance-related;
- time-sensitive;
- simple;
- safe to trigger without looking.

The best assignments depend entirely on the workflow.

For one person, a footswitch may belong to recording.

For another, it may trigger a transport or navigation function.

For someone else, it may never be needed.

Again:

Customisation follows the work.

Clip Based Looper

One particularly interesting assignable function is Clip Based Looper.

This connects directly with the recording concepts from Chapter 19.

DrivenByMoss uses the currently selected MIDI clip slot.

If that slot is empty, a new clip is created using the configured New Clip Length, and playback begins.

The footswitch then controls overdub according to whether it is held.

Conceptually:

```
Selected Clip Slot
        │
        ▼
Is there a clip?
   │           │
   │ no        │ yes
   ▼           ▼
Create Clip    Use Clip
   │           │
   └─────┬─────┘
         ▼
      Playback
         │
         ▼
Hold Footswitch
         │
         ▼
      Overdub
         │
         ▼
Release Footswitch
         │
         ▼
Stop Overdubbing
```

That is considerably more than a simple button assignment.

It creates a performance workflow.

Hands-Free Loop Building

Imagine playing a MIDI keyboard.

Both hands are occupied.

The basic loop exists, and you want to add another layer.

With an appropriately assigned footswitch:

```
Play
  ↓
Hold Footswitch
  ↓
Overdub
  ↓
Release
  ↓
Listen
```

The hands remain on the instrument.

The recording state is controlled by the foot.

This can feel much more natural than reaching away from the keyboard every time the loop changes state.

New Clip Length Matters Here

Clip Based Looper also demonstrates why the configuration choices from Chapter 21 matter.

If the selected slot is empty, the new clip uses the configured New Clip Length.

So:

```
Configuration
      │
      ▼
New Clip Length
      │
      ▼
Clip Based Looper
      │
      ▼
Performance behaviour
```

A setting made before the session affects what happens during the session.

This is exactly what we meant by:

Configure beforehand so that you have less to configure while making music.

A Simple Looper Workflow

Suppose New Clip Length is set to four bars.

A workflow might be:

1. Select the destination track and slot
Establish where the loop belongs.

2. Begin the Clip Based Looper operation
If necessary, the new four-bar clip is created.

3. Play the foundation
Record the basic musical idea.

4. Let it loop
Listen.

5. Hold the footswitch
Overdub becomes active.

6. Add another part
Continue playing.

7. Release the footswitch
Overdub stops.

8. Listen again
Decide whether the clip needs another layer.

This is the kind of workflow where hardware customisation becomes genuinely musical.

Customisation Can Reduce Mode Switching

A useful custom control can sometimes avoid a journey through several other controls.

Without an assignment:

```
leave current task
      ↓
find required function
      ↓
perform it
      ↓
return
```

With an assignment:

```
press
```

This does not mean that modes are bad.

Modes are what make the X-Touch powerful.

But if one operation repeatedly forces you out of the mode where you actually want to work, a custom button may provide a useful shortcut.

Expansion

Customisation changes what the existing hardware does.

Expansion changes how much physical hardware is available.

The Mackie MCU model was designed to support additional channel strips.

DrivenByMoss therefore supports multi-device configurations.

For the X-Touch family, the obvious companion is the X-Touch Extender.

Why Add an Extender?

A single X-Touch gives us eight channel strips.

That has shaped much of this guide:

```
Tracks 1–8
     ↓
BANK
     ↓
Tracks 9–16
```

An Extender gives us more physical strips simultaneously.

Conceptually:

```
X-Touch

1   2   3   4   5   6   7   8


+

Extender

9  10  11  12  13  14  15  16
```

Now sixteen channels can be available without banking.

Expansion Does Not Change the Mental Model

This is important.

An Extender does not invalidate everything we learned about banking and context.

It merely makes the visible window wider.

With one X-Touch:

```
Project
────────────────────────────────────────

        ┌────────────────┐
        │   8 channels   │
        └────────────────┘
```

With an additional eight-channel surface:

```
Project
────────────────────────────────────────

        ┌────────────────────────────────┐
        │          16 channels           │
        └────────────────────────────────┘
```

The idea remains:

The physical surface is a window onto the project.

The window is simply larger.

Main and Extender Roles

DrivenByMoss distinguishes between controller roles.

A device can be configured as:

- Main
- Extender
- MCU Extender

The Main device provides the full main-controller role, including controls such as the Master fader and transport functionality.

An Extender primarily contributes additional channel control.

Conceptually:

```
MAIN
  │
  ├── Channel strips
  ├── Master
  ├── Transport
  └── Main commands


EXTENDER
  │
  └── Additional channel strips
```

This division reflects the physical design of MCU-style systems.

MCU Extender

DrivenByMoss also distinguishes an MCU Extender role for devices that use the original Mackie MCU Extender protocol.

This matters because not every additional controller communicates in exactly the same way as the main MCU unit.

For a straightforward X-Touch plus X-Touch Extender setup, choose the role appropriate to the actual hardware and connection rather than assuming that every expansion device should be configured identically.

Multiple Main Devices

DrivenByMoss also supports multiple devices configured as Main.

That is a more specialised configuration.

For most Project XTC readers, the simpler mental model is sufficient:

```
Main X-Touch
     +
additional channel surface
```

More elaborate multi-device layouts should be configured only when there is a clear reason for them.

Restart After Extender Changes

Changes to the Extender Setup require the DrivenByMoss extension to be restarted before the new configuration takes effect.

This is worth remembering because otherwise the settings may appear not to have worked.

A sensible workflow is:

```
Change Extender Setup
        ↓
Restart extension
        ↓
Test banking and channel order
```

Do not repeatedly alter settings simply because the old state remains active before the required restart.

What Does More Hardware Actually Buy Us?

It is easy to think:

Sixteen faders must be twice as good as eight.

Not necessarily.

More physical channels offer some clear advantages.

Less Banking

The obvious advantage is fewer bank changes.

For a sixteen-track project:

One X-Touch

```
Tracks 1–8
    ↓ BANK
Tracks 9–16
```

X-Touch + eight-channel Extender

```
Tracks 1–16
simultaneously
```

That can make broad mixing considerably more immediate.

More Simultaneous Faders

Two hands can move several faders.

With more physical strips visible at once, more of the mix remains physically accessible.

This can be especially useful for:

- live mixing;
- automation passes;
- balancing Groups;
- performance-oriented work.

More Immediate Mute and Solo

The advantage is not only faders.

Additional strips also mean more simultaneously available:

- MUTE buttons;
- SOLO buttons;
- SELECT buttons;
- ARM buttons;
- V-Pots;
- display information.

The physical representation of the mixer becomes broader.

But More Hardware Also Means More Space

An Extender costs:

- money;
- desk space;
- another connection;
- more configuration;
- more physical reach.

The correct question is not:

Would sixteen faders be nice?

Of course they would.

The better question is:

Does banking interrupt my workflow often enough that more physical channels are worth the cost and space?

That is a much more useful decision.

Expansion and the Dub Mixing Desk

Our Chapter 20 dub workflow gives an obvious example.

With eight strips:

```
Drums
Bass
Skank
Organ
Percussion
Vocal
FX 1
FX 2
```

already fits rather neatly onto one X-Touch.

But a larger performance mix might contain:

```
Drums
Bass
Skank
Organ
Piano
Percussion
Vocal 1
Vocal 2
FX Return 1
FX Return 2
FX Return 3
FX Return 4
```

...

An Extender could keep considerably more of that surface physically available at once.

For a performance-style mix, that may genuinely change how the controller feels.

You bank less.

You react more.

More Controls Are Not Automatically Better

There is a useful parallel with custom buttons.

More functionality is only useful when it remains understandable.

A single well-learned X-Touch may be more effective than a huge surface whose layout you have not internalised.

Likewise:

```
4 useful custom assignments
```

may be better than:

```
20 assignments you cannot remember
```

The goal is not maximum capability.

It is maximum fluency.

Build a Personal Layer on Top of the Standard One

Project XTC has spent most of its time describing a common baseline.

That matters.

If every user begins with a completely different control map, documentation becomes impossible and troubleshooting becomes much harder.

So think of customisation as a second layer:

```
DrivenByMoss standard mapping
             │
             ▼
     Learn the system
             │
             ▼
   Personal customisation
```

Do not erase the standard mental model.

Build on it.

Keep the Obvious Things Obvious

If a button labelled PLAY plays, that is easy to remember.

If a custom button labelled F2 performs some personal action, that is also manageable.

Problems arise when familiar controls are remapped so aggressively that the surface no longer makes visual sense.

The printed legends are useful.

Use them where possible.

Save custom assignments for places intended to be custom.

Document Your Personal Setup

Once a controller becomes customised, your own documentation becomes valuable.

A simple file might record:

```
Project XTC Personal Configuration

F1  → ...
F2  → ...
F3  → ...

Footswitch 1 → ...
Footswitch 2 → ...

New Clip Length → ...

Track Navigation → Hierarchical
```

If the system ever needs to be recreated, this turns a reconstruction exercise into a checklist.

Customisation and Updates

DrivenByMoss evolves.

Bitwig evolves.

Available actions or configuration options may change.

After a major update, verify custom assignments just as you would verify normal controller behaviour.

A simple test might be:

```
F-buttons
   ✓
Footswitches
```

```
   ✓

Clip Based Looper
   ✓

Extender
   ✓
```

If something has changed, consult the current DrivenByMoss documentation before rebuilding the setup from scratch.

Leave Room for Discovery

One reason Project XTC grew from its original form was that the X-Touch could do more than we had initially realised.

An apparently obscure command such as:

```
OPTION + MARKER
```

led us to audit the complete DrivenByMoss MCU feature set.

That audit uncovered:

- Marker Mode;
- deeper modifier functions;
- Mixer Edit Modes;
- Automation;
- layers and drum pads;
- Master Mode;
- advanced recording;
- assignable controls;
- Extenders.

There will probably be further discoveries.

That is not a flaw.

It is what happens when a flexible controller meets actively developed software.

Don't Customise Away the Possibility of Learning

There is one subtle risk with customisation.

If we immediately assign a custom button to solve every unfamiliar task, we may never learn the standard DrivenByMoss workflow that already solves it.

So when you encounter friction:

```
Problem
   ↓
Is there already a normal X-Touch method?
   │
```

```
├── yes → learn it
│
└── no / awkward
          ↓
     customise
```

This preserves the common vocabulary of the controller while still allowing personal refinement.

A Sensible Customisation Process

A disciplined approach might be:

1. Use the normal setup

Work with it long enough to identify genuine friction.

2. Write the problem down

For example:

I repeatedly need to perform X and the current route interrupts me.

3. Look for an existing X-Touch workflow

There may already be one.

4. If necessary, choose a custom control

Use an F-button, footswitch or assignable Action.

5. Test it in a real session

Does it actually help?

6. Keep or remove it

Do not preserve a customisation merely because you spent time creating it.

7. Document anything important

Future you will be grateful.

Your X-Touch Does Not Have to Be My X-Touch

This guide needs a common baseline so that instructions remain meaningful.

But beyond that baseline, two users may reasonably develop different surfaces.

One might prioritise:

`Automation`

`Mixing`

```
Device control
```

Another:

```
Launcher recording

Clip Based Looper

Footswitch control
```

Another:

```
Large mixer

Extender

Many simultaneous faders
```

Another may change almost nothing.

All of those are valid.

The controller serves the workflow.

Not the other way around.

From Fixed Hardware to Adaptable Surface

When we first looked at the X-Touch, the hardware appeared fixed:

```
buttons
faders
knobs
displays
```

Now we can see several dimensions of adaptability.

```
Modes
   ↓
change what controls mean

Modifiers
   ↓
temporarily extend controls

Configuration
   ↓
change controller behaviour

Custom assignments
   ↓
add personal shortcuts

Extenders
   ↓
increase physical capacity
```

So although the hardware itself is fixed, the surface as experienced by the user is not.

The Important Idea

Customisation is not about making the X-Touch more complicated.

It should do the opposite.

A good customisation removes friction.

A good assignment shortens a repeated path.

A good footswitch keeps your hands on the instrument.

A useful Extender keeps the channels you care about physically available.

The test is simple:

Does this make the work easier to perform and easier to understand?

If yes, keep it.

If not, remove it.

End of the Numbered Chapters

We began with a piece of hardware.

Eight motor faders.

Eight channel strips.

Eight V-Pots.

A collection of buttons whose labels came from the Mackie Control world.

At first, the surface could look almost overwhelming.

But the underlying ideas turned out to be much simpler.

```
BANKS
   → move the window

SELECT
   → establish focus

MODES
   → change the view

MODIFIERS
   → temporarily extend a control

DISPLAYS
   → tell us what the controls mean

MOTOR FADERS
   → communicate in both directions

HIERARCHY
   → move between levels
```

```
CUSTOMISATION
   → adapt the surface to the work
```

Those ideas are more useful than memorising hundreds of commands.

Because when an unfamiliar function appears, we now have a way to understand it.

From Controller to Companion

The title of this project is X-Touch Companion.

That word matters.

A useful companion does not demand constant attention.

It does not make you think about its internal machinery every moment.

It becomes familiar.

Predictable.

Available when needed.

The same should eventually be true of the X-Touch.

At first:

```
Which button?
```

```
Which mode?
```

```
Which modifier?
```

Later:

```
I want the chorus.
```

```
I want the delay.
```

```
I want that fader.
```

```
I want to record this.
```

```
I want the snare quieter.
```

And your hands know what to do.

The Destination

The ultimate aim was never to become an expert in Mackie MCU button combinations.

It was never to avoid the mouse at all costs.

And it was never to use every feature simply because DrivenByMoss provides it.

The aim was to shorten the distance between:

```
musical intention
       │
       ▼
physical action
       │
       ▼
musical result
```

When that distance becomes small enough, the controller itself begins to disappear from conscious attention.

You stop thinking:

I am operating the X-Touch.

You think:

The vocal needs more delay.

And then you do it.

That is where Project XTC has been heading all along.

What Comes After the Chapters?

The numbered teaching chapters are now complete.

But a guide such as this also needs something different:

a place to look things up.

The next section of Project XTC will therefore be the Quick Reference.

It will organise the verified DrivenByMoss/X-Touch commands in two ways:

• by physical control;
• by task.

The teaching journey ends here.

The reference begins next.